

复杂装备系统导航式建模技术

李子航

2023 年 6 月

中图分类号：N945.23

UDC 分类号：621

复杂装备系统导航式建模技术

作者姓名	李子航
学院名称	机械与车辆学院
指导教师	王国新教授
答辩委员会主席	王建群教授
申请学位	工学硕士
学科专业	机械工程
学位授予单位	北京理工大学
论文答辩日期	2023 年 6 月

Navigation-Based Modeling Technique for Complex Equipment System

Candidate Name:	<u>LI Zihang</u>
School or Department:	<u>School of Mechanical Engineering</u>
Faculty Mentor:	<u>Prof. WANG Guoxin</u>
Chair, Thesis Committee:	<u>Prof. WANG Jianqun</u>
Degree Applied:	<u>Master of Science in Engineering</u>
Major:	<u>Mechanical Engineering</u>
Degree by:	<u>Beijing Institute of Technology</u>
The Date of Defence:	<u>June, 2023</u>

复杂装备系统导航式建模技术

李子航

北京理工大学

研究成果声明

本人郑重声明：所提交的学位论文是我本人在指导教师的指导下进行的研究工作获得的研究成果。尽我所知，文中除特别标注和致谢的地方外，学位论文中不包含其他人已经发表或撰写过的研究成果，也不包含为获得北京理工大学或其它教育机构的学位或证书所使用过的材料。与我一同工作的合作者对此研究工作所做的任何贡献均已在学位论文中作了明确的说明并表示了谢意。

特此申明。

签 名： 日期：

关于学位论文使用权的说明

本人完全了解北京理工大学有关保管、使用学位论文的规定，其中包括：①学校有权保管、并向有关部门送交学位论文的原件与复印件；②学校可以采用影印、缩印或其它复制手段复制并保存学位论文；③学校可允许学位论文被查阅或借阅；④学校可以学术交流为目的，复制赠送和交换学位论文；⑤学校可以公布学位论文的全部或部分内容（保密学位论文在解密后遵守此规定）。

签 名： 日期：

导师签名： 日期：

摘要

基于模型的系统工程 (Model-Based Systems Engineering, MBSE) 提供了一种统一的、可共享的、可追溯的系统设计方法, 帮助设计人员更好地理解系统, 并在整个系统生命周期中保持模型的一致性。MBSE 建模方法论是这一领域的产物, 旨在指导设计人员设计出“正确”的系统。然而, 随着 MBSE 应用的广泛和深入, 目前对于方法论的设计和应用存在一定的问题, 表现在以下几个方面: (1) 现有方法论抽象的描述方式无法适应多层次、多领域的复杂装备系统开发场景, 其定制化和可扩展性较弱; (2) 缺乏支持构建系统全局模型的建模语言, 系统模型之间缺乏显式的关联关系, 给系统架构的维护和扩展带来困难; (3) 方法论与建模工具的融合不足, 导致方法论对系统设计的驱动并没有取得理想的效果。

本文针对上述问题, 开展了复杂装备系统导航式建模技术的研究, 主要包括以下四个方面:

(1) 复杂装备系统导航式建模技术。首先介绍了本文的技术基础——KARMA 语言及其建模工具, 并分析了解决上述问题的设计需求。然后, 提出了复杂装备系统导航式建模框架, 以元对象机制和数据建模为总体框架, 为本文提供技术指导。最后, 讨论了本研究的应用。

(2) 面向导航式建模的元模型设计。首先, 利用实体关系图分析了元模型所需覆盖的元素, 保障所用术语的全面和统一。然后, 基于元元模型 GOPRR-E (该元元模型包括图、对象、点、属性、关系、角色和扩展七个要素) 和业务流程标注, 对实体关系图进行扩展, 设计实体的图形化表达, 即元模型, 旨在为方法论的流程建模提供手段, 达到流程驱动设计和串联架构模型的效果。

(3) 复杂装备系统导航式建模语言设计。首先, 研究了 GOPRR-E 元元模型与可扩展标记语言 (Extensible Markup Language, XML) 的映射关系。在此基础上, 设计了导航式建模语言的抽象语法和具体语法, 用 XML 实现了对流程模型的存储和描述。最后, 基于所提建模语言设计了流程模型、XML 模型和文件结构模型的自动转换方法, 提升了设计的自动化水平, 减少了设计人员重复的建模工作。

(4) 复杂装备系统导航式建模系统设计及验证。首先, 设计了支持上述流程模型、XML 模型和文件结构模型构建及转换的建模系统, 并根据实际应用需求开发了

流程模型与设计、分析、仿真和验证所需功能的接口。然后，通过对比分析和案例分析，验证了复杂装备系统导航式建模技术的正确性和工具的可行性。

关键词：基于模型的系统工程；MBSE；建模语言；KARMA 语言；架构设计；模型驱动

Abstract

Model-based systems engineering provides a unified, shareable, and traceable system design approach that helps designers better understand the system and maintain model consistency throughout the system lifecycle. MBSE methodology is the product of this field and aims to guide designers in designing "correct" systems. However, as the application of model-based systems engineering becomes more widespread and profound, there are some issues with the design and application of methodology, including: (1) the existing methodology's abstract descriptive approach cannot adapt to complex system development scenarios with multiple hierarchies and domains. As a result, it has weak customization and scalability; (2) there is a lack of modeling language that supports the modeling of a global model for the system. And there is a lack of explicit association between system models, which makes it difficult to maintain and extend system architecture; (3) the integration of methodology and modeling tool is insufficient, leading to less-than-ideal effects of methodology on system design.

In order to address the above issues, this paper conducts research on navigation-based modeling technique for complex equipment system, mainly including the following four aspects:

(1) Navigation-based modeling technique for complex equipment system. First, the technical foundation of this paper is introduced, which is the KARMA language and its modeling tool. The design requirements for addressing the above issues are analyzed. Then, a navigation-based modeling framework for complex equipment system is proposed, which provides methodological guidance using meta-object facility and data modeling as a technical framework. Finally, the application of this research is discussed.

(2) Meta-model design for navigation-oriented modeling. First, the elements required for the meta-model are analyzed using an entity relationship diagram to ensure comprehensive and unified use of terminology. Then, based on the GOPPRR-E meta-meta model and business process modeling notation, the meta-model is designed. It provides a means of process modeling for methodology, which aims to achieve process-driven design and connect the architecture model.

(3) Navigation-based modeling language for complex equipment system. First, the mapping relationship between the GOPPRR-E meta-meta model and the XML is studied.

Based on this, the abstract syntax and concrete syntax of the navigation-based modeling language are developed, realizing the description of process models at the storage level. Finally, a automatic conversion method for process model, XML model, and file structure model is designed to improve automation levels and reduce modeling work for researchers.

(4) Navigation-based modeling system design and validation. First, a modeling system that supports the above process model, XML model, and file structure model conversion is designed. And the interface between the process model and required functions for design, analysis, simulation, and validation is developed based on actual application requirements. Then, through comparative analysis and case study, the correctness of the navigation-based modeling technique for complex equipment system and the feasibility of the tool are verified.

Key Words: Model-Based Systems Engineering; MBSE; Modeling Language; KARMA; Architecture Design; Model-Driven

目录

第 1 章 绪论	1
1.1 研究背景	1
1.2 国内外研究现状	2
1.2.1 MBSE 发展现状	2
1.2.2 MBSE 方法论	4
1.2.3 现有建模工具中方法论应用	6
1.2.4 存在的主要问题	9
1.3 研究意义	10
1.4 研究内容和组织架构	11
1.5 本章小结	12
第 2 章 复杂装备系统导航式建模技术	13
2.1 引言	13
2.2 技术基础及需求分析	13
2.2.1 技术基础	13
2.2.2 导航式建模需求分析	15
2.3 复杂装备系统导航式建模框架	17
2.3.1 元对象机制	18
2.3.2 数据建模	19
2.3.3 结合元对象机制和数据建模的总体框架	20
2.4 复杂装备系统导航式建模 workflow	22
2.4.1 方法论设计框架	22
2.4.2 面向 V 模型的导航式建模 workflow	23
2.5 本章小结	26
第 3 章 面向导航式建模的元模型设计	27
3.1 引言	27

3.2 元模型设计技术	27
3.2.1 实体-关系图	27
3.2.2 GOPPRR-E 元元模型	28
3.3 概念数据模型设计	30
3.4 逻辑数据模型设计	35
3.4.1 业务流程建模标注	35
3.4.2 基于 GOPPRR-E 的元模型开发	36
3.5 案例验证	41
3.6 本章小结	43
第 4 章 复杂装备系统导航式建模语言设计	44
4.1 引言	44
4.2 元模型与 XML 转换规则设计	44
4.2.1 XML	44
4.2.2 GOPPRR-E 元元模型到 XML 的映射	45
4.3 物理交换规范设计	46
4.3.1 抽象语法	47
4.3.2 具体语法	50
4.4 语义解析及模型转换	53
4.4.1 模型转换需求	54
4.4.2 代码生成器	55
4.4.3 语言解释器	1
4.5 案例验证	1
4.6 本章小结	1
第 5 章 复杂装备系统导航式建模系统设计及验证	1
5.1 引言	1
5.2 导航式建模系统设计	1
5.2.1 系统功能概览	1
5.2.2 技术框架	1

5.3 能力验证	1
5.3.1 定量分析	1
5.3.2 定性分析	1
5.4 应用验证	1
5.4.1 案例背景	1
5.4.2 MOFLP 建模方法论	1
5.4.3 导航式建模驱动火星车设计	1
5.4.4 应用总结	1
5.5 本章小结	1
结论	1
攻读学位期间发表论文与研究成果清单	1
致谢	1

图目录

图 1.1	MBSE 发展概览	2
图 1.2	Rhapsody 表达文件结构	7
图 1.3	Capella 提供建模导航	7
图 1.4	MagicDraw 方法论导航模型	8
图 1.5	基于 SysML 的 OOSEM 设计流程	9
图 1.6	论文组织结构	11
图 2.1	KARMA 语言支持多架构统一建模	14
图 2.2	利用 KARMA 语言执行模型转换方案	15
图 2.3	霍尔三维系统模型	15
图 2.4	MBSE 的三维模型	16
图 2.5	元对象机制	18
图 2.6	数据模型层次	19
图 2.7	结合元对象机制和数据建模的总体框架	20
图 2.8	导航模型的三种形式	22
图 2.9	方法论设计框架	23
图 2.10	导航式建模开发阶段 workflow	24
图 2.11	导航式建模应用阶段 workflow	24
图 2.12	图模型、KARMA 语言和特定域模型的一体三面	25
图 3.1	GOPPRR-E 元元模型	29
图 3.2	基础实体关系图	31
图 3.3	ISO/IEC/IEEE 15288:2015 中过程的结构	32
图 3.4	面向导航式建模的概念数据模型	34
图 3.5	基于 GOPPRR-E 的导航式建模的元模型	41
图 3.6	利用导航式建模构建的 DoDAF 流程模型	42
图 4.1	GOPPRR-E 元元模型与 XML 的映射规则	46
图 4.2	导航式建模语言抽象语法	47
图 4.3	指标验证对象的属性信息（示例）	48
图 4.4	导航式建模的主模型和剖视图模型	55

图 4.5	代码生成器 workflow	55
图 4.6	元模型与文件结构模型的映射规则	56
图 4.7	遍历生成泳道类子节点的概念图	59
图 4.8	语言解释器 workflow	60
图 4.9	KARMA 语言项目的元元模型结构	61
图 4.10	动态注册 workflow	62
图 4.11	阶段控制 workflow	65
图 4.12	DoDAF 流程模型的 XML 文件	66
图 4.13	XML 文件的语法验证	66
图 4.14	流程模型在应用阶段锁定	67
图 4.15	流程模型自动转换生成文件结构	67
图 5.1	导航式建模系统功能架构	69
图 5.2	导航式建模系统技术框架	72
图 5.3	Arcadia	75
图 5.4	MagicGrid	76
图 5.5	OOSEM	77
图 5.6	Harmony-SE	78
图 5.7	“毅力号”火星车	82
图 5.8	MOFLP 建模方法论	83
图 5.9	新建导航式建模文件	86
图 5.10	MOFLP 方法论流程模型	87
图 5.11	通过方法论流程模型创建/访问架构模型的工作流	88
图 5.12	毅力号火星车下降过程状态机图	93
图 5.13	通过方法论流程模型执行行为仿真的 workflow	94
图 5.14	毅力号下降过程高度仿真结果	94
图 5.15	动力系统仿真模型	96
图 5.16	通过方法论流程模型生成 Modelica 模型的工作流	96
图 5.17	通过方法论流程模型执行 Modelica 仿真的 workflow	97
图 5.18	行驶速度的仿真结果	97

表目录

表 2.1	MBSE 的时间维和逻辑维过程矩阵	17
表 2.2	需求分析矩阵举例	17
表 3.1	实体到系统需求的追溯矩阵	34
表 3.2	导航式建模的图元模型	36
表 3.3	导航式建模的对象元模型	37
表 3.4	导航式建模的关系元模型	38
表 3.5	导航式建模的角色元模型	38
表 3.6	导航式建模的属性元模型	39
表 4.1	导航式建模语言关键字	50
表 5.1	导航式建模支持表达的方法论概览	74
表 5.2	与同类系统的对比	79
表 5.3	MOFLP 方法论术语解释	82
表 5.4	“毅力号”火星车架构模型汇总	89
表 5.5	NASA 公布的“毅力号”下降过程数据	92
表 5.6	与基于 KARMA 的设计流程对比	98

第 1 章 绪论

1.1 研究背景

随着技术革命和产业革命的加深，现代各领域的产品系统与装备复杂程度不断升级，研发过程中的不确定性也快速增加，过去单一的“机械”系统正在向“机械、电气、通信”等多领域组成的复杂装备系统发展；“软件定义”概念出现后，任何复杂产品和系统都开始具备智能化和网络化的特性；在移动互联网、物联网、大数据影响下，系统之间的信息交互使得破除了传统的系统边界，多个系统聚合形成体系（System of Systems, SoS）^[1]，这对设计人员在需求分析、设计、权衡、验证和确认等诸多方面提出了挑战^[2]。

基于模型的系统工程（Model-based Systems Engineering, MBSE）是解决上述复杂装备系统设计问题的一种关键手段^[3]，已经在航空航天领域展开了广泛的应用^[4]。MBSE 是一种系统工程方法，通过建立系统模型，实现对复杂产品系统的描述，从概念设计阶段开始支持系统整个生命周期的需求、设计、分析、验证和确认活动^[5]。基于这种模型驱动设计的特性，MBSE 已成为西方工业强国实现数字化转型的重点^[6]。建模语言、建模工具和建模方法论是 MBSE 应用实践的三大支柱^[7]。建模语言是一种建模规范，通过语义和语法规则赋予了系统模型区别于传统 PPT、Visio 图中元素的实际意义，用于开发系统模型；建模工具是建模语言和建模方法论的载体，支持设计人员在工具上进行建模、仿真、验证和分析活动；建模方法论是一种建模指南，是指导设计人员利用建模语言和建模工具开展系统设计、模型管理的基础。

目前，研究人员对建模语言和建模工具及二者的结合做了比较充分的研究，而对于方法论和建模工具的深度融合的研究相对薄弱。在多学科、多系统层级耦合的复杂装备系统研发过程中，设计人员要么结合使用多种成熟的方法论，而这些方法论一般由特定建模工具绑定支持，这将导致数据异构，不利于对系统模型的管理；要么定制设计一套专用方法论，然而由于并不存在专用的定制开发工具，这种定制化的方法论往往隐式存在于设计人员的脑中，强烈依赖于特定人员，不利于模型的维护和扩展。随着模型数量级的增加，上述问题会暴露得更加明显，将严重降低设计效率和可靠性。如何有效解决这类问题，使得方法论能够与系统设计活动深度融合，是本文的研究方

向。

1.2 国内外研究现状

1.2.1 MBSE 发展现状

基于 MBSE 三大支柱的概念，本文从工具、语言、概念（扩展了原术语方法论，包括 MBSE 方法论和一些新出现的概念）和应用方面对 MBSE 的发展进行了梳理。MBSE 发展阶段的定义参考 Haskins 等的研究^[8]，依照 Gartner 提出的技术成熟度曲线来分析 MBSE 技术的生命周期^[9]，该曲线包括五个阶段：1）萌芽期，在早期阶段技术出现并随后迅速扩张；2）泡沫期，技术取得多项成就的同时又面临多项挑战，这一阶段引起了大量研究者、投资人的关注；3）低谷期，技术的应用出现一些问题，难以取得突破；4）复苏期，第二代和第三代技术出现，进入更广泛的应用；5）成熟期，技术成为主流并得到市场认可。

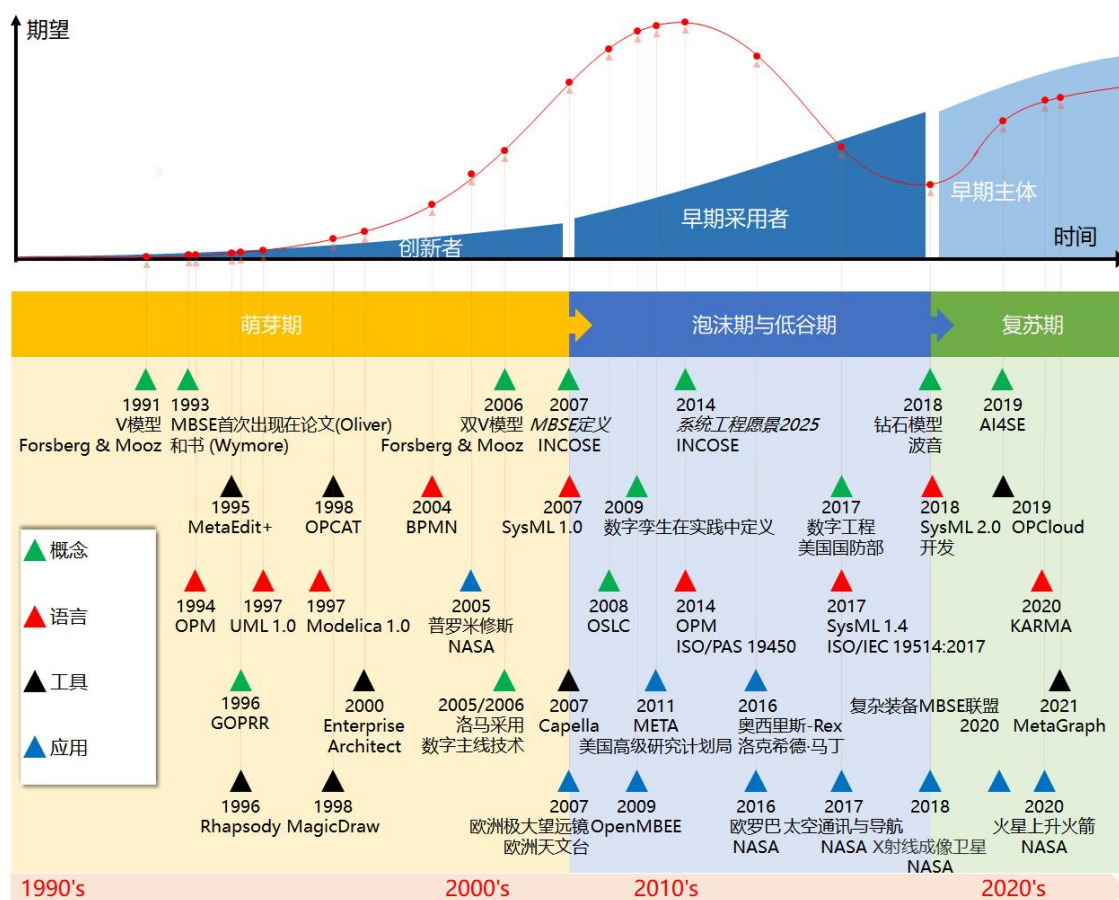


图 1.1 MBSE 发展概览

如图 1.1 所示，坐标系的纵轴代表人们对 MBSE 的期望，横轴代表时间。MBSE

在过去 30 年的发展可以分为三个阶段，这样划分而不是使用原来的五个阶段的原因在于：1) 研究人员在早期就清晰地意识到了 MBSE 挑战和技术不足^[10]。因此，低谷期可能很短，所以本文合并了泡沫期和低谷期；2) MBSE 目前处于开发和应用第二代技术的早期阶段，尚未进入成熟期。因此，本文没有在图中划分出成熟期。

在萌芽期，MBSE 是一个没有引起广泛关注的新概念，只有部分研究人员开始探索 MBSE，这些人为创新者。1993 年，MBSE 的概念出现于 Oliver D. 发表在国际系统工程协会国际研讨会（International Council on Systems Engineering International Symposium, INCOSE IS）的论文中^[11]，文中的 MBSE 概念模型描述了系统工程过程。同年，Wayne W. 出版了 Model-based Systems Engineering 一书^[12]，介绍了与系统工程相关的术语和离散系统仿真的基本数学理论。后续研究人员开始提出多种建模语言，如对象过程方法论（Object-Process Methodology, OPM）^[13]、统一建模语言（Unified Modeling Language, UML^[14]）和系统建模语言（System Modeling Language, SysML^[15]），SysML 和 UML 都是由图形符号和信息模型共同组成的，能够形象、直观地描述系统。UML 是一种面向对象的软件建模语言，在表达能力和覆盖范围方面都极为出色。SysML 在 UML 的基础上，从更广泛的系统角度进行设计，通过不同的图描述系统的结构和功能等。建模工具作为建模语言的配套工具出现，如 Rhapsody^[16]、MagicDraw^[17]、Enterprise Architect^[18]和 Capella^[19]。MetaEdit+ 工具的出现是 MBSE 在萌芽期的一次技术突破，相比于其他工具，其引入了元建模概念，使用图、对象、属性、关系和角色（Graph、Object、Property、Relationship、Role, GOPRR）概念数据模型作为通用元模型，即作为一种语言，而不是直接使用特定的建模语言^[20]。建模语言和工具的出现使 MBSE 从理论过渡到实践，并为其应用奠定了基础。

INCOSE 在 2007 年精确定义了 MBSE 的概念并沿用至今，这标志着 MBSE 进入了发展的第二个阶段，MBSE 的定义为“通过形式化的建模方法，支持从概念设计阶段开始并持续到整个开发和生命周期后期的系统需求、设计、分析和验证与确认活动^[5]”。在这个阶段，更多的研究人员作为早期采用者开始研究 MBSE，提出了大量新兴技术。生命周期协作开放服务（Open Services for Lifecycle Collaboration, OSLC）是一种用于链接异构数据的规范，它在整个产品生命周期中集成了所有异构数据，加强了系统模型与其他异构模型的交互^[21]。此外，美国国防部（Department of Defense, DoD）将数字工程作为一项战略，MBSE 是其中的重要一环^[22]。同时出现了数字孪生的实践，旨在以数字化的形式创建物理实体系统在生命周期中的虚拟表示^[23]，以提升仿真结果

的真实性,这与 MBSE 的理念在一定程度上达到了一致。第一代建模语言和工具在这个阶段已经趋于成熟,NASA、欧洲航天局(ESA)、洛克希德·马丁公司和其他组织已经开展了广泛的实际应用,并取得了显著的成果。美国国家航空航天局采用 MBSE 方法在 Radio Aurora Explorer (RAX) CubeSat^[24-28]任务展开了十余年的探索。美国航空领域的学者在该小型卫星项目开发过程中,通过扩展对不同行为和场景进行建模的能力扩展了 MBSE 框架,并集成了几个仿真工具来执行基于 SysML 的行为模型,包括子系统功能和航天器的内部状态,验证了 MBSE 在采集真实卫星运行状态方面的适用性。NASA 下属喷气推进实验室将制定的 MBSE 发展战略应用于多个项目,有效地缩短了开发周期,提升了产品质量^[29]; Kussigit Santosa 等提出在实验动力反应堆开发过程中,MBSE 能够帮助工程师更细致地分解系统功能,描述系统内在联系,并有助于之后设计工程师的审查以及开发新的设计方案^[30]。

目前,MBSE 技术正处于复苏期。这将是 MBSE 整个生命周期(从概念出现到技术落后)中持续最长的时期。第二个阶段的标志性产物——SysML 已成为 MBSE 领域内应用最广泛的建模语言之一。通过对 SysML 和 MBSE 十多年来应用情况的需求调研,SysML 开发团队于 2018 年正式将 SysMLv2.0 转入研发阶段,目的是提高第一代 SysML 的准确性、表达性和可用性^[31]。与 SysML 发布第二代技术的做法类似,Dori D.团队研发了第二代建模环境 OPCloud,不仅支持概念建模,而且具有详细的定量设计能力^[32]。MBSE 在这一阶段开始强调与新兴数字化技术的融合。波音公司开始关注虚拟世界与物理世界的融合,提出了 MBE Diamond 方法论,方法论基于是系统工程 V 模型,下半部分由物理模型的设计和交付产物组成^[33],这是原始 V 模型的体现;上半部分由虚拟模型和数字孪生模型体现,包括建模和仿真;物理设计和模型仿真通过数字线程连接^[34]。McDermott 等提出了 AI4SE 的概念,旨在将 AI 集成到系统工程活动中,以便在产品生命周期中高效处理大量数据^[35]。本文的研究基础——KARMA 语言及其建模工具 MetaGraph,在这一阶段由北京理工大学、洛桑联邦理工学院和中科蜂巢科技有限公司开发,该建模工具旨在实现多种语言的统一建模,并使用图、对象、点、属性、关系、角色和扩展(Graph、Object、Point、Property、Relationship、Role、Extension, GOPRR-E)^[36]。

1.2.2 MBSE 方法论

建模方法论是 MBSE 的三大支柱之一,其目的是指导系统工程师有效且正确地开展系统设计活动^[37]。MBSE 领域的典型方法论包括 Harmony-SE、面向对象的系统工

程方法（Object-oriented Systems Engineering Method, OOSEM）、Arcadia、对象过程方法论（Object Process Methodology, OPM）和 MagicGrid。除了典型的方法论，美国国防部体系架构框架（DoDAF）也具有类似的功能，本文将其也纳入方法论的范围。

1) Harmony-SE^[38]。该方法论由 IBM 公司推行，核心思想是“服务请求驱动”，即基于系统 V 模型展开，分析系统中的需求活动服务^[39]。与 OOSEM 一样，该方法论利用了 SysML 的“模块”来表述系统对象的最小单元，由“模块”组成的活动图、状态图和序列图可以描述不同的行为场景，而这三种视图互相补充后可以精确描述系统架构的行为。Harmony 系统工程方法学的顶层架构包括三部分：能力需求分析，系统功能分析，设计综合^[40]。需求分析是指收集利益相关人的需求，并将其转化为功能性和非功能性的系统需求；功能分析是将需求转化为一致的系统操作的过程；设计综合用于开发同时满足功能需求和非功能需求的体系架构。

2) OOSEM。该方法论使用 SysML，具有面向对象和面向过程的优势，支持系统从顶层至底层的设计、分析和验证^[41]。OOSEM 还旨在简化与面向对象软件开发、硬件开发和测试的集成。具体来讲，OOSEM 利用面向对象的概念协调 MBSE 和其他建模技术的关系，工程师能够灵活地针对不同对象采取操作，减小系统的复杂度。此外，模型集成的方式确保了系统各视图之间的一致性^[42]。

3) Arcadia。该方法论是由法国泰勒斯 THALES 开发的基于 SysML 扩展的 MBSE 方法论，用于软、硬件和系统架构设计^[43]。Arcadia 是在不同的工程视角上构建的一种方法论，支持在系统上下文、需求建模和解决方案建模之间建立清晰的分离，主要包含运行分析、系统分析、逻辑架构设计和物理架构设计四大流程，并附带终端产品架构分解功能以开展后续开发设计^[44]。

4) OPM。OPM 于 1994 年被以色列科学家 Dori D. 正式提出，对象过程方法论被定义为用于获取知识和设计系统的概念建模方法^[13]。基于状态对象及其转换过程的最小通用逻辑，OPM 可以用于形式化地描述各种领域中人工和自然系统的功能、结构和行为^[45]。它考虑了系统设计的行为和结构，并将其与认知相结合，通过图形和自动生成的自然语言文本的双模式表达系统设计。

5) MagicGrid^[46]。该方法论是由达索公司开发的完全适配 SysML 且无需对其进行扩展的 MBSE 方法论。MagicGrid 将系统设计划分为问题域、解决域和实施域，问题域又分为黑盒和白盒两个阶段，通过构建需求模型、行为模型、结构模型和参数模型，建立需求到功能、逻辑和物理架构之间严格的关联、追溯和验证关系，实现系统

的验证和确认^[47]。MagicGrid 制定了严格的模型构建步骤，问题域黑盒阶段将兴趣系统定义为一个黑盒，关注系统如何与外界环境产生交互，不关注其内部的结构和行为；问题域白盒阶段将黑盒打开，关注系统应具备的功能，实现功能到需求的追溯；解决域通过构建行为模型、结构模型和参数模型为系统提供一个或多个解决方案。

6) Vitech MBSE。Vitech MBSE 是 Vitech 公司首席执行官和首席方法论专家 James E. Long 开发的一套 MBSE 方法论^[48]。方法论提出使用系统定义语言 (SDL) 管理模型，以一个统一的信息模型管理模型的语法和语义^[49]。尽管 Vitech MBSE 方法论在概念上是“独立于工具”的，但是其具体内容在 INCOSE 国际研讨会上仍主要采用其公司研发的 CORE 工具集进行了展示^[37]；

7) DoDAF。DoDAF 早期是美军为了满足各兵种需求，增强协同作战能力而设计的方法论^[50]。经过不断的发展变化，DoDAF 已经更新至 2.0 版本，旨在为体系架构的设计提供指南，增强体系架构的有效性和实用性，以便更精确地对其进行分析和评估。目前 DoDAF 由 8 个视角（全景视角、数据视角、标准视角、能力视角、作战视角、服务视角、系统视角、项目视角）和 52 个视图组成^[51]，视图可以包括文档、表格、图和其他可视化工具，从不同的角度提供不同的手段帮助设计人员开展体系架构设计，促进了各个组织之间的协作和信息共享。

目前大多数方法论基于建模语言，因此其服务范围也仅限于建模语言所能表达的系统设计层级和系统领域。但在面对涉及多学科、多系统层级的复杂装备设计时，对建模方法论的兼容性和自由度提出了更高的要求。

1.2.3 现有建模工具中方法论应用

目前关于在 MBSE 建模工具中如何支持方法论应用方面开展了大量的研究工作，已经形成了多种应用形式。

为支持 Harmony-SE，IBM 基于 UML/SysML 的模型驱动开发集成环境开发了相应的建模工具 Rhapsody^[39]。基于其提供的系统工程工具箱，Rhapsody 支持按 Harmony-SE 的流程生成相应的文件结构（图 1.2），同时创建空白的模型视图。

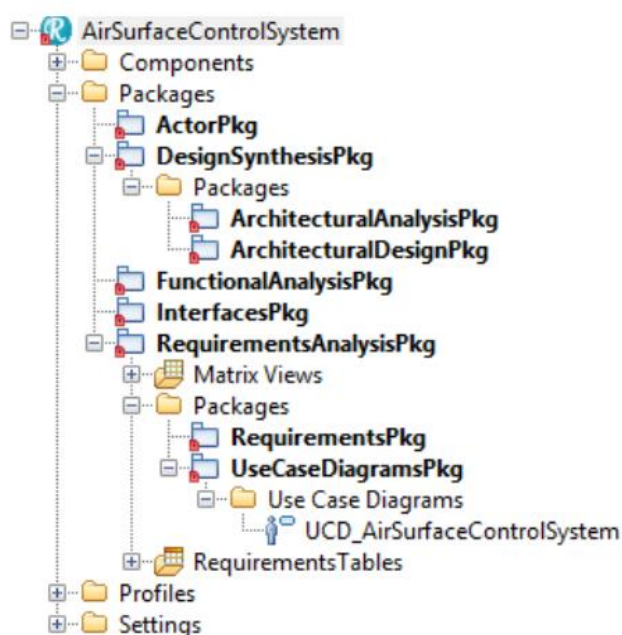


图 1.2 Rhapsody 表达文件结构

Capella 工具是支持 Arcadia 的平台解决方案^[43]。Capella 工具提供运行分析、系统分析、逻辑架构、物理架构和 EPBS 的向导式插件（图 1.3），设计人员可以在各个分析阶段根据需要创建所需视图。所有视图通过该向导插件管理，从而实现对系统架构的早期分析验证，实现了 Arcadia 方法论的生态搭建^[52]。



图 1.3 Capella 提供建模导航

MagicDraw 是支持 MagicGrid 系统工程方法论的建模工具，具有团队协作功能，

支持 UML、SysML 建模，支持工程师进行需求验证、设计决策权衡、模型一致性检查及需求追溯分析^[53]。MagicDraw 提供 MagicGrid 的项目模板，包括 MagicGrid 的项目结构和表单模型（图 1.4），构建的图模型可以拖入表单模型中，实现快速访问和模型管理。

	PILLAR				
		REQUIREMENTS	BEHAVIOR	STRUCTURE	PARAMETERS
DOMAIN	PROBLEM (BLACK BOX)	B1-W1 Stakeholder Needs Stakeholder Needs	B2 Use Cases Use Cases	B3 System Context System Context	B4 Measurements of Effectiveness Measurements of Effectiveness
	PROBLEM (WHITE BOX)		W2 Functional Analysis Functional Analysis	W3 Logical Subsystem Communication Logical Subsystems Communication	W4 Measurements of Effectiveness —
	SOLUTION	S1 System Requirements System Requirements	S2 System Behavior —	S3 System Structure System Structure	S4 System Parameters System Parameters
		SS1 Subsystem Requirements —	SS2 Subsystem Behavior Subsystem Behavior	SS3 Subsystem Structure Subsystem Structure	SS4 Subsystem Parameters —
	IMPLEMENTATION	I1 Physical Requirements —	Software, Electrical, Mechanical		

图 1.4 MagicDraw 方法论导航模型

对于部分独立于工具的 MBSE 方法论，例如图 1.5 中 OOSEM，研究人员利用 SysML 的活动图或其他描述业务流程的模型描述该方法论架构^[54]。然而，这种利用特定建模语言构建流程模型的形式只能描述流程，因为设计 SysML 的初衷就是用于描述系统而不是管理系统，也就无法起到串联系统全生命周期模型的作用，不利于模型管理和应用。

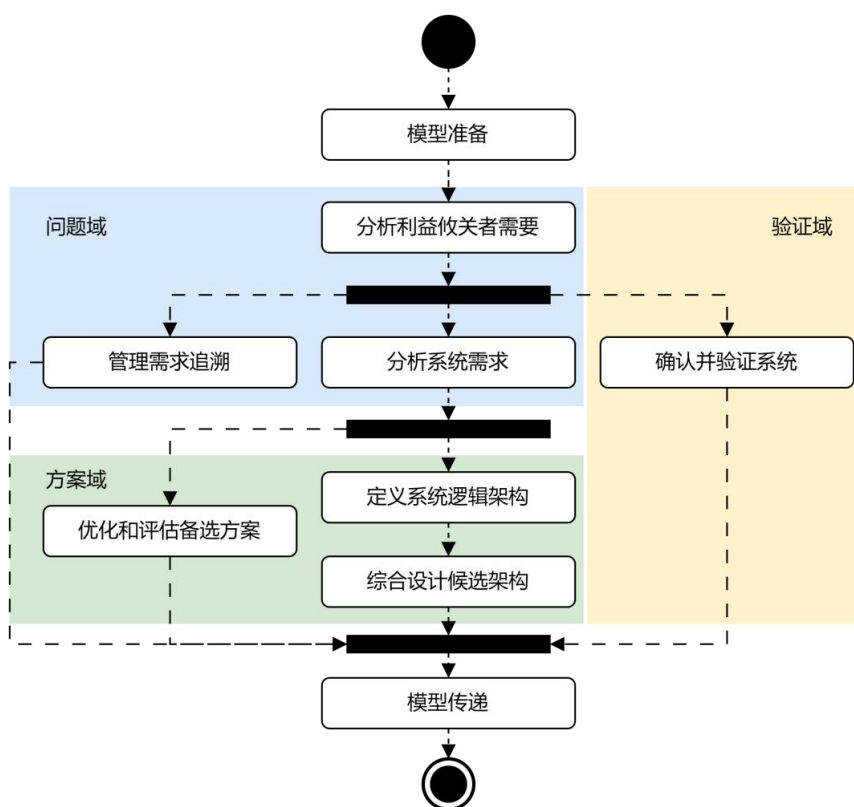


图 1.5 基于 SysML 的 OOSEM 设计流程

综上所述，目前建模工具对于方法论的表达主要存在四种模式：1）文件结构表达；2）向导式表达；3）表单模型式表达；4）利用现有建模语言，自主构建流程模型表达。尽管已经形成了诸多成熟的 MBSE 建模方法论表达模式，这些模型的表达仍是基于所适配的建模语言和建模工具，并没有形成一种统一且标准的表达形式。

1.2.4 存在的主要问题

根据上述国内外研究现状的分析，研究人员在 MBSE 的各领域，尤其是方法论，已经做了诸多探索，但仍然存在以下问题：

1) 现有方法论无法适应多层次、多领域的复杂装备系统开发场景。MBSE 应用已经从过去单一系统试点开始向涉及多系统层级、多学科的复杂装备系统深入。然而，当前方法论往往针对特定系统或领域研发，其定制化和可扩展性较弱，因此无法满足系统生命周期的设计需求。此外，当前方法论的使用受建模语言和建模工具限制，若采用多种方法论开发复杂装备系统，不同方法论所使用的建模语言及所适配的建模工具不同（例如选择 MagicGrid 为建模方法论，就要使用 SysML 建模语言和 MagicDraw 建模工具），这会导致不同工具体系之间模型不一致及接口问题。

2) 现有 MBSE 研究忽略了面向设计流程的建模对复杂装备系统设计的重要性。目前 MBSE 研究主要关注如何更清晰地描述具体的架构“模型”，这关系到诸多建模语言的研究（例如用模块定义图描述系统组织结构），却忽略了“怎么用”模型更清晰地描述“系统”，即方法论的流程模型要素表达，缺乏设计流程模型的编织，每一个架构模型就只是一个单独的“点”，无法形成一条显式存在的“线”。此外，方法论的流程建模涉及大量同系统生命周期多层次、多领域要素的映射，而当前该过程并未形成统一的建模语言规范，实际的模型要素以设计人员根据理解自行设计为主，不利于项目中其他学科人员的协作，对于系统架构的维护和扩展造成困难。

3) 现有方法论缺乏与建模工具的深度融合导致设计过程仍复杂。MBSE 旨在通过模型驱动系统设计。然而，目前方法论在建模工具的应用一般侧重于表达，仅仅是将方法论电子化（类似于纸质文档到 Word 电子文档的转变，实际上仍是基于文档）。这样的问题在于，没有充分发挥模型的优势，导致基于方法论构建的流程模型驱动系统设计的能力不足，无法支持模型的查看、创建、访问、执行等功能。

1.3 研究意义

根据工信部发布的《“十四五”信息化和工业化深度融合发展规划》，要提升智能制造供给支撑能力，推进 MBSE 规模应用^[55]。与方法论相关的研究是当前 MBSE 研究的不足之处，却是指导复杂装备系统设计的关键环节。因此，本研究以 KARMA 语言及其建模工具为基础，集成 KARMA 语言关键能力，提出复杂装备系统导航式建模的概念，通过导航式建模以流程的形式构建 MBSE 方法论的模型，并提供辅助设计服务，旨在补足 MBSE 方法论研究的缺陷，这对于推进 MBSE 规模应用，加快数字化转型具有重要的现实意义，具体表现在：

(1) 对 MBSE 应用的意义。随着 MSBE 实践应用的深入，我国复杂装备研制领域已经开始尝试结合现有研发流程和实践，提出适应自身研制体系的方法论^[56]。然而，这种自研的方法论并不被国外建模工具直接支持。本文所提复杂装备系统导航式建模技术为领域人员提供了在建模工具中构建方法论的途径，有助于复杂装备领域实践方法论落地，推进 MBSE 实践应用。

(2) 对复杂装备 MBSE 建模平台的意义。建模平台是复杂装备设计人员构建系统架构模型的手段，本研究开发的导航式建模系统强化了方法论与建模平台的融合，通过导航模型集成建模过程中全生命周期所需功能，真正用模型驱动全生命周期的设

计、分析、仿真等操作，对于智能制造发展中建设数字化研发体系，提升 MBSE “软”实力有重要意义。

1.4 研究内容和组织架构

本文围绕复杂装备系统导航式建模技术，研究了方法论的模型要素表达、模型转换和工具融合，涉及面向导航式建模的元模型设计、复杂装备系统导航式建模语言设计、复杂装备系统导航式建模系统设计及验证。本文共分为 6 章，其组织结构如图 1.6 所示。

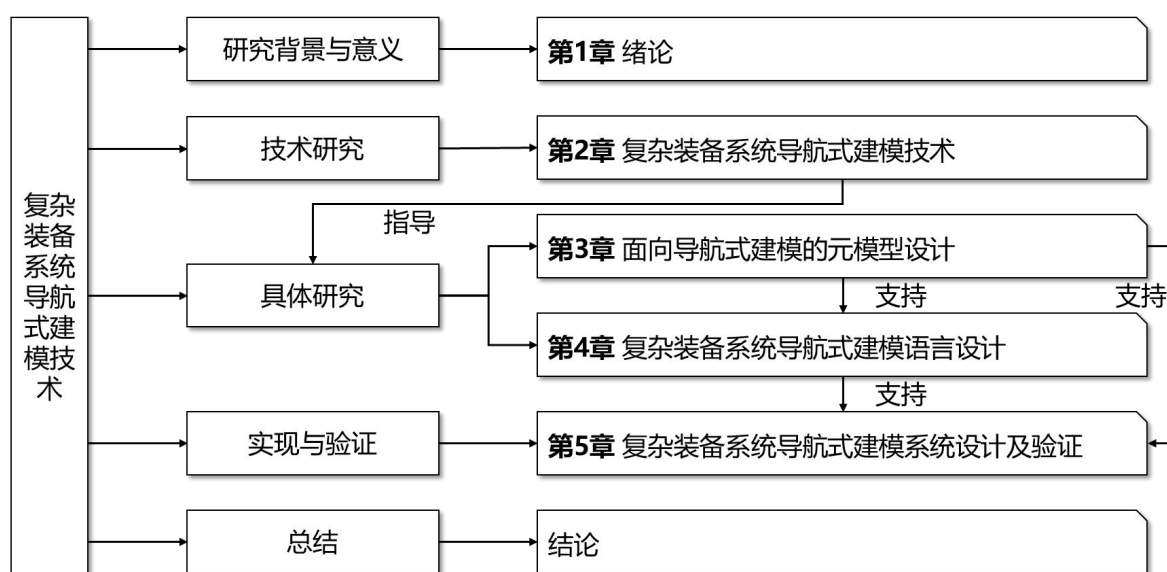


图 1.6 论文组织结构

第一章：绪论。介绍本文的研究背景，对国内外研究现状进行综述，包括 MBSE 发展现状、MBSE 方法论和 MBSE 方法论应用，分析了当前 MBSE 方法论相关研究存在的问题，引出本文研究意义，最后介绍本文的研究内容和组织结构。

第二章：复杂装备系统导航式建模技术。首先针对 MBSE 方法论应用问题，明确本文所提技术的技术基础，并根据 MBSE 的特点分析导航式建模设计需求。然后结合元对象机制和数据建模理论提出导航式建模的技术框架，旨在描述方法论的模型要素。最后，讨论了有效使用所提技术指导复杂装备系统设计的工作流。

第三章：面向导航式建模的元模型设计。首先，分析构建导航式建模元模型的元建模技术。然后，结合研发流程的设计需求，利用实体-关系图设计概念数据模型。最后，利用 GOPRR-E 元元模型扩展概念数据模型，开发导航式建模的元模型支持方法论的流程建模。

第四章：复杂装备系统导航式建模语言设计。本章首先研究 GOPRR-E 元元模型到 XML 的映射机制。然后，基于 XML 研究导航式建模语言的具体语法和抽象语法。最后，设计 XML 模型、流程模型和文件结构模型的模型转换机理，并开发配套的模型转换算法。

第五章：复杂装备系统导航式建模系统设计及验证。首先，围绕导航式建模技术，提出实现导航式建模系统的技术框架，并给出系统功能架构。然后，从定量和定性两个角度，将本文所设计的系统与同类产品进行对比，评估系统能力。最后，以“毅力号”火星车的 MBSE 设计流程为例，采用本文所提系统构建导航模型，驱动系统设计活动，以验证本文所提技术的可行性和有效性。

结论：对本文的研究工作和创新点进行总结，并分析研究存在的不足和未来的工作展望。

1.5 本章小结

本章阐述了论文的研究背景，综述了 MBSE 方法论研究中存在的问题，并提出了复杂装备系统导航式建模的概念。基于此，本章论述了研究意义并对论文的研究内容和组织结构进行简要介绍。

第 2 章 复杂装备系统导航式建模技术

2.1 引言

MBSE 方法论的目的是指导设计人员完成架构设计活动，但是实际应用过程中方法论往往隐式存在于设计人员的脑中，在面对复杂装备系统设计（需要设计大量的架构模型）时，会出现设计流程不透明、依赖于特定的设计人员的问题，导致系统模型的可维护性和可追踪性较差。以往这方面的研究一般由建模工具厂商主导，以建模工具为基础支持自身产品体系下的方法论，这种模式以功能展示为主要目的，依然没有对方法论的模型要素表达和实际应用展开深入研究。

基于上述问题本章开展理论研究，提出复杂装备系统导航式建模技术，旨在支持方法论的模型要素表达，驱动复杂装备系统架构设计。首先，介绍了 KARMA 语言的多架构建模能力及其扩展元素，并基于 KARMA 语言总结了复杂装备系统导航式建模的关键要素，以作为技术框架的需求输入。然后，研究复杂装备系统导航式建模技术框架，将元对象机制引入数据模型的概念中，定义导航式建模的模型层次，为复杂装备系统导航式建模提供理论基础。最后从业务角度提出复杂装备系统导航式建模的工作流，为设计人员的使用提供指南。

2.2 技术基础及需求分析

2.2.1 技术基础

为了解决在使用不同建模语言和工具的过程中，构建的模型异构导致方法论无法从应用层面指导、管理全部系统模型的问题，本研究提出以 KARMA 语言及其建模工具为基础，利用 KARMA 语言保障模型同构、“系统模型”同平台的多架构统一建模特性，开发导航式建模系统管理“系统模型”，实现方法论的定制化开发，驱动模型的导航、创建、仿真等功能。

KARMA 语言（(K)Combination of ARchitecture Model specificAtion）是一种基于 GOPRR-E 元元模型构建的多架构统一建模语言，以 KARMA 语言为代表的语义建模是第二代建模语言技术^[57]，支持不同系统层级、不同系统视角、不同领域的模型构建与描述^[58]。

KARMA 语言的表现形式是文本，利用文本描述模型特征。在 KARMA 语言的配

套建模工具中，设计人员在工具界面设计模型的同时，工具后台会自动生成 KARMA 语言的形式化表达。图 2.1②展示了图 2.1①中 SysML 模块定义图的 KARMA 语言形式化表达，其中第一行 Object 对图中方框内的对象元模型属性进行了定义，第二行 Property 表示其拥有的属性为“运载火箭”，第三行 annotation 定义了此对象元模型的固定语法形状、位置和颜色等；同样，图 2.1④KARMA 语言也可以对故障树模型进行形式化表达。而第一代建模语言受其范式限制只能基于固定的元模型表达，无法同时描述多个系统或领域。例如，SysML 能对图 2.1①模块定义图进行表达，但是不能对图 2.1③故障树（SysML 不具备此类元模型）进行表达，导致“系统模型”异构。

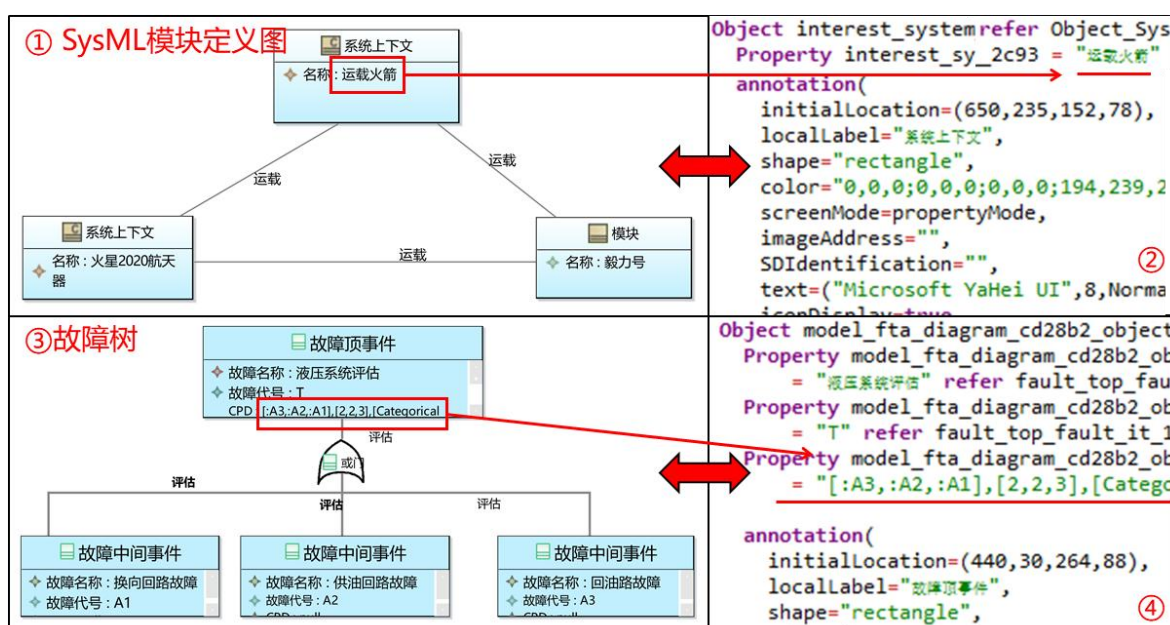


图 2.1 KARMA 语言支持多架构统一建模

然而，尽管目前 KARMA 语言及其配套建模工具已经在多架构建模方面取得了重要进展，但是其不支持对于建模方法论的描述，不同模型及功能之间缺乏较好的连接性，这导致设计人员在建模过程中缺乏设计指导，建立的架构模型较为松散，缺乏整体性，不利于前期设计与后期管理。结合 KARMA 语言的优势，本文旨在开发导航式建模系统以弥补当前利用建模工具开展方法论深度应用的劣势。

除了核心的建模能力外，KARMA 语言还具有如代码生成、架构驱动等辅助设计功能，其中代码生成功能是本研究中利用导航式建模实现在设计过程中对特定域仿真模型管理的基础（详见 2.4.2）。利用 KARMA 语言代码生成部分的语法^[59]，通过提取所构建的模型属性信息，实现系统架构模型到工具外部异构模型的模型转换，具体

语法规则示例如图 2.2 所示：（1）定义生成的特定域文件；（2）对同类对象元模型的参数属性进行遍历提取；（3）将提取的参数属性填入映射语言的固定模板；（4）打印至文件。

```

1. codeGenerator 代码生成器名  ←
2.   generate 函数名  ←
3.     fileName "生成文件路径" write //新建文件并开始先文件写入
4.     for 1:n(某个或某类 Object)  ←
5.       for 1:j(Object 的某个属性)  ←
6.         //提取形式化代码属性并按固定模板向基语言转换  ←
7.         print("固定模板"+valueof(this));  ←
8.       end  ←
9.     end  ←
10.    ... //重复上述 for 循环, 完成全部类型的映射  ←
11.    close//关闭文件  ←
12.  end 函数名  ←
13. end 代码生成器名  ←

```

图 2.2 利用 KARMA 语言执行模型转换方案

2.2.2 导航式建模需求分析

霍尔系统工程模型由霍尔在 1962 年出版的《系统工程方法论》一书中提出^[60], 该模型将系统工程作为一种解决问题的实际方法, 后经其不断改良, 最终定义为一个三维结构模型, 具体分为: 专业维, 时间维, 逻辑维^[61]。如图 2.3 所示, 三个维度的具体含义如下:

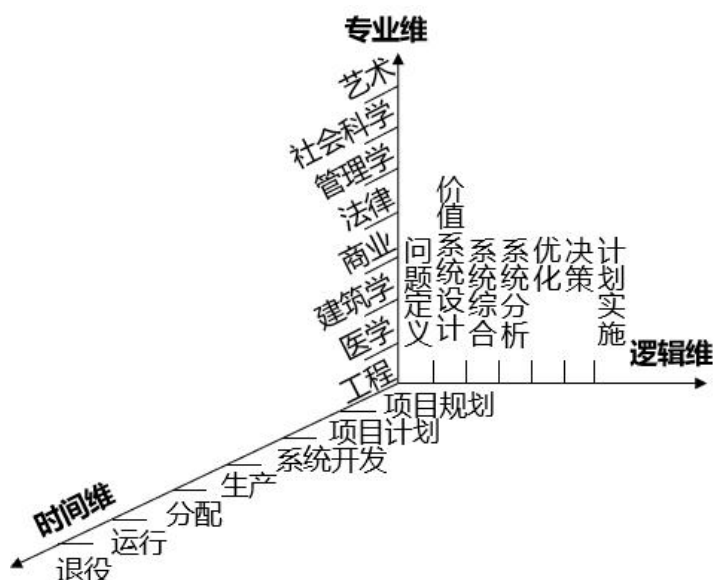


图 2.3 霍尔三维系统模型

时间维表示系统研制过程中, 从开始到结束的系统工程活动的全生命周期: 项目

规划、项目计划、系统开发、生产、分配、运行和退役阶段。逻辑维描述了时间维每个阶段要做的工作和要遵循的思维过程，有七个逻辑步骤，分别是问题定义、价值系统设计、系统综合、系统分析、优化、决策和计划实施^[62]。专业维列出了时间-逻辑二维系统工程方法可以应用的主题领域，包括：艺术、社会科学、管理学、法律等方面的知识或技能。

由于霍尔三维系统模型的提出年代较早，且针对的是普适性问题，不适于分析 MBSE 的活动。为了更好地将霍尔三维系统工程模型应用于 MBSE 中，本研究参考经典霍尔三维系统工程模型，设计了 MBSE 的三维模型，如图 2.4 所示。首先，将专业维改进为系统维，系统维代表 MBSE 设计对象的系统层级，包括体系级、系统级、分系统级、组件级等。原因在于，MBSE 设计过程关注设计对象的系统层级，不同系统层级对象所对应的建模语言、建模工具和建模方法论都有较大差异。然后，根据 MBSE 的具体活动将逻辑维修改为系统工程活动维，包括需求、设计、分析、验证和确认，这一维度来源于 INCOSE 对 MBSE 的应用定义^[5]。时间维参考 NASA 对项目生命周期的定义，考虑到本研究的范围，忽略与产品/系统具体实现的阶段，将时间维定义为概念研究、概念开发和初步设计三个阶段^[63]。

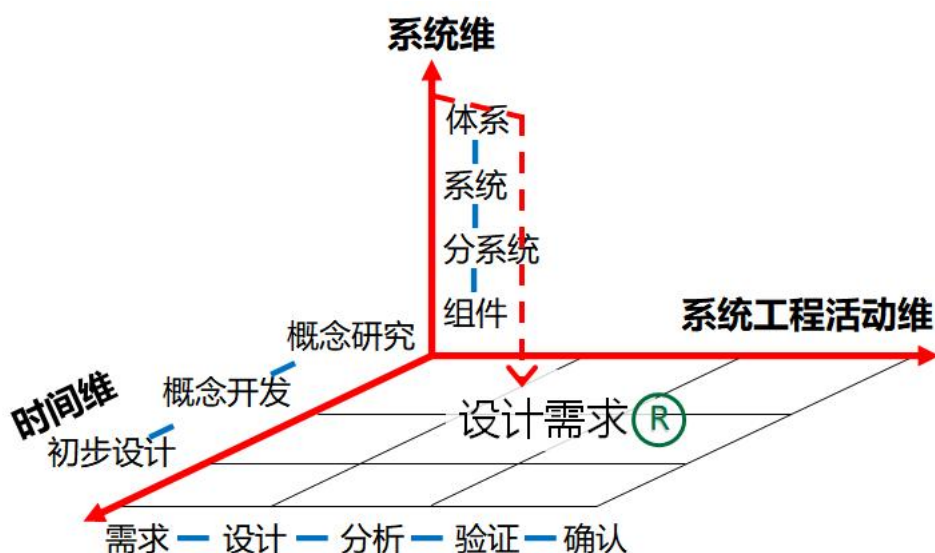


图 2.4 MBSE 的三维模型

通过 MBSE 的三维模型将系统工程活动维度和产品/装备开发演化的时间维度区分开，一维化二维的过程有助于拓宽设计人员的视角，便于深入探索解决方案，提升系统设计的精确性和准确性。如表 2.1 所示，将时间维的系统阶段和系统工程活动的步骤整合，得到 MBSE 的霍尔管理矩阵，元素 A_{ij} 为 MBSE 在该阶段所执行的步骤或

活动，从而全面分析 MBSE 的各个活动细节。

表 2.1 MBSE 的时间维和逻辑维过程矩阵

时间维 系统工程活动维	概念研究	概念开发	初步设计
需求	A ₁₁	A ₁₂	A ₁₃
设计	A ₂₁	A ₂₂	A ₂₃
分析	A ₃₁	A ₃₂	A ₃₃
验证	A ₄₁	A ₄₂	A ₄₃
确认	A ₅₁	A ₅₂	A ₅₃

如表 2.2 所示，分析二维矩阵的具体元素在复杂装备系统各个层级下的表现，进而分析本研究的设计需求。首先从不同系统层级的角度（纵向）看，各级系统设计需求主要体现在建模与仿真上，针对不同层级、领域的系统建模与仿真工具不同。从系统工程活动维度（横向）看，MBSE 活动过程存在建模、仿真、计算验证、需求条目管理与追溯、模型快速生成、模型转换和可视化展示的需求。

表 2.2 需求分析矩阵举例

系统工程活 动维	A11	A21	A32	A43	A53
体系	使命条目化	使命概念建模、 模型生成	可行性、可 靠性分析	对抗推演仿真、指 标验证、模型转换	需求追溯、 可视化展示
系统	系统需求条 目化	系统概念建模、 模型生成	同上	行为仿真、指标验 证、模型转换	同上
分系统	分系统需求 条目化	分系统概念建 模、模型生成	同上	特定域仿真、指标 验证、模型转换	同上

2.3 复杂装备系统导航式建模框架

本文在元建模方法和数据模型层次的基础上提出了复杂装备系统导航式建模技术框架，研究利用元元模型实现对复杂装备系统导航式建模元模型的构建，利用数据模型实现流程模型到建模语言文件数据的存储，建立精细化的导航式建模模型层次，

2.3.1 元对象机制

MOF 由 4 级结构组成，称为 M3-M0 层，分别是 M0 层系统视角层，M1 层模型层，M2 层元模型层和 M3 层元元模型层。通过这 4 个层级实现了在同一框架下定义模型和元模型（元来自希腊语的 **Meta**，表达超越的意思），实现模型的交换和集成。如图 2.5 所示：

图 2.5 元对象机制

M1 为模型层，模型是元模型的实例，用于描述系统整个生命周期的活动。例如通过 SysML 构建表达系统活动的时序图或表达架构的模块定义图；

型，同时这些元模型都是由元元模型构成的；

M3 为元元模型层，在元模型层之上，具有最高的抽象级别。本质上元模型和元元模型没有区别，只是在 MOF 中的抽象水平不同。通过这种方式，设计人员可以按照需求解除元模型的限制——利用元元模型的组合自行构建多领域元模型库，实现不同模型的抽象与交互。

MOF 的 4 级结构是描述复杂装备系统模型语义的基础，通过对 M3-M0 层级的嵌套循环，可以精准地将建模语言语义定位到适当的系统层级，提升建模语言设计的有效性。基于上述优势，本研究采用 MOF 作为基础框架之一，设计复杂装备系统导航式建模系统和建模语言。

2.3.2 数据建模

数据建模是利用形式化的建模技术将真实世界的的数据抽象，确定业务流程中数据的范围、形式、类型等，形成信息系统模型的过程。数据建模是信息系统设计的核心业务，直接影响系统架构设计。良好的数据模型是对数据进行挖掘、分析、维护和管理的基础。

美国国家标准协会在 1975 年定义了数据建模的三个层级，分别为概念数据模型、逻辑数据模型和物理交换规范，如图 2.6:

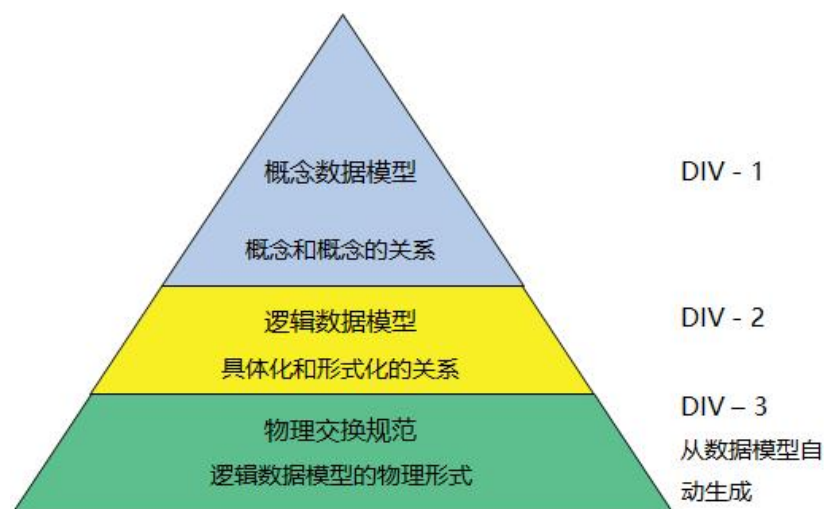


图 2.6 数据模型层次

概念数据模型是不同用户群体看到信息系统的不同视图的组合，关注系统的实体、关系和基数，不过分关注内部细节，目的是设计符合业务需求的数据集合，保障术语的一致性，主要参与人员为不同的利益相关者。

逻辑数据模型进一步挖掘概念数据模型，对实体及其关系进行细化，确定实体应具有的属性及约束等，形成具体信息域结构。

物理交换规范包括数据存储的细节以及具体的数据库和硬件实现，主要关注如何将逻辑数据模型转换成为标准的存储形式^[65]。

在 MBSE 领域，美国国防部体系结构框架（DoDAF 2.0）就采用数据建模的方式搭建，整合了体系架构设计所需的本体、概念模型、元模型、视角和模型，为利益相关方提供包括全景、数据与信息、标准、能力、作战、服务、系统和项目八个视角，支持体系设计过程中的数据分析、挖掘^[66]。因此，基于 DoDAF 的探索，将数据建模的概念应用于 MBSE 是可行的。

2.3.3 结合元对象机制和数据建模的总体框架

尽管 MOF 严格定义了复杂装备系统建模的层级结构，但是其对于如何从元元模型构建元模型缺乏详细的论述，导致所设计的建模语言存在术语缺失、难以在不同的利益相关者之间达成一致、关系结构不完整的风险。而数据建模目前主要应用于构建数据库，其在如何利用模型表达系统视角上存在劣势。

针对以上缺点，本研究结合 MOF 在构建架构模型和数据建模在搭建信息模型方面的优势，融合了 MOF 与数据建模理论，提出了复杂装备系统导航式建模的总体框架（图 2.7），核心在于将数据模型层次融入到 MOF 的三个层次中，设计具有图形化表达能力的文本式建模语言：

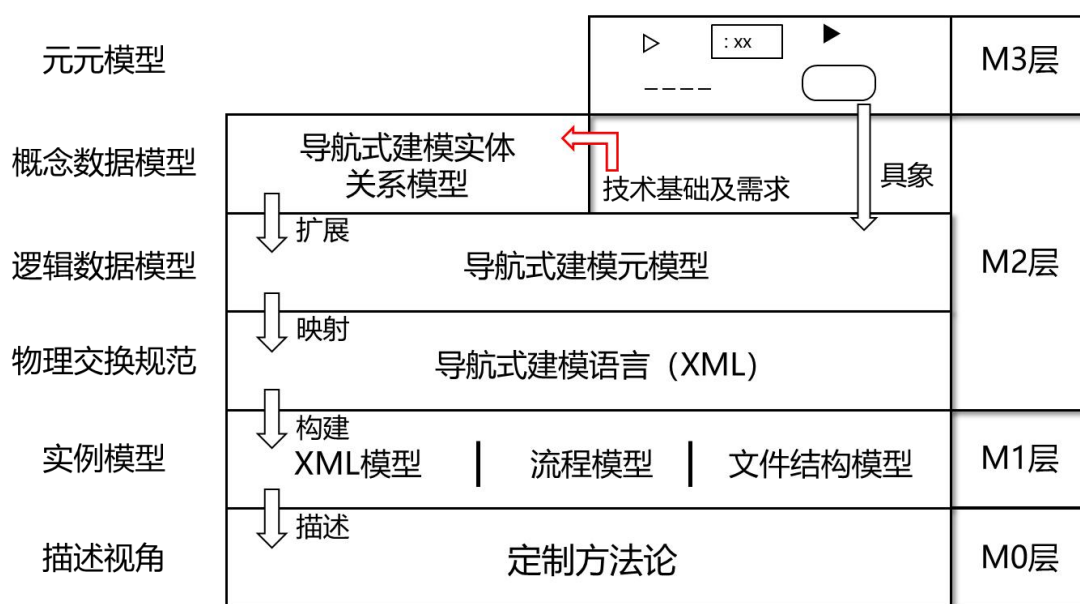


图 2.7 结合元对象机制和数据建模的总体框架

(1) M3 层

M3 层即元元模型层，这一层要求选择表达能力强的元元模型来描述元模型。M3 层是整个框架的基础，良好的元元模型能够构建语义丰富的元模型，保障整个复杂装备系统导航式建模技术应用的全面性。

(2) M2 层

M2 层即元模型层，即构建方法论的模型元素，是复杂装备系统导航式建模的核心。设计人员利用 M2 层的元模型即可搭建 MBSE 方法论的流程模型，管理项目模型。本研究将 M2 层拆分，与数据建模过程中所产生的三个模型层级映射：

1) 概念数据模型层

概念数据模型层以 2.2 分析所得的导航式建模的技术基础及需求为输入，充分考虑 MBSE 活动的业务需求，规范复杂装备系统导航式建模术语，对导航式建模过程中所必要的模型实体、实体之间的关系进行定义。

2) 逻辑数据模型层

基于第一阶段设计的概念数据模型，构建实体到导航式建模的元模型的映射，即根据规范的术语结构开发方法论的模型元素表达。主要方法为结合 M3 层的元元模型特性对实体关系模型进行扩展，展开与建模相关的细节设计，最终具象形成元模型。

3) 物理交换规范层

物理交换规范约束了设计人员利用元模型建模后所产生数据的存储格式，即本文所提建模语言。规范的存储格式有助于设计人员交流方法论，以及后续方法论相关模型要素的转换与生成。建模语言基于 XML 设计，主要原因在于 XML 也是一种元语言，其具有足够的能力覆盖现行建模语言的标准。此外，除了本课题目标平台外，其余主流 MBSE 建模平台，如 MagicDraw、Rhapsody 以及 Enterprise Architect 等均支持 XML 的导入导出，已经在 SysML、UML 与 XML 的映射机制上展开了技术应用，所以采用 XML 作为模型的存储格式有较强的可行性。

通过概念、逻辑和物理的构建顺序，既保障了复杂装备系统导航式建模语言语义构建的完整性和准确性，还同时解决了图形化建模的元模型构建和文本式数据存储问题。

(3) M1 层

实例模型层来自设计人员利用导航式建模搭建的导航模型。本研究对标现行主流方法论模型表达范式，提出了流程模型、XML 模型和文件结构模型一体三面的导航

模型形式（图 2.8），流程模型是对方法论流程的模型化描述，XML 模型用于存储和轻量化描述，文件结构模型通过文件夹的形式描述方法论的层级结构。设计人员只需任意构建其中一类模型，就可以自动转换生成另外两类模型，提升研发效率。



图 2.8 导航模型的三种形式

(4) M0 层

描述视角层由流程模型、XML 模型和文件结构模型共同描述，旨在描述一套完整的定制化方法论。

综上所述，技术框架的概念数据模型和逻辑数据模型设计对应本研究的第 3 章，物理交换规范和实例模型设计对应本研究的第 4 和第 5 章。

2.4 复杂装备系统导航式建模 workflow

2.4.1 方法论设计框架

本研究所开发复杂装备系统导航式建模技术应用于 MBSE 方法论的设计，因此本节对如何定制化开发 MBSE 方法论进行简要研究，但这部分并不是本研究的重点。

Franke H.-J.等^[67]在方法检索辅助系统的研究中对方法的基本结构进行了分析。首先每种方法都要具备输入和输出参数，输入参数描述需要执行的设计活动，输出参数表示应用该方法的结果。输入输出参数可以是信息，也可以是实物。其次，在将输入参数转换为输出参数过程中，会涉及以下几个方面：

- (1) 方法的流程或算法的步骤；
- (2) 方法采用者的基本素质，例如技能、经验、资格；
- (3) 一般条件，例如必要基础设施，工具的可能应用领域。

如图 2.9 所示，基于 Franke 等的研究，本文提出了一套方法论设计框架，框架主要分为两部分：

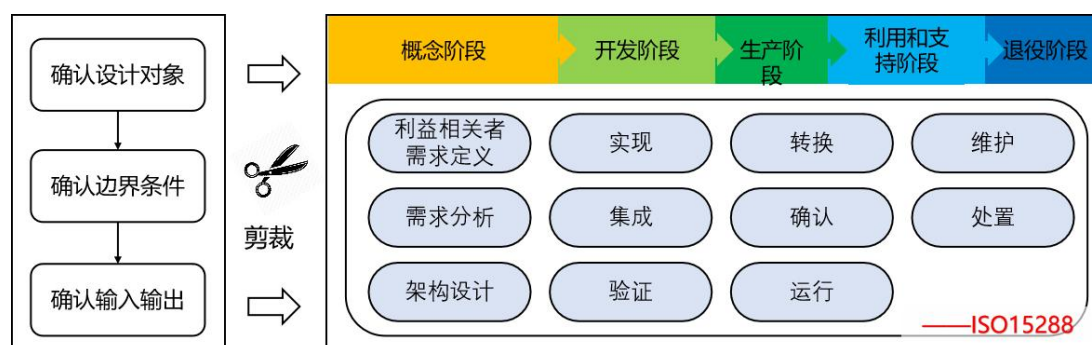


图 2.9 方法论设计框架

（1）在正式展开方法论的设计前

1) 首先要确定设计对象，这是方法论的基石。针对体系设计的对象，有 DoDAF 架构；针对系统设计的对象，有 MagicDraw 等方法论；针对多系统层级的对象，可能存在多种方法论结合使用的情况。因此设计对象的确认，对于方法论的设计至关重要；

2) 然后，确认边界条件。确认边界条件并非指系统设计过程中确认系统的边界，指的是确认设计人员、基础设施等外在的边界条件，包括设计人员的技术、创造力、抽象能力、经验，使用的工具等；

3) 确认输入输出产物。产物可以是一组模型、一组参数，也可以为一个具体的系统。

这些前提条件的确认，有助于后续剪裁设计和具体设计流程的优化。

（2）方法论设计过程中

参考 ISO/IEC 15288 标准，方法论流程需要在特定对象的实践经验基础上，结合生命周期阶段和技术流程元素进行剪裁设计。具体来说，生命周期阶段包括概念阶段、开发阶段、生产阶段、利用和支持阶段、退役阶段，技术流程元素包括利益相关者需求定义、需求分析、架构设计、实现、集成、验证、转换、确认、运行、维护和处置。此外，直接对现有成熟方法论进行剪裁也是当前应用较多的设计模式^[68]。

2.4.2 面向 V 模型的导航式建模 workflow

本文针对复杂装备系统导航式建模技术，提出了一套面向 V 模型的导航式建模 workflow，从业务的角度描述复杂装备系统导航式建模的应用。V 模型是系统开发生命周期的图形表示，总结了项目生命周期开发及相应可交付成果开展的主要步骤，描述了在产品开发过程中要执行的活动和必须产生的结果^[69]。

如图 2.10 和图 2.11，面向 V 模型的导航式建模 workflow 分为开发和应用两个阶段。

(1) 开发阶段

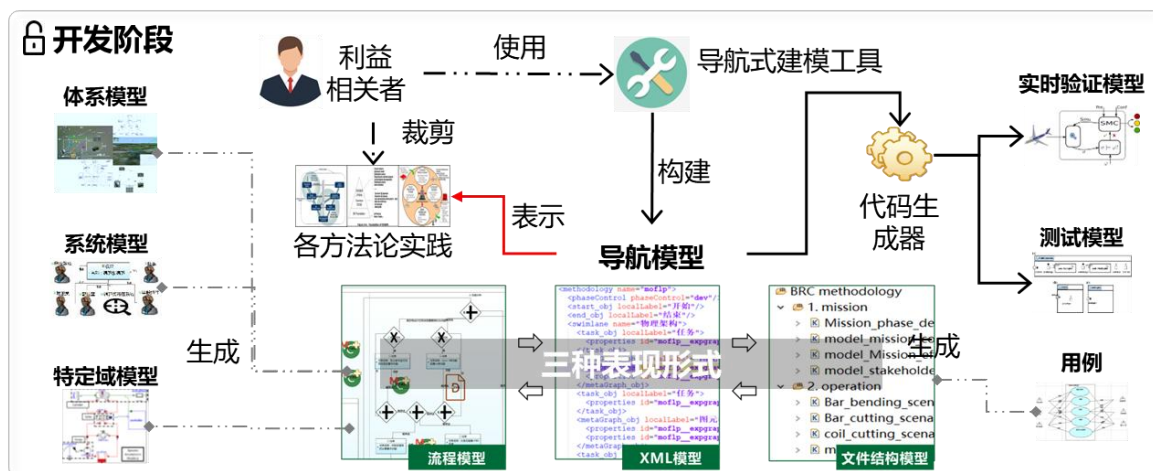


图 2.10 导航式建模开发阶段 workflow

开发阶段指利用导航式建模开发方法论的阶段，在这个阶段未开展系统设计。流程模型为开放状态，可以进行建模操作。利益相关者首先基于 2.4.1 所提方法论设计框架或直接对现有方法论实践进行裁剪。然后使用导航式建模工具建模，完成方法论设计。构建好的流程模型可以快速创建 V 模型各阶段所需的模型模板，从而获得整个系统设计的文件结构，并保持关联，使用户可以通过流程模型快速访问，提升了模型创建和访问的效率，具体包括体系模型、系统模型、特定域模型、测试模型、验证模型、确认模型等。

(2) 应用阶段

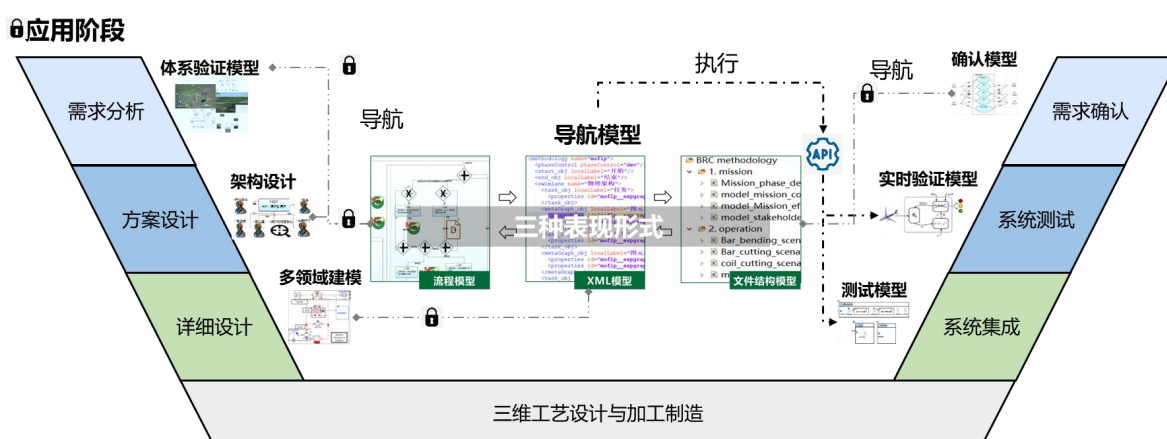


图 2.11 导航式建模应用阶段 workflow

应用阶段研发人员可以直接通过流程模型直接访问到各阶段所需构建的系统模型，执行设计、分析活动。

由于系统的设计涉及多种学科、多个层级，整个系统模型处于不断地交付和传递中。为了避免系统模型向下交付过程中下游设计人员误操作，本研究开发了流程模型只读技术，能够实现在应用阶段锁定流程模型，具体技术细节见 4.4.3。

在系统工程 V 模型右半部分，针对所生成的由外部工具运行的异构仿真模型，通过打通与特定域仿真工具的接口（Application Programming Interface, API），支持流程模型直接调用特定域仿真工具仿真。

（3）系统架构模型到特定域模型转换子 workflow

上文所述的在应用阶段涉及的特定仿真验证模型，如 Modelica 模型，这些模型对于复杂装备系统导航式建模系统属于异构模型，由其他专门的仿真工具执行。为了实现导航模型对这些异构的特定域模型的统一管理，基于模型驱动思想提出系统架构模型到特定域模型转换的子 workflow，实现导航模型对“系统模型”的管理与表达。

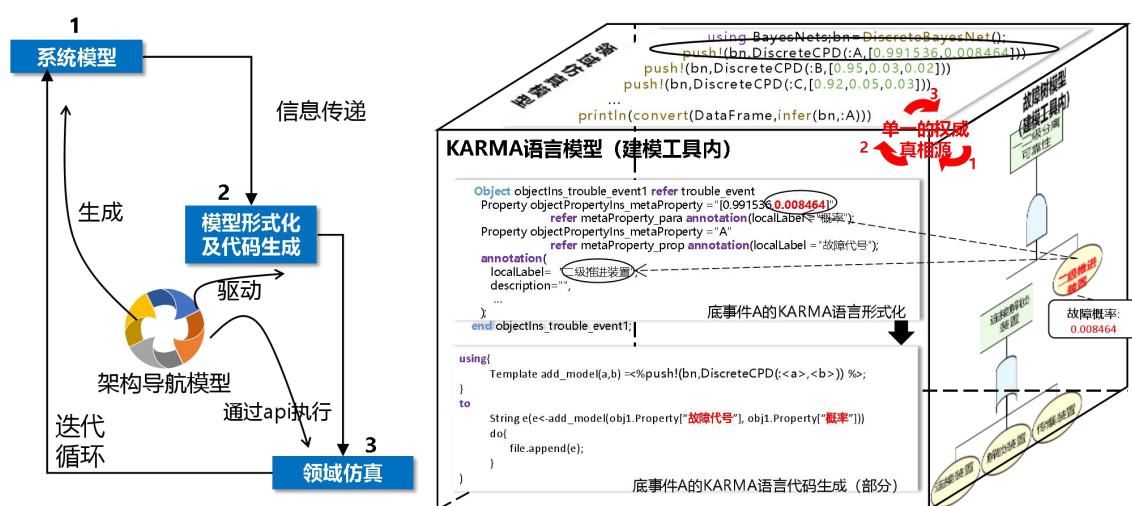


图 2.12 图模型、KARMA 语言和特定域模型的一体三面

系统架构模型到特定域模型转换的子 workflow 如图 2.12 所示，通过导航模型驱动，整个流程涵盖了系统模型、KARMA 语言和特定域模型的转换，是一个一体三面的流程。结合 MBSE 的可靠性分析案例，展开介绍：

1) 利用导航模型生成系统模型，设计人员完成系统模型的建模后，系统模型则具备充分的信息，例如立方体的第 1 面中故障树模型的底事件 A 故障的概率为 0.008464；

2) 利用导航模型驱动执行已经编写的 KARMA 语言代码生成脚本，使 KARMA 语言形式化后的系统模型转换为特定域模型（外部）。立方体的 2 号面展示了底事件

A 的 KARMA 语言形式化代码, KARMA 语言同样描述出了该对象的为底事件 A, 概率属性为 “[0.991536,0.008464]”, 事件故障代号为 “A”; 然后, 根据 KARMA 语言代码生成的语法编写代码生成脚本, 通过遍历相关对象元素, 提取形式化代码属性, 并添加相应的固定模板约束参数属性, 使其成为可靠性计算工具的可执行脚本。图中实例代码片段旨在提取 metaObject_trouble_event 一类元模型对象(底层事件属于该类元模型对象)中的必要信息——故障代号和概率, 并按照 “push!(bn,DiscreteCPD(故障代号,概率))” 的映射规则生成可靠性计算脚本, 中间事件、顶层事件按照相同的方法利用代码生成向可靠性计算脚本映射;

3) 利用导航模型的特定域仿真功能, 通过 API 执行步骤二中生成的外部模型进行快速仿真。如立方体顶面的椭圆框所示, “底事件 A” 图模型对象所映射生成的可靠性计算脚本为 “push!(bn,DiscreteCPD(:A,[0.991536,0.008464]))”, 括号内的内容 “A” 和 “[0.991536,0.008464]” 即为第一步中 KARMA 语言对此对象形式化的属性, 其余为固定模板; 通过调用可靠性计算工具的 API 运行该脚本, 获得最后的可靠性分析结果;

4) 如果分析结果不理想, 可以对系统模型进行修改, 直接利用导航模型驱动该系统模型的代码生成和仿真步骤即可完成快速迭代分析过程, 而无需修改其他模型, 提升设计效率。

2.5 本章小结

本章针对当前方法论与系统架构设计深度融合的问题, 提出了一种复杂装备系统导航式建模技术。复杂装备系统导航式建模技术旨在为方法论的流程建模提供标准手段, 并为架构设计提供便利, 提高架构设计的自动化水平, 减少设计人员重复劳动。首先, 从 KARMA 语言建模工具和 MBSE 设计流程出发, 分析了导航式建模技术的设计需求。然后, 对导航式建模技术的实现进行分析, 提出了结合数据建模和元对象机制的技术框架, 保障了导航式建模语言术语、元模型定义的全面性和一致性。最后, 探讨了所提建模技术在工作流, 结果表明导航式建模技术能够在整个系统设计周期提供支持。

第3章 面向导航式建模的元模型设计

3.1 引言

在 MBSE 中，元模型是构建模型的基本元素，例如 SysML 就是用于系统建模的一系列元模型的集合。因此，为了解决方法论的模型要素缺乏专用的建模规范，定制化开发受限于工具厂商自研模板的问题，首先要关注导航式建模元模型的设计，从模型层面实现对方法论的描述。

基于所提技术框架，本章的元模型设计分为概念数据模型设计和逻辑数据模型设计两部分，而逻辑数据模型是概念数据模型的扩展。首先，分析了元模型的设计技术。然后，结合 MBSE 中有关于方法、过程的国际标准和调研，通过实体-关系图构建概念数据模型的基本元素，并验证其对于设计需求的满足性。最后，基于 GOPRR-E 元模型和业务流程建模标注开展元建模，在概念数据模型的基础上添加流程建模的细节，形成完整的导航式建模元模型。

3.2 元模型设计技术

3.2.1 实体-关系图

实体关系建模的概念在 1976 年由 Peter C. 提出，服务于数据库的设计。实体关系图（Entity-Relationship Diagram, ERD）是实体关系建模的手段，用图形化的方式描述特定知识领域中相互关联的事物，在软件工程领域用于定义和描述业务领域中的流程和需要的数据或信息结构^[70]。在一般情况下，E-R 图表示由业务流程监控和指导的实体和事件的记录，而不是流程本身。

E-R 图由实体、属性和关系组成。实体是对系统关注点的分类，比如学校里的实体包括学生、课程、教师等，绘制形式为框；属性是指实体的属性，比如学生的姓名、性别、年龄等；关系是指实体之间的关系，比如学生和课程之间的选课关系，教师和课程之间的授课关系等，绘制形式为线。通过绘制 E-R 图可以帮助设计人员更好地理解数据库的结构和功能，以及实体和实体之间的关系，还可以帮助设计人员更好地设计数据库，以及更好地管理数据库。

数据建模的概念数据模型和 E-R 图之间存在密切的关联。概念数据模型是一种抽象的数据模型，根据业务需求，概念数据模型描述了数据库中的实体、属性和实体之

间的关系。而 E-R 图则是一种图形化的表示，它可以用来描述概念数据模型中的实体、属性和实体之间的关系。因此，利用 E-R 图能够准确地表达数据建模过程中对概念数据模型的设计。

因此，本章采用 E-R 图对 M2 层的概念数据模型展开设计，承接复杂装备系统导航式建模的业务需求，统一、准确地表达导航式建模的术语，为后续开发元模型奠定基础。

3.2.2 GOPPRR-E 元元模型

为了实现复杂装备系统导航式建模技术框架 M2 层元模型的创建，本文对 M3 层采用的元元模型展开研究。目前有多种元元模型适用于元对象机制，并对元模型构建提供了支持，主要包括 ARIS、Ecore、GME、MS DSL Tools、Visio 和 GOPPRR-E。

ARIS 是一种用于企业建模的元建模方法。用户只能修改已经存在的元模型，而供应商可以创建全新的元模型。ARIS 的元元模型包括对象定义类型、关联定义类型、模型等；Ecore 是 Eclipse 建模框架中的元元模型，该框架支持 Eclipse 应用程序的开发。Ecore 与元对象机制强相关，包括 E 类、E 包、E 引用等元元模型；通用建模环境（Generic Modeling Environment, GME）是主要用于电气工程领域特定领域建模的工具，具有第一类对象、原子、属性、集等元元模型；微软特定域语言工具（Microsoft Domain-Specific Language Tools, MS DSL Tools）允许在 Visual Studio 中定义语言和生成器，元建模是隐式描述的，元元模型包括领域类、领域关系、语言等；Visio 是一个通用的建模和数据可视化工具，元模型定义是通过模板实现的，其元元模型为模板、母版和属性；图、对象、点、属性、角色、关系和扩展（GOPPRR-E）是北京理工大学研发的 KARMA 语言建模工具中使用的元元模型，支持典型的特定域建模任务，如（元）建模和代码生成。

在 Kern H. 对部分元元模型分析的基础上^[71]，本研究添加了对 GOPPRR-E 元元模型的分析，开展元元模型表达能力的对比研究。第一个方面考察元建模概念中可用的第一类元素，第一类建模概念中存在依赖关系多的元元模型排名靠前，结果为：GOPPRR-E > GME > MS DSL Tools > ARIS > Ecore $\equiv$ Visio；第二个层面考察关系的定义，关系模式数量多的元元模型排名靠前，结果为：GOPPRR-E > GME > DSL Tools > ARIS > Ecore > Visio；第三个层面考察元建模中重用、模块化的概念，对比不同元元模型的继承能力，结果为：GOPPRR-E > GME > Ecore > MS DSL Tools > ARIS。

综合上述对比，GOPPRR-E 元元模型是表达能力和抽象能力最强的元元模型。从

具体的定义看，它提供了六种元元模型以及其间的规则和约束，支持描述更多领域元模型之间的关联和约束关系。这六种元元模型可分为属性元素和非属性元素两类：前者指属性元模型，用来描述非属性元素的性质，不能单独存在；而后者包括图、对象、点、角色、关系等。元素之间的关系如图 3.1。

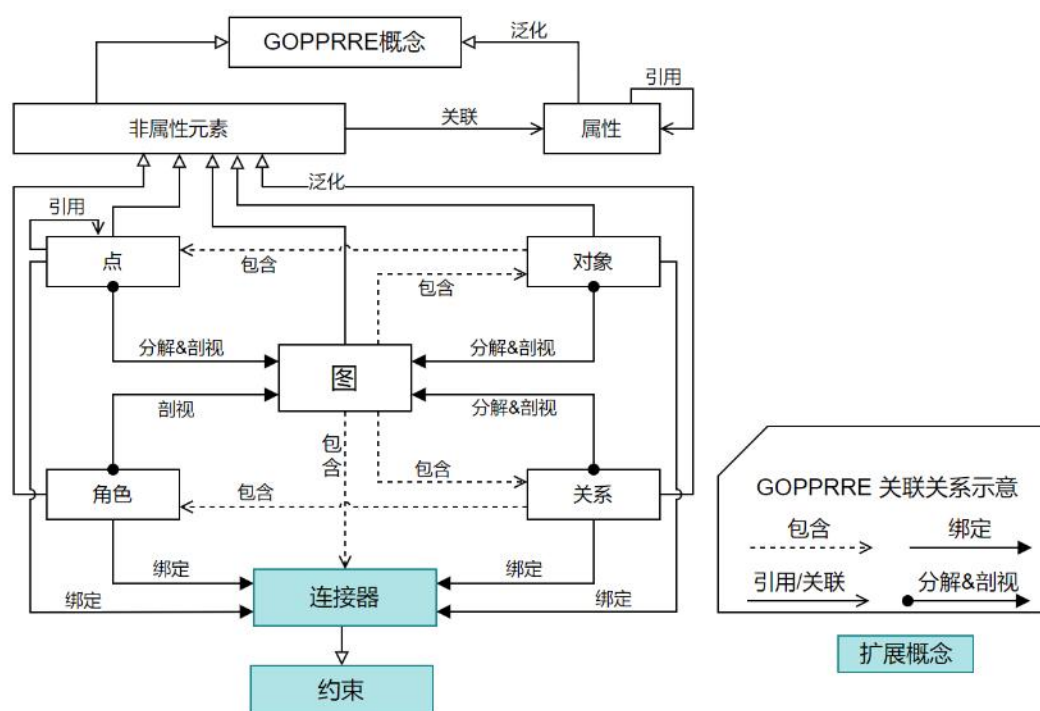


图 3.1 GOPPRR-E 元元模型

（1）图元元模型

图元元模型是由对象、关系类型和连接器（约束）组成的容器，其中对象和关系元元模型包含在其中，而图元元模型还可以与属性元模型相关联。连接器是由对象、点、角色和关系组成的，形成绑定关系。

（2）对象元元模型

对象元元模型定义一组或一类对象，每个对象具有相同的特征，可以独立于关系元元模型单独存在，并在可视化模型中以矩形等符号表示。对象元元模型是图元元模型的主要元素，与点元元模型存在包含关系，并通过关联属性元元模型增加了对象的语义。

（3）关系元元模型

关系元元模型定义了对对象之间的连接方式，是图元元模型中的另一个重要元素，与角色元元模型存在包含关系，与属性元元模型存在关联关系。在建模域中，对象模

型被连接起来，并被定义为参与对象模型的笛卡尔积子集。

（4）角色元元模型

角色元元模型定义了对象元元模型如何连接关系元元模型，并与属性元元模型建立关联关系。由于 GOPRR-E 支持角色概念，因此关系通过角色间接连接对象。其中，GOPRR-E 概念中的关系元元模型的一些特征被继承到了角色元元模型中。此外，角色的标识符必须在元模型对应实例的关联中是唯一的。

（5）点元元模型

点元元模型与属性元元模型建立关联关系，允许对象通过角色连接的附加语义或约束进行匹配。点元元模型用于限定某个对象可以被连接的关系的角色元模型。

（6）属性元元模型

属性元元模型是其他元元模型元素的属性，在建模域中可以包含具体值。最重要的功能是定义元数据类型，如常规数据类型（如字符串、整数等）和元模型类数据类型（如对象数据、图数据等）。

（7）扩展

扩展概念用于描述六种元元模型之间的关联关系，并说明其规则和约束。这些扩展概念包括包含、绑定、分解、剖视和引用/关联。其中，包含描述元元模型之间的组合关系，如将对象和点组合在一起。绑定描述了对对象、点、角色和关系元元模型之间的约束方式，用于定义元建模语言的规则。一个对象的点可以对应多个角色，每个角色至少对应一个对象的点，每个关系绑定两个角色。分解是指约束一个对象（或关系、点）元模型在建模域可以分解为一个图实例。剖视是指约束一个对象（或关系、点、角色）元模型在建模域中可以从不同视角映射为多个图实例。引用/关联定义了点或属性元模型的引用，并指示对象、关系等如何与属性连接。

综合对元元模型的对比、分析研究，GOPRR-E 元元模型的描述能力最强，并且扩展定义了元元模型的关联关系，尤其是其中对剖视和分解的定义，全面支持了导航式建模元模型的设计。因此，本研究采用 GOPRR-E 元元模型作为 M3 层的元元模型开展复杂装备系统导航式建模元模型开发。

3.3 概念数据模型设计

为了充分描述方法论并对复杂装备系统导航式建模元模型的设计提供完备且统一的术语概念，开展复杂装备系统导航式建模的概念数据模型设计，研究导航式建模

需要覆盖的业务需求和实体，采用的设计工具为 E-R 图。

首先，对方法论自身的概念展开研究，方法论的概念与过程、方法和工具紧密相关。Martin^[72]对过程、方法和工具的定义为：

过程（Process）是按照逻辑顺序执行的一系列任务，以实现特定目标。过程主要定义了“做什么”，但并没有规定执行每个任务的具体方法。通过提供多个级别的聚合结构，过程可以在不同的细节级别上进行分析和定义，以满足不同的决策需求。

方法（Method）由执行任务的技术组成，通常定义了每个任务“怎么做”。在不同的级别上，过程、任务都是通过方法执行的。每种方法本身也是一个过程，用于执行特定方法的一系列任务。这意味着在一个抽象层次上定义的“怎么做”，在下一个较低层次上就变成了“做什么”。

工具（Tool）的正确使用可以提高任务的效率。工具的主要目的是促进“怎么做”的完成。从更广泛的角度来看，工具可以增强“做什么”和“怎么做”这两个方面。大多数用于支持系统工程的工具都是基于计算机或软件的，也被称为计算机辅助工程工具。

在此基础上，方法论的定义为相关过程、方法和工具的集合，是对一类问题的相关过程、方法和工具的应用。方法和过程本质的区别在于设计人员看待问题的抽象层次，因此方法本质上也是一个过程。考虑到概念数据模型设计的简洁性和统一性，本研究将方法视为过程的一种，即导航式建模需要描述方法论的内容为过程和工具。

因此，如图 3.2 所示，导航式建模元模型的概念数据模型的基础实体为过程和工具。



图 3.2 基础实体关系图

其次，基于 2.2.2 部分三维结构的需求分析结果，对过程实体和工具实体进行展开分析。

（1）过程实体剖析

如图 3.3 所示，ISO/IEC/IEEE 15288:2015 中对过程的结构进行了定义，过程的实施依赖于活动、任务和备注。所有过程都具有目的和结果，并且每个过程至少包含一个活动，而活动是将相关任务分组的结构。活动提供了一种查看过程中相关任务的方

法，以改善过程的理解和沟通。每个活动至少包含一个任务，而任务是实施过程的详细规定。备注表达了对潜在的实施或使用范围的解释，用于更好地描述过程的意图。因此，本文将过程、活动、任务和备注都定义为实体。考虑到当只存在一个任务时，任务本身相当于活动。从设计人员构建方法论流程模型的角度看，如果针对本质相同的同一种元素反复建模会降低设计效率，因此这种情况下无须对任务进行分组，在概念数据模型的实体关系图中将任务设计为可以单独存在的实体。具体的关系为，每个过程包含 N ($N \geq 1$) 个任务和活动，每个活动包含 N ($N \geq 1$) 个任务，每个任务包含 N ($N \geq 0$) 个备注。

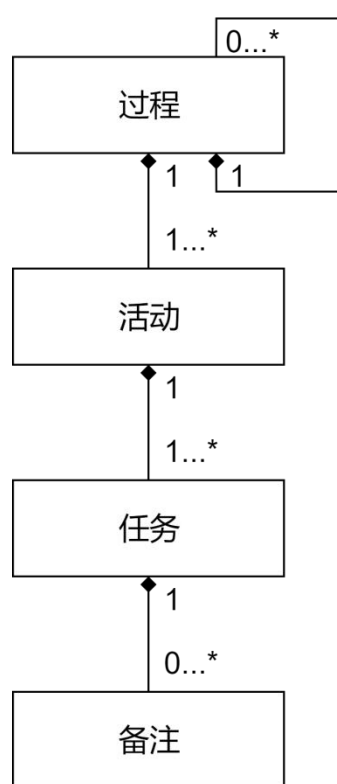


图 3.3 ISO/IEC/IEEE 15288:2015 中过程的结构

根据 2.2.2 中 MBSE 的三维模型，设计人员在执行 MBSE 设计活动的过程中涉及多种层级、多种阶段、多种步骤，其中关键的需求是对多种层级、多种阶段和多种步骤的设计进行有效划分，通过这种划分可以提升方法论流程模型的可读性和可理解性，清晰地明确特定活动、任务是由哪些研究人员或哪个设计阶段完成。

本研究提出将视角（Viewpoint）和视图（View）作为两个实体，对方法论设计过程中涉及的活动、任务进行有效划分。参考 ISO/IEC/IEEE 42010:2011 的定义，视角是描述系统工程中的利益相关者的不同需求或关注点的一组元素、约束和方法，是一

种系统化的思考和描述方式。视角可以看作是一种框架或模板，用于定义系统的各个方面，例如需求、功能、行为、可靠性、安全性等。视图是对视角的实例化，是从特定视角下对系统进行建模的结果，可以为不同的利益相关者提供不同的信息，帮助他们理解和管理系统。视图可以包括多个元素，如数据流、组件、接口等，用于表示系统某个方面的特定信息，是一个抽象的表达，用于表示系统的一部分。

综上所述，视角描述了系统工程中的问题和关注点，视图是对视角的实例化。在 MBSE 中，通过使用视图和视角来划分系统工程活动不同方面的元素是可行的。在 E-R 图中，视图、视角以及其他过程实体的关系为：每个视角包含 N 个视图、 N 个活动和 N 个任务 ($N>0$)，每个视图包含 N 个任务和 N 个活动。

(2) 工具实体剖析

在 MBSE 活动中，应用工具的主要目的就是推进“怎么做”的完成，因此在 MBSE 活动中的各类设计需求就是工具实体的具体表现。根据 2.2.2 中分析的导航式建模设计需求，主要包括不同层级，不同领域的仿真、架构设计、模型转换、行为仿真、需求管理、方案可视化和指标验证等需求。

在实际需求的基础上，结合 KARMA 语言工具基础，设计具体的工具实体。KARMA 语言建模工具包含 KARMA 语言工具和 KARMA 外部扩展工具两部分，用于支持系统生命周期活动的设计。KARMA 语言工具支持利用模型库构建的图元模型开展系统架构模型设计，支持架构模型之间的模型转换快速生成，支持架构模型向其他非 KARMA 格式模型的代码生成，支持行为模型的混合状态机仿真，支持架构模型属性信息的指标验证。KARMA 外部扩展工具提供一系列辅助设计功能，支持 ReqIF 标准数据格式的需求数据管理，支持自由构建表、构建设计结构矩阵（矩阵的横向和纵向的元素一致）、构建关系映射表单（矩阵的横向和纵向的元素可以不一致）进行系统元素的设计、管理和追溯，支持构建甘特图开展项目规划，支持生成桑基图、关系图和树状映射图以可视化的方式快速展示设计方案。

上述 KARMA 语言工具和 KARMA 外部扩展工具基本能满足导航式建模工具的应用需求，但是对于领域仿真方面支持不足，例如不支持建模工具与体系推演仿真拉通。因此，本研究除开展对导航式建模和语言的研究，还将研究导航式建模系统与各类领域仿真的打通，包括体系仿真、多物理场仿真和 Simulink 仿真。

综上所述，工具实体的扩展包括图元模型、架构驱动、代码生成、混合状态机仿真、指标验证、设计结构矩阵、需求管理、表、关系映射表单、甘特图、桑基图、关

系图、树状映射图、体系仿真、多物理场仿真和 Simulink 仿真。因为任务是执行设计活动的最小单元，所以上述工具实体存在的关系为每个任务执行 $N(N \geq 0)$ 个工具扩展实体。最终设计的概念数据模型 E-R 图如图 3.4 所示。

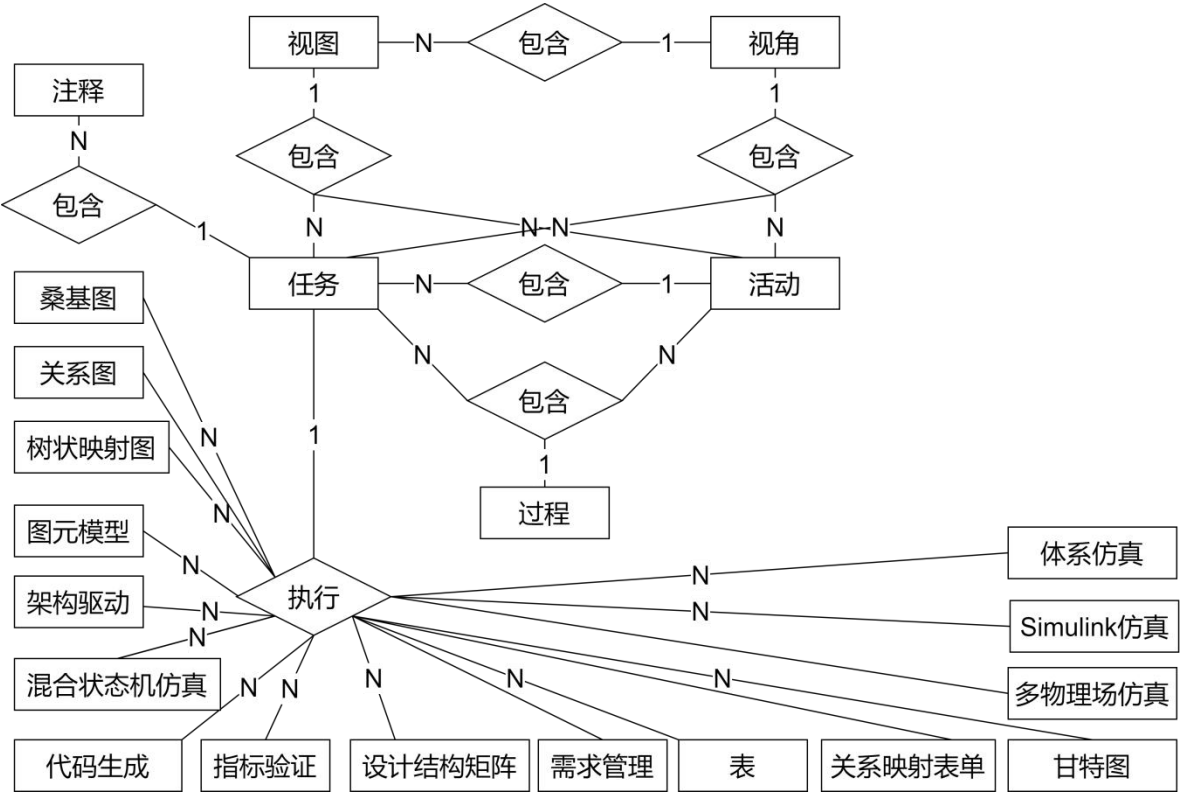


图 3.4 面向导航式建模的概念数据模型

为了保障概念数据模型完整地满足了设计需求，构建实体到导航式建模系统设计需求的追溯矩阵，对每个实体进行跟踪和验证。如表 3.1 所示，全部需求都与实体产生了至少一条追溯关系，验证了概念数据模型设计的完整性和对需求的满足性。

表 3.1 实体到系统需求的追溯矩阵

需求	架构设	模型转	行为仿	指标验	需求管	领域仿	方案可
实体	计 (13)	换 (8)	真 (7)	证 (7)	理 (10)	真 (9)	视化 (9)
过程 (7)	追溯	追溯	追溯	追溯	追溯	追溯	追溯
活动 (7)	追溯	追溯	追溯	追溯	追溯	追溯	追溯
任务 (7)	追溯	追溯	追溯	追溯	追溯	追溯	追溯
备注 (7)	追溯	追溯	追溯	追溯	追溯	追溯	追溯
视图 (7)	追溯	追溯	追溯	追溯	追溯	追溯	追溯

视角 (7)	追溯	追溯	追溯	追溯	追溯	追溯	追溯
图元模型 (1)	追溯						
架构驱动 (2)	追溯	追溯					
代码生成 (1)		追溯					
混合状态机仿真 (1)			追溯				
指标验证 (1)				追溯			
设计结构矩阵 (2)	追溯				追溯		
需求管理 (2)	追溯				追溯		
表 (2)	追溯				追溯		
关系映射表单 (2)	追溯				追溯		
甘特图 (1)	追溯						
多物理场仿真 (1)						追溯	
Simulink 仿真 (1)						追溯	
体系仿真 (1)						追溯	
桑基图 (1)							追溯
关系图 (1)							追溯
树状映射图 (1)							追溯

3.4 逻辑数据模型设计

3.4.1 业务流程建模标注

由于逻辑数据模型的设计实际上对应了导航式建模的元模型，目的是使设计人员能够通过建模的形式将方法论的流程模型化，而 E-R 图表达的概念数据模型主要从业务角度出发，目的是满足业务需求，缺乏特定表达流程或专门服务于流程建模的实体概念，因此引入业务流程建模标注 (Business Process Modeling Notation, BPMN) 作为对建模要素的补充。

BPMN 是一种图形化的业务流程建模语言，用于描述业务流程中的活动、事件、网关和流程之间的连接关系，不仅可以很好地满足各部门之间理解和管理业务流程及相互关系的需求，而且可以从高度抽象的业务流程框架中获取到所需要的要素并将其分解为更细致的业务流，从而提高企业工作的效率和效果^[73]。

BPMN 包括三种基本元素：流程元素、泳道元素和连接元素。流程元素包括业务

流程中的任务、活动、事件和网关，用于表示流程中的各种状态和流程顺序。泳道元素用于将流程分组或者指定任务的所有者或责任人。连接元素用于表示业务流程中各个元素之间的连接关系。具体含义如下：

- 1) 活动：活动是业务流程中的一个步骤，包括任务、事件、消息和子流程等。
- 2) 连接器：连接器用于连接活动，包括流程线、控制线、消息线和异常线等。
- 3) 对象：对象是业务流程中的一个实体，包括数据对象、资源对象和组织对象等。
- 4) 池：池是一个容器，用于定义业务流程的范围，可以是一个组织、一个系统、一个部门等。
- 5) 泳道：泳道是一个容器，用于定义业务流程的步骤，包括任务、事件、消息等。

BPMN 的主要目的是提供一种通用的、标准化的方法，使业务流程的设计、管理和实现更加可靠、可重用、易于理解和共享。许多业界软件厂商和业务流程管理软件都支持 BPMN，并将其作为其建模语言之一。同时，BPMN 可以与其他相关技术和标准（如 BPEL、XML、UML 等）结合使用，从而提供更加丰富和灵活的流程建模环境。

3.4.2 基于 GOPPRR-E 的元模型开发

本节对上述过程实体、工具实体以及 BPMN 中提取的关键流程元素进行抽象，利用 GOPPRR-E 元元模型设计元模型。

在 E-R 图中，过程实体包含了活动和任务，是最顶层的元素。它描述了整个方法论的过程，通过构建不同的活动、任务和任务的实现，实现对方法论流程的描述。如表 3.2 所示，依据 GOPPRR-E 元元模型，利用图元元模型将过程设计为图元模型。为了贴合 MBSE 对图表的描述，将过程重新定义为方法论流程图。

表 3.2 导航式建模的图元模型

元模型	元元模型	含义
方法论流程图（过程）	图	用于描述方法论的流程，包含组成方法论流程模型的全部实体元素及其之间的关系。

除了过程实体外，E-R 图中的其余实体都依据 GOPPRR-E 元元模型映射为对象。由于这些实体要素是从业务表达的角度设计，从流程的角度看，这些要素具备一定的

描述能力，但缺乏指导流程设计的能力，不足以组成完备的元模型。此外，概念数据模型只是一系列概念，如何将概念数据模型有效转化为图形化表示的模型也是本节研究的一个重点。因此，结合对 BPMN 的研究，抽象其核心业务流程描述模型，定义实际的图形化表达范式，完成导航式建模元模型的构建。

首先，提取 BPMN 与概念数据模型中的相似信息，统一术语描述。BPMN 中存在于活动、任务、道和池的概念。活动实际上是对一类模型描述的一般术语，其具体涵盖的元素类型包括任务和子流程。任务是原子性的，即最基本的活动，不可再拆分；子流程是非原子性的，是一个复合的活动。这二者的概念与概念数据模型中的任务和活动的概念一致，因此将 BPMN 任务映射为任务，子流程定义为活动（复合任务）。道和池同属于泳道的一种，类似于概念数据模型中的视角和视图。但是在 BPMN 的定义中，道和池用于描述不同系统参与者的责任，而概念数据模型中的视图和视角是从系统工程理论出发，描述系统的不同方面。因此，针对导航式建模元模型开发，借鉴 BPMN 对道和池的表现形式，定义视图和视角为泳道，其元模型类型为对象。

其次，融合 BPMN 中的其他流程元素。在 BPMN 中除了活动和泳道类元素外，还包括事件类元素、网关类元素和连接对象。事件类元素表达业务流程期间发生的事件，逻辑数据模型提取其中的开始、中间和结束事件作为对象元模型，用来标志方法论的进程；网关类元素用于控制流程的分支和合并，提取其中的排他网关、并行网关、包含网关作为对象元模型，用于方法论流程的分支与合并；连接对象用于将业务流程模型中的各模型元素连接起来形成一个基本的业务流程，提取其中的顺序流、消息流、关联和备注作为关系元模型。

基于 GOPRR-E 元元模型，对实体关系图和 BPMN 进行融合与抽象，获得的对象和关系元模型的设计如表 3.3 和表 3.4 所示（部分元模型含义前文已经展开了介绍，此处不再赘述）：

表 3.3 导航式建模的对象元模型

元模型	元元模型	含义
图元模型、代码生成、架构驱动、混合状态机仿真、指标验证（KARMA 类元素）	对象	见 3.3 概念数据模型设计。
需求管理、表、设计结构矩阵、关系映射表、甘特图（表单类元素）	对象	见 3.3 概念数据模型设计。
桑基图、关系图、树状映射图（知识图谱	对象	见 3.3 概念数据模型设计。

类元素)		
视角、视图、活动（泳道类元素）	对象	见 3.3 概念数据模型设计。
开始	对象	表示流程的起点。
中间	对象	表示子流程的终点。
结束	对象	表示流程的终点。
任务	对象	见 3.3 概念数据模型设计。
排他网关	对象	根据条件，将流程分支成多个可能的路径， 其中只有一个路径可以被选择执行。
并行网关	对象	将流程分成多个并行的路径，所有路径同 时执行。
包含网关	对象	根据条件，允许多个路径被选择执行。

表 3.4 导航式建模的关系元模型

元模型	元元模型	含义
顺序流	关系	表示流程中不同元素的执行顺序。
消息流	关系	表示流程中不同元素之间的消息传递关系。
关联	关系	表示流程中不同元素之间的关联关系，如任务需要使用一个 代码生成。
备注	关系	表示流程中不同元素与备注之间的连接关系。

根据 GOPPRR-E 元元模型的定义，关系元模型需要依赖角色元模型才能实现和对象元模型的连接及方向约束。基于上述关系元模型定义，角色元模型的定义如表 3.5 所示：

表 3.5 导航式建模的角色元模型

元模型	元元模型	含义
顺序流始端	角色	连接顺序流关系和任务/活动/事件类/网关类对象。
顺序流终端	角色	连接顺序流关系和任务/活动/事件类/网关类对象。
消息流始端	角色	连接消息流关系和任务/活动/视图/视角/工具类实体对应的对象。
消息流终端	角色	连接消息流关系和任务/活动/视图/视角/工具类实体对应的对象。
关联始端	角色	连接关联关系和工具类实体对应的对象。

关联终端	角色	连接关联关系和任务/活动对象。
备注始端	角色	连接备注关系和备注对象。
备注终端	角色	连接备注关系和其他除备注以外的对象。

逻辑数据模型设计的主要目的之一就是对概念数据模型进行细化，丰富其属性信息，因此利用 GOPRR-E 元元模型对概念数据模型展开属性设计，用于描述在方法论的流程模型设计过程中所表达的详细内容。

由于本研究所设计的导航式建模技术不仅具备描述方法论流程的能力，还具备对整个设计过程中所涉及模型的访问、管理、创建和执行能力，这些功能通过对相关工具类元模型的二次开发实现，具体实现于第 5 章展开。为了配合实际使用过程中对模型的访问、管理、创建和执行工作，需要使特定对象元模型能够明确找到对应的模型文件，因此本研究设计了“文件名称”属性元模型作为标记，为后续技术实现提供支持。

此外，针对 KARMA 语言的特性，KARMA 语言建模工具在设计时强调通用性表达，这对于工具的建模能力和表达是有益的。这一设计理念也体现在了设计结构矩阵、关系映射表单这类 KARMA 语言外部扩展工具上，在构建这类表单具体元素的过程中，工具不提供筛选功能，而是同时提供项目中所有 KARMA 模型中的图元模型、对象元模型、关系元模型、角色元模型、点元模型、属性元模型和需求管理模型供设计人员选择。事实上，在不同的设计阶段，关注的表单元素不同，在面对特定的应用时同时提供全部类型的模型元素会降低表格的设计和查找效率。针对导航式建模的特定使用，通过添加过滤文件类型、过滤文件和过滤元模型的属性，为这类表单的使用提供筛选功能。

其余属性元模型的设计不过多介绍，所设计的全部属性元模型如表 3.6 所示：

表 3.6 导航式建模的属性元模型

元模型	元元模型	含义
名称	属性	表示视角和视图所表达的分析的角度。
文件名称	属性	表示项目中或将要创建的特定文件名称。
任务名称	属性	表示任务执行的具体内容。
图元模型	属性	表示 KARMA 语言模型的图元模型类型，如活动图。
描述	属性	表示备注描述的内容。

过滤文件类型	属性	表示表单过滤的文件类型，包括.karma 类型和.reqif 类型。
过滤文件	属性	表示表单过滤的文件名称。
过滤元模型	属性	表示表单过滤的元模型，例如图元模型。

GOPRR-E 元元模型的扩展用于描述元模型之间的规则和约束，根据本研究中设计的导航式建模的元模型开展规则和约束设计：

（1）包含规则：方法论流程图（图元模型）包含其余所有元模型。

（2）绑定规则：包括关系与角色的绑定和角色与对象的绑定两部分。1）关系与角色的绑定：顺序流两端绑定顺序流始端和终端，消息流两端绑定消息流始端和终端，关联两端绑定关联始端和终端，备注两端绑定备注始端和终端；2）角色与对象的绑定，参见表 3.5。

（3）引用/关联规则：描述对象元模型所引用的属性元模型。引用“名称”属性的对象包括视图和视角；引用文件名称属性的对象包括架构驱动、代码生成、图元模型、指标验证、需求管理、表、甘特图、设计结构矩阵、关系映射表单、关系图、桑基图、树状映射图；引用任务名称属性的对象为任务；引用图元模型属性的对象为图元模型；引用过滤文件类型、过滤文件、过滤元模型三个属性的对象包括设计结构矩阵、关系映射表单；引用描述属性的对象为备注。

（4）剖视规则：根据所设计的概念数据模型特性，视图和视角表达系统工程过程中对特定问题的分析，活动表达多个任务的组成。在不同的抽象层级上，这三个对象元模型本质上都是“过程”或方法论流程图的一种。因此，利用 GOPRR-E 对视图、视角和活动进行扩展定义，定义视图、视角、活动对象元模型到方法论流程图元模型的剖视，设计人员可以在剖视的模型中建模，从而实现模型分区。

综上所述，基于 GOPRR-E 元元模型开发导航式建模元模型如图 3.5 所示。通过 GOPRR-E 元元模型的表达，实现了对概念数据模型和 BPMN 的融合、抽象和细化，为复杂装备系统导航式建模提供了模型手段。

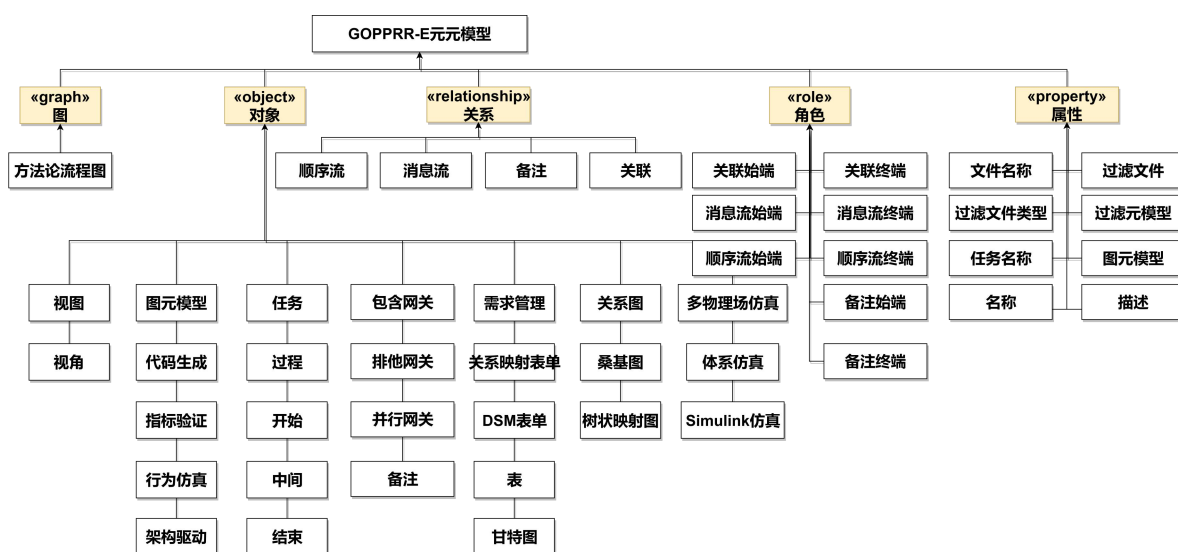


图 3.5 基于 GOPRR-E 的导航式建模的元模型

3.5 案例验证

基于本章所设计导航式建模元模型可以搭建方法论的流程模型,通过一个 DoDAF 案例验证本文所提导航式建模元模型表达方法论流程模型的有效性,并展示其具体的实施流程。

(1) 建立 DoDAF 与导航式建模元模型的映射。流程模型的构建参考成熟 MBSE 建模工具 MagicDraw 中对于 DoDAF 的呈现形式,主要建模工作流用顺序流表示,模型间的输入输出用消息流表示,8 个视角用视角泳道表达,各建模步骤的主要内涵用任务或活动表达,每个任务关联不同类型的视图,这些视图包括图元模型、关系映射表单、甘特图、表。

(2) 在画布界面建模复现 DoDAF 流程模型。如图 3.6 所示,以最上侧的“开始”为起点,蓝色实心箭头为顺序流指导体系架构人员的建模活动流程,即“开始”、AV-1、CV-1、CV-2、CV-4、OV-1、OV-5b、OV-2、SvcV-1、SvcV-4、SV-1、SV-2、SV-4、SV-5a、AV-2、AV-1、“结束”。虚线空心箭头代表消息流,代表该任务所依赖的消息输入,以 CV-2 的构建为例, CV-2 的构建需要 CV-3、CV-5、CV-6 和 PV-3 的消息为输入。

所构建 DoDAF 流程模型与 MagicDraw 建模工具中的示例达到了同等的表达效果。通过将基于文本的方法论模型化,有助于加强设计人员对方法论的理解,驱动复杂装备系统的快速设计与早期验证。

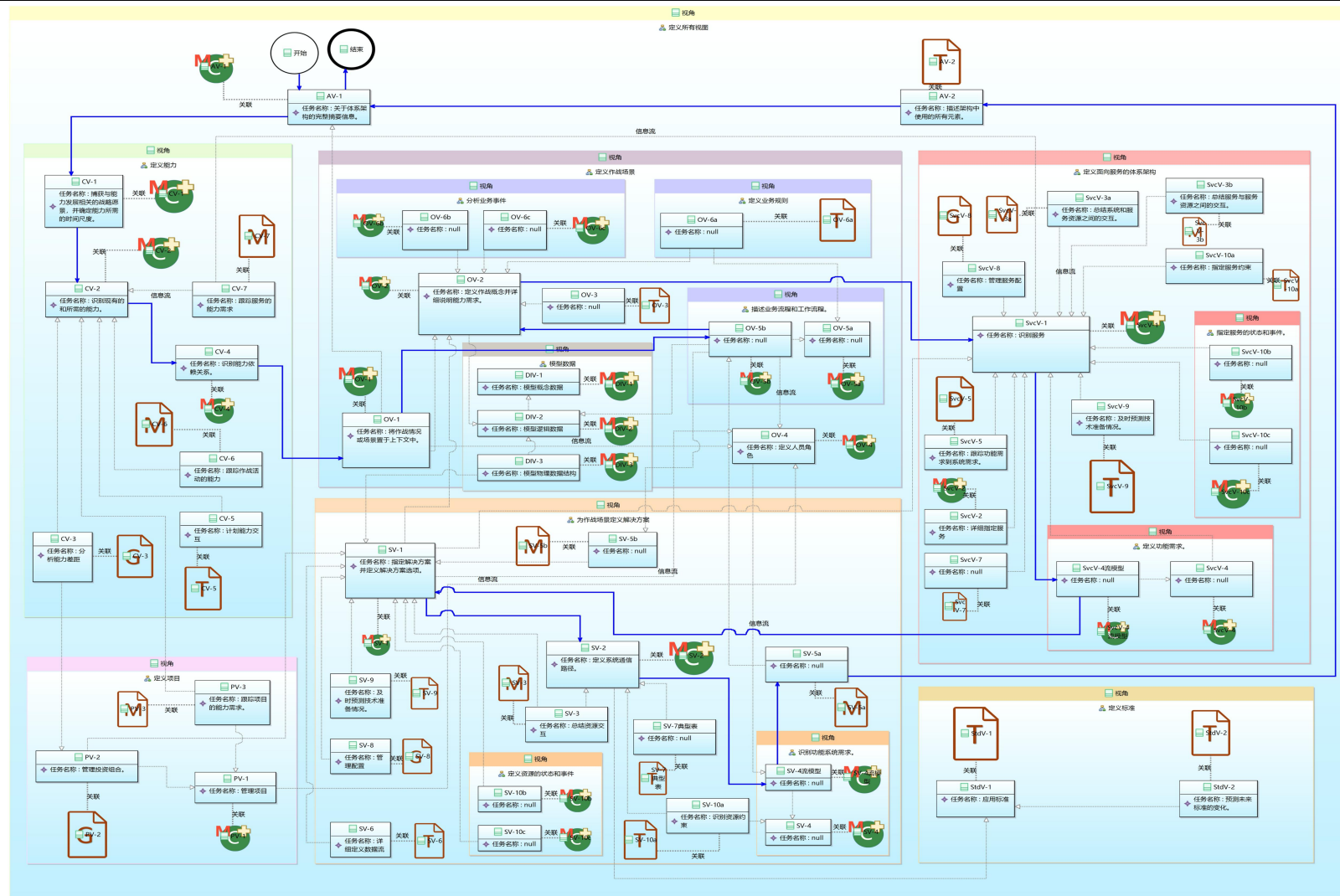


图 3.6 利用导航式建模构建的 DoDAF 流程模型

3.6 本章小结

针对当前方法论的模型元素表达缺乏专用领域模型的问题，本章提出基于元建模构建导航式建模元模型，包括概念数据模型和逻辑数据模型设计两个阶段。概念数据模型基于 E-R 图构建，所有实体通过追溯矩阵追溯到了设计需求，每个设计需求都至少被一个实体所追溯，保障了元模型定义的全面性。在概念数据模型的基础上，逻辑数据模型基于 GOPPRR-E 元元模型构建，提升了导航式建模元模型的表达能力，使其能够充分描述方法论的设计流程。最后，通过一个案例实践了导航式建模元模型的表达能力，并将其与专业建模工具的同类方法论表达模式进行了对比分析。结果表明，本章构建的导航式建模元模型能够有效支持方法论流程的图形化表达。

第4章 复杂装备系统导航式建模语言设计

4.1 引言

尽管元模型能够支持方法论流程的描述，但元模型只是一种表达形式而不具备存储的能力，另一方面根据复杂装备系统导航式建模技术框架，方法论的描述视角除了流程模型外还包括文本式和文件结构式，多种视角配合描述有助于提升方法论表达的深度和设计人员使用的友好性。本章的研究旨在定义导航模型的存储规范和转换机制，以保证三种导航模型能够有效存储、生成和转换，从而维护设计视角的全面性并提升设计效率。

本章基于 XML 开展导航式建模语言的设计，对应技术框架中的物理交换规范，以规范存储、生成和转换格式。首先，研究基于 GOPRR-E 元元模型开发的导航式建模元模型到 XML 的映射机理。其次，根据映射机理定义导航式建模语言，包括抽象语法和具体语法，抽象语法用于表达语言的元素关系，具体语法用于定义语言的具体形式。最后，研究 XML 模型与流程模型、文件树状结构的模型转换机制，实现构建任意一种都能转换生成其他两类模型，提升设计的自动化程度。

4.2 元模型与 XML 转换规则设计

4.2.1 XML

第三章中设计的导航式建模元模型能够为设计人员构建方法论的流程模型提供支持，但是对于所构建模型的存储形式，也就是物理交换规范没有明确定义。XML 是一种用于存储、传输和重构任意数据的标记语言和文件格式，在数据建模中常用于实现数据的存储^[74]。XML 的关键结构包括标记、元素和属性，具体含义如下所示：

（1）标记

标记是一种以<开头和以>结尾的标记结构。标记分为三种类型：开始标记，例如<methodology>；结束标记，例如</methodology>；空元素标记，例如<methodology />。XML 的标记与 HTML 的标记类似但存在不同，不同之处在于 XML 标记是自定义的，可以根据具体需求进行扩展和定义。这对于本研究中的建模语言设计具有重要价值。

（2）元素

元素以开始标记开始并以匹配的结束标记结束，或者仅由空元素标记组成。开始

标记和结束标记之间的字符是元素的内容，并且可能包含由标记或其他元素组成的多个子元素。例如：`<methodology>MOFLP</methodology>`或`<methodology />`。

（3）属性

属性由存在于开始标记或空元素标记中的“名称-值”对组成。例如，`<methodology name="MOFLP" />`，其中属性的名称是“name”，值是“MOFLP”。每个属性只能对应一个值，每个属性在每个元素上最多出现一次。

XML 模式定义（XML Schema Definition, XSD）是一种 XML 的模式文档，用于定义 XML 文档的结构和内容。XSD 可以指定 XML 文档中包含哪些元素，以及这些元素的数据类型、取值范围等等。通过 XSD，可以使 XML 文档的结构更加清晰、易于理解和处理。XSD 定义了一些基本的数据类型，如字符串、整数、日期。同时，XSD 还支持用户自定义数据类型和元素的复杂类型，如序列、选择和重复。通过这些复杂类型的组合，可以描述更为复杂的数据结构。

因此，本章利用 XSD 约束 XML 的结构和内容，从而表达逻辑数据模型到 XML 的映射，实现导航式建模语言的设计，使设计人员构建的方法论流程模型存储为规范、可读和易于数据交换的文本文件。

4.2.2 GOPPRR-E 元元模型到 XML 的映射

由于逻辑数据模型是通过 GOPPRR-E 元元模型构建的，所以研究 GOPPRR-E 元元模型到 XML 的映射模式是开发导航式建模语言的关键。基于元对象机制，GOPPRR-E 元元模型与 XML 的映射如图 4.1 所示：

M3 层为 GOPPRR-E 元元模型，这与 XML 的基本结构元素和属性对应。通过定义不同的标记和不同的元素嵌套实现对 GOPPRR-E 元元模型中非属性元元模型的表达，例如对象（Object）元元模型映射为`<object />`。属性（Property）元元模型与 XML 属性的概念一致，可以直接表达为`<object property="" />`；

M2 层为利用 GOPPRR-E 元元模型构建的元模型，这一级别与 XSD 对应。元模型通过定义 GOPPRR-E 元元模型之间的约束和规则构建而来，具有一定的语义、语法。XSD 构建的过程与之类似，需要制定 XML 文档中包含的元素、数据类型的规范；

M1 层为利用元模型构建的模型，这一级别与 XML 文档文件对应。在 MBSE 中，模型的构建需要遵循元模型的规范。而在 XSD 的规范下，XML 文档的生成同样需要遵循其规范，否则将不能通过文档规范性验证。

M0 层为描述的视角，在本研究中无论是导航式建模语言还是 XML 文件，其目

的都在于描述开发复杂装备系统的方法论，只是其存在形式不同。

综上所述，通过构建 GOPPRR-E 元元模型到 XML 的映射，论证 XML 作为导航式建模语言的可行性，为导航式建模语言设计奠定理论基础。

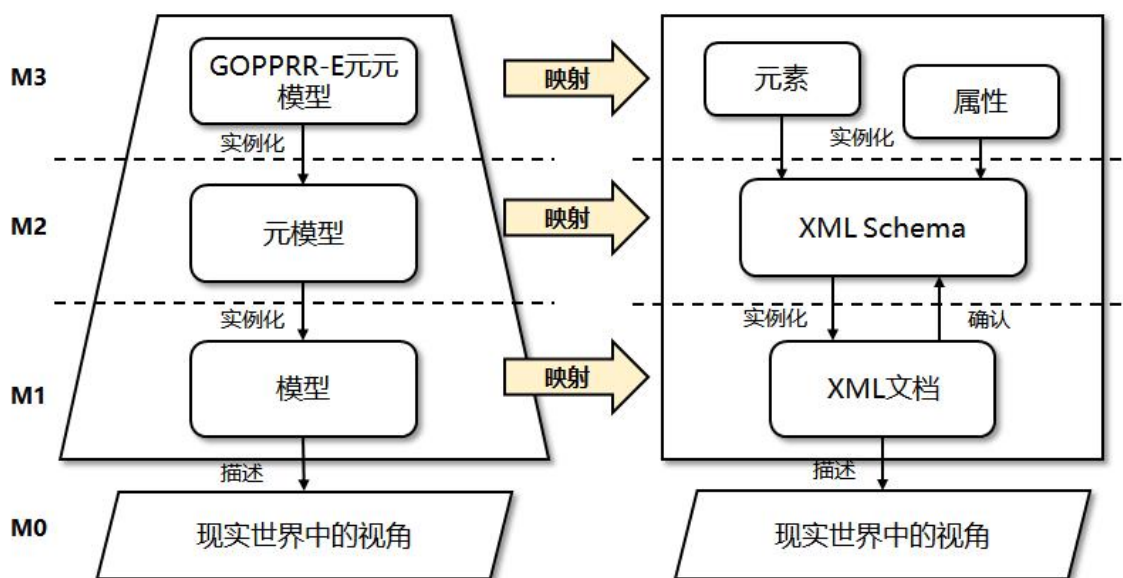


图 4.1 GOPPRR-E 元元模型与 XML 的映射规则

4.3 物理交换规范设计

作为 XML 模型，导航式建模语言（即物理交换规范）本质用于模型的物理存储，在对方法论流程的表达方面与图形化的方式存在差距，因此在描述上侧重于轻量化，注重对方法论的结构信息展开描述，充分利用 XML 语义自明性的优势。本章基于 XML 开展导航式建模语言的设计，分为抽象语法和具体语法两部分，其含义如下：

抽象语法是程序语言语法结构的一种抽象表示形式，是在具体语法基础上进行抽象和简化得到的。抽象语法通常只关注程序语言中的重要语法结构和语义，忽略具体语法中的细节和底层实现。抽象语法是程序语言设计和程序分析中常用的概念，可以帮助程序员更好地理解程序语言的语法结构和语义。

具体语法是指程序语言中真正被计算机所理解和执行的语法结构，通常是由编译器或解释器进行解析和转换的语法结构。具体语法包含了程序语言中的所有语法元素，如关键字、运算符、变量、常量。为了确保计算机精确地理解和执行程序语言中的每个元素，具体语法通常是非常具体和细节化的。

4.3.1 抽象语法

抽象语法依照 XML 的语法规则在导航式建模元模型的基础上映射，如图 4.2 所示。本文用 UML 类图表达导航式建模语言的抽象语法，图中包括三类元素：1) 类，在图中用一个矩形表示，其中包括类的名称、属性和方法。类映射到 XML 语言元素的概念，例如最顶层的类对象代表导航式建模语言的根节点为物理交换规范 `<physical>` 标签，并且拥有 `name`（名称）属性；2) 组合关系，在图中用一个菱形加实线表示，菱形表示整体对象，实线表示组成部分对象。组合关系指一个类对象包含另一个类对象，被包含的对象是组成类对象的一部分，也就是整体和部分之间的关系。组合关系是一种强关联关系，表示整体对象拥有其组成部分，而且组成部分不能存在于其他整体对象中；3) 泛化关系，在图中用一个空心三角形加实线表示，空心三角形指向父类，实线连接子类和父类。表示一个类是另一个类的特殊形式，也就是子类和父类之间的关系。子类继承了父类的属性和方法，并且可以添加自己的属性和方法。上述两个关系，对应所设计的导航式建模语言中元素之间的关系，例如建模方法（`<methodology>`）元素由对象元素和泳道元素组成，其中泳道元素继承自建模方法，拥有 `localLabel` 属性和与建模方法一致的元素关系。

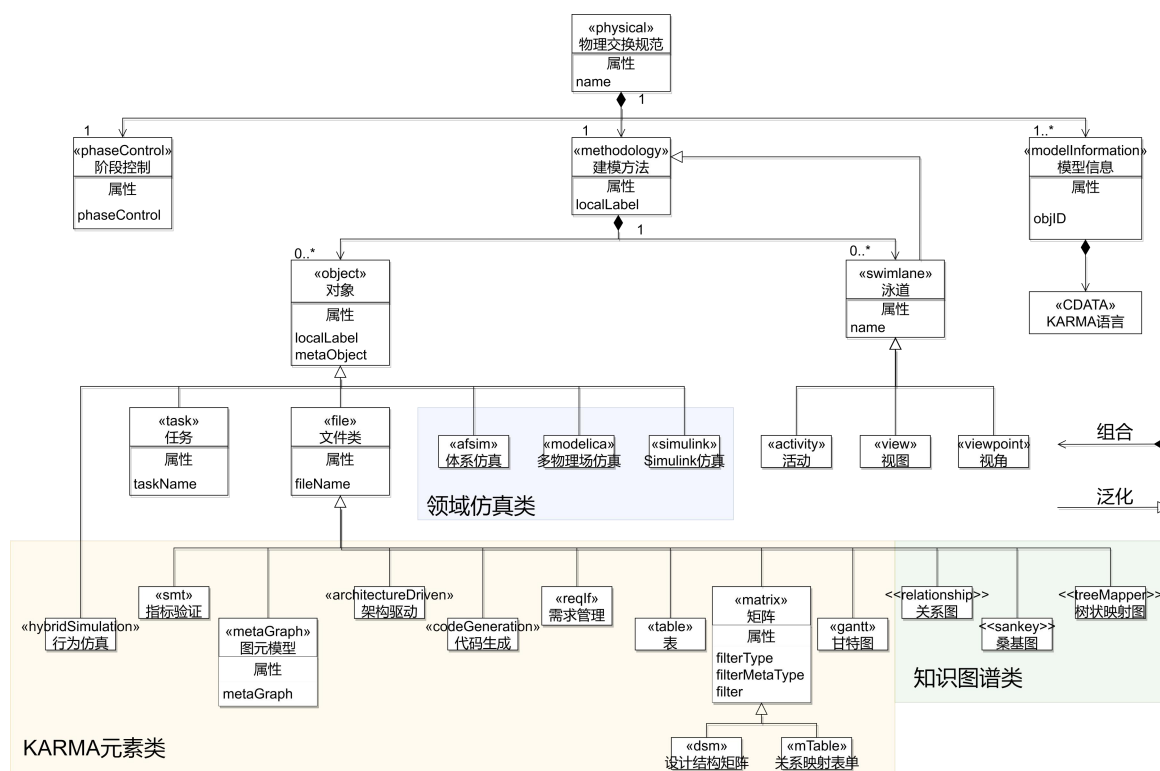


图 4.2 导航式建模语言抽象语法

导航式建模语言的根节点为物理交换规范，标记定义为<physical>，根节点作为整个模型存储的基础，具有属性 name，表示此 XML 模型文件的名称。根节点由阶段控制、建模方法和模型信息组成，这几类元素共同完成方法论各类信息的存储。

（1）建模方法<methodology>

建模方法映射元模型中方法论流程图的概念，每个 XML 模型文件存在一个建模方法元素。建模方法节点包括一个属性<localLabel>，与元模型中方法论流程图的本地名称一致，描述该建模方法的名称。建模方法元素节点由对象元素和泳道元素组成，原因如下：1）尽管泳道在元模型中是对象的一种，但是为了在 XML 模型中清晰地表达方法的层级结构，在语法中将泳道元素与对象元素并列；2）由于 XML 模型文件是文本形式，其表达任务之间逻辑关系的能力弱，因此建模方法元素节点重点表达方法论的层级结构和具体内容，没有将其他元模型映射为建模方法元素节点的子元素。

1）对象<object>

根据组合关系的定义，每个建模方法允许存在大于等于零个对象。对象作为一个较高层级的父类元素，与逻辑数据模型中构建的对象元模型一一映射。但是同上所述，XML 模型重在表达结构，因此不包括对象元模型中表达逻辑关系的对象，如开始、结束。

对象具有 localLabel 属性和 metaObject 属性，分别表示该对象的本地名称和其元模型类型 ID。这样设计的目的如图 4.3 所示，导航式建模系统中每一个构建的模型元素的 localLabel（本地名称）都可以被修改，方便设计人员对元素命名。在修改了默认标签的情况下，仅凭 XML 文本无法分辨出该元素具体对应的模型元素，因此增加 metaObject 标签用于指明其元类型。



图 4.3 指标验证对象的属性信息（示例）

尽管抽象语法中的对象类与逻辑数据模型的对象元模型建立了映射关系，但是其

基本设计模式与逻辑模型面向过程、功能的设计模式不同，其设计思路是面向对象的，充分利用 XML 的优势，实现对特定子类的重用和扩展，提升设计效率。因此，本研究对逻辑数据模型的对象元模型进行抽象，设计了对象类及其子类的层级结构，所有子类都是从对象类泛化而来，具有对象类的属性。根据对象元模型是否具有文件名称属性区分，不存在文件名称属性的对象包括行为仿真、体系仿真、多物理场仿真、Simulink 仿真和任务，其中任务单独具有任务名称属性，因此扩展了该属性；具有文件名称的对象泛化为文件类，包括指标验证、图元模型、架构驱动、代码生成、需求管理、矩阵、甘特图、关系图、桑基图、树状映射图。其中图元模型类还扩展了图元模型为属性，矩阵类还扩展了过滤文件类型、过滤元模型和过滤文件为属性，设计结构矩阵和关系映射表单继承了矩阵类。

2) 泳道<swimlane>

泳道含有三个子类，视图、视角和活动，每个 XML 模型文件至少存在零个泳道。视图、视角和活动继承了建模方法元素，因此这三个元素均可作为父元素拥有子元素嵌套。视图、视角和活动对建模方法元素的继承过程相当于在逻辑数据模型设计过程中的剖视定义，这也是对元模型的一次映射。

(2) 阶段控制<phaseControl>

阶段控制元素是导航式建模语言中额外定义的新元素，每个 XML 模型文件含有一个阶段控制元素。基于 2.4.2 设计的导航式建模工作流，方法论流程图模型在使用过程中需要分应用阶段控制其读写。通过单独建立特定模型对象控制整个模型对象的读写并不可行，因为在锁定模型画布的情况下，无法再通过改变对象模型来解锁流程模型画布。因此，本研究将阶段控制元素设计在导航式建模语言的 XML 模型中，通过修改存储文件中的阶段控制标记信息控制流程模型的只读和设计状态。

(3) 模型信息<modelInformation>

模型信息元素是导航式建模语言中额外定义的新元素。模型信息用于表达整个画布中对象、关系的位置布局、大小和外观，是利用模型转换将 XML 模型文件转换为图形化的模型的重要元素。每个导航式建模的 XML 模型文件至少含有一个模型信息元素，保存方法论流程图模型自身的信息。可能存在多个模型信息元素的原因在于，流程图模型中往往存在多个泳道，而泳道作为剖视图模型也需要被存储。

模型元素由一个 CDATA 元素组成，也就是 XML 中的纯文本数据，该元素用于盛放特定模型的 KARMA 语言模型（由流程模型形式化而来）。由于 KARMA 语言

语法较为复杂，所以这部分采用可以包含任何字符，包括特殊字符（如<、>、&等），而不会被解析器解析的 CDATA XML 块，避免解析过程中出现障碍。

模型信息可以具有一个对象名称。导航式建模语言中定义视图、视角和活动这类元素是对建模方法元素（<methodology>）的继承，对应于元模型中构建的其与方法论流程图的剖视关系。在实际建模活动中，这类剖视关系的构建是基于对象的，即视图、视角和活动本身是一个对象，而剖视图模型创建于这些对象中。因此，为了实现剖视关系的信息描述，需要获得每个剖视图所属的对象名称属性，从而将方法论流程图中的每一个剖视图模型与其对应的对象关联。而方法论流程图自身不具有对象名称属性，主要原因在于其自身并非由对象剖视而来，而是作为该 XML 模型文件的最高层模型存在。

4.3.2 具体语法

基于 XML 模式定义导航式建模语言的 XML 标记如表 4.1 所示，共 23 种。

表 4.1 导航式建模语言关键字

physical	methodology	modelInformation	task	afsim	modelica
activity	view	hybridSimulation	viewpoint	smt	metaGraph
simulink	codeGeneration	architectureDriven	reqIf	table	dsm
mTable	gantt	relationship	sankey	treeMapper	

基于 XML 模式定义的语法规则，本研究设计了导航式建模语言的具体语法，接下来对其主要部分展开介绍：

（1）物理交换规范<physical>

物理交换规范作为一个复杂类型（在 XSD 语法中，复杂类型表示不只拥有文本的类型），必须按顺序包含 phaseControl、methodology 和 modelInformation 三个子元素（其中，阶段控制和建模方法子元素均有且只有一个，模型信息子元素至少有一个），并必须具有属性 name。具体语法如下：

```

1.  <xsd:complexType name="physical">
2.  <xsd:sequence>
3.  <xsd:element name="phaseControl" type="mbse:phaseControl"
4.  minOccurs="1" maxOccurs="1">
5.  </xsd:element>
6.  <xsd:element name="methodology" type="mbse:methodology" minOccurs="1"
7.  maxOccurs="1"></xsd:element>
8.  <xsd:element name="modelInformation"
9.  type="mbse:modelInformation" minOccurs="1" maxOccurs="unbounded">

```

```

10. </xsd:element>
11. </xsd:sequence>
12. <xsd:attribute name="name" type="xsd:string" use="required"></xsd:attribute>
13. </xsd:complexType>

```

(2) 阶段控制<phaseControl>

阶段控制是物理交换规范的子元素，该元素必须具有阶段控制属性，用于控制导航模型的应用阶段。属性值限制为“dev”开发阶段和“app”应用阶段两个字符。具体语法如下：

```

1. <xsd:complexType name="phaseControl">
2.   <xsd:attribute name="phaseControl" use="required">
3.     <xsd:simpleType>
4.       <xsd:restriction base="xsd:string">
5.         <xsd:pattern value="(dev|app)"></xsd:pattern>
6.       </xsd:restriction>
7.     </xsd:simpleType>
8.   </xsd:attribute>
9. </xsd:complexType>

```

(3) 模型信息<modelInformation>

模型信息是物理交换规范的子元素，可以具有对象名称属性。由于模型信息含有的 KARMA 语言信息是 CDATA 格式，而在 XSD 中没有相应的约束定义，因此通过定义该类型为 mixed（混合），使模型信息类型允许包括 CDATA 的文本格式。

```

1. <xsd:complexType name="modelInformation" mixed="true">
2.   <xsd:attribute name="objID" type="xsd:string" use="optional"></xsd:attribute>
3. </xsd:complexType>

```

(4) 建模方法<methodology>

建模方法是物理交换规范的子元素，必须具有本地名称属性。建模方法包含 0 到无穷个名为 allObjects 的组，该组包含全部对象元素和泳道元素，表示所有对象元素和泳道元素都是建模方法的子元素，并且可以重复出现。组的具体内容在（7）介绍。

```

1. <xsd:complexType name="methodology">
2.   <xsd:sequence>
3.     <xsd:group ref="mbse:allObjects" minOccurs="0"
4.       maxOccurs="unbounded">
5.     </xsd:group>
6.   </xsd:sequence>
7.   <xsd:attribute name="localLabel" type="xsd:string" use="required"></xsd:attribute>
8. </xsd:complexType>

```


(5) 泳道<swimlane>

泳道是建模方法的子元素，是一个抽象的、用于继承的元素。泳道继承了建模方法的元素定义，即所有对象元素和泳道元素自身都可以是泳道的子元素。另外，泳道被视图、视角和活动继承，这三个子元素是具体的元素类型，可以作为标记出现在 XML 模型实例文件中。

```

1. <xsd:complexType name="swimlane">
2.   <xsd:complexContent>
3.     <xsd:extension base="mbse:methodology">
4.       <xsd:attribute name="name" type="xsd:string"/></xsd:attribute>
5.     </xsd:extension>
6.   </xsd:complexContent>
7. </xsd:complexType>

```

(6) 对象<object>

对象是建模方法的子元素，必须具有本地名称和元对象两个属性，是一个抽象的、用于继承的元素。由抽象的对象元素可以泛化出全部具体对象元素。

```

1. <xsd:complexType name="object">
2.   <xsd:attribute name="localLabel" type="xsd:string" use="required"/></xsd:attribute>
3.   <xsd:attribute name="metaObject" type="xsd:string" use="required"/></xsd:attribute>
4. </xsd:complexType>

```

以图元模型为例，该元素是一个具体的对象元素，是 XML 模型实例文件的标记。图元模型继承自文件类，而文件类继承自对象。图元模型还对继承做出了扩展，增加了图元模型属性

```

1. <xsd:complexType name="metaGraph">
2.   <xsd:complexContent>
3.     <xsd:extension base="mbse:file">
4.       <xsd:attribute name="图元模型" type="xsd:string"/></xsd:attribute>
5.     </xsd:extension>
6.   </xsd:complexContent>
7. </xsd:complexType>

```

(7) 组<allObjects>

组在 XML 模型定义中用来定义一组元素，以便在多个位置重复使用。为了便于建模方法、泳道对全部具体元素的引用，设计 allObjects 组用于管理这些元素。allObjects 通过 choice 元素声明，指定在组中的元素只能选择一个出现。建模方法元素对组的引用限制了其出现频次为 0-∞，结合此处对组的定义，这表示建模方法允许

引用组中任意元素为子元素，且无顺序和数量限制，泳道类元素与组的引用关系同理。

```

1.  <xsd:group name="allObjects">
2.    <xsd:choice>
3.      <xsd:element name="subProcess" type="mbse:subProcess"></xsd:element
   >
4.      <xsd:element name="view" type="mbse:view"></xsd:element>
5.      <xsd:element name="viewpoint" type="mbse:viewpoint"></xsd:element>
6.      <xsd:element name="hybridSimulation" type="mbse:hybridSimulation"><
   /xsd:element>
7.      <xsd:element name="task" type="mbse:task"></xsd:element>
8.      <xsd:element name="afsim" type="mbse:afsim"></xsd:element>
9.      <xsd:element name="modelica" type="mbse:modelica"></xsd:element>
10.     <xsd:element name="simulink" type="mbse:simulink"></xsd:element>
11.     <xsd:element name="metaGraph" type="mbse:metaGraph"></xsd:element>
12.     <xsd:element name="codeGeneration" type="mbse:codeGeneration">
13.       </xsd:element>
14.     <xsd:element name="architectureDriven" type="mbse:architectureDrive
   n"></xsd:element>
15.     <xsd:element name="smt" type="mbse:smt"></xsd:element>
16.     <xsd:element name="reqIf" type="mbse:reqIf"></xsd:element>
17.     <xsd:element name="table" type="mbse:table"></xsd:element>
18.     <xsd:element name="gantt" type="mbse:gantt"></xsd:element>
19.     <xsd:element name="dsm" type="mbse:dsm"></xsd:element>
20.     <xsd:element name="mTable" type="mbse:mTable"></xsd:element>
21.     <xsd:element name="relationship" type="mbse:relationship">
22.       </xsd:element>
23.     <xsd:element name="sankey" type="mbse:sankey"></xsd:element>
24.     <xsd:element name="treeMapper" type="mbse:treeMapper">
25.       </xsd:element>
26.   </xsd:choice>
27. </xsd:group>

```

4.4 语义解析及模型转换

基于复杂装备系统导航式建模技术框架，本研究所设计的导航式建模涉及三类模型，分别为实例化的方法论流程模型、XML 模型和文件结构模型，因此本节重点讨论这三类模型的模型转换技术。其中，方法论流程模型和文件结构模型的转换通过 XML 模型介质实现，从而将六类模型转换简化为四类模型转换。

4.4.1 模型转换需求

本研究所设计的导航式建模系统建立于 KARMA 语言建模工具的基础上，共有一套图的构建和编译系统，在该模式下图模型与 KARMA 语言具备双向转换的能力。然而，本文基于 XML 设计的导航式建模语言旨在结构化地描述设计人员所构建的方法论结构，其所需要的信息无法与 KARMA 语言产生直接的映射，具体原因在于：

(1) 从模型文件意义上看，KARMA 语言模型本质上用于描述图的结构信息，例如有哪些对象，哪些关系，布局和外观等，而不能显性地描述图的内容，如论文由正文和其他部分组成。XML 模型旨在提供一个直观的方法论结构视图，用于直接描述方法论中涉及的模型的层级结构，例如描述论文的章节。

(2) 从文件机理上看，在 KARMA 语言建模工具框架下，剖视涉及多文件。例如，在一个图（也就是一个 KARMA 语言文件）中构建了对象，而这个对象可以剖视到其他图，并在当前图中展现其他图的模型信息，这个过程会额外创建另一个图的 KARMA 文件用于存储剖视图。因此，图模型及其包含的多个剖视图，包括嵌套的剖视图的信息分别存储于多个文件。这与通过一个导航模型文件控制全局模型的概念违背。

因此，本节在 KARMA 语言与图模型相互转换的基础上，研究如何在 XML 中兼容多组 KARMA 语言文件信息，并实现图模型、XML 和文件结构的转换。这样的好处在于，避免对图模型复杂的原生计算机数据开展模型转换的研究，而且 KARMA 语言提供了良好的数据模型接口便于数据操作。最终，实际的模型转换工作包括，XML 模型与 KARMA 语言（多文件）的相互转换、文件结构模型与 XML 模型的相互转换。这里给出后续设计的基础定义，如图 4.4：方法论流程模型同时存储主模型和剖视图模型，主模型是方法论流程图，其 ID 与导航模型文件名称一致。而主模型中可能存在泳道类元素，这些元素对应的模型为剖视图模型，泳道类元素嵌套的泳道类元素也为剖视图模型。

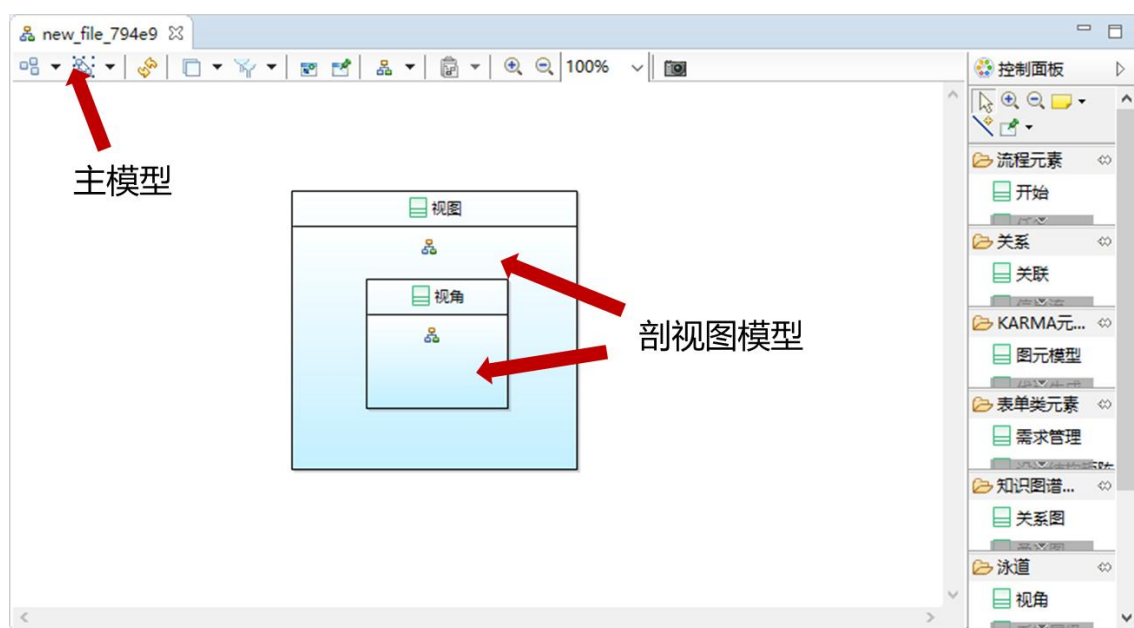


图 4.4 导航式建模的主模型和剖视图模型

4.4.2 代码生成器

如图 4.5 所示，代码生成器用于将方法论流程模型和文件结构模型生成 XML 模型。

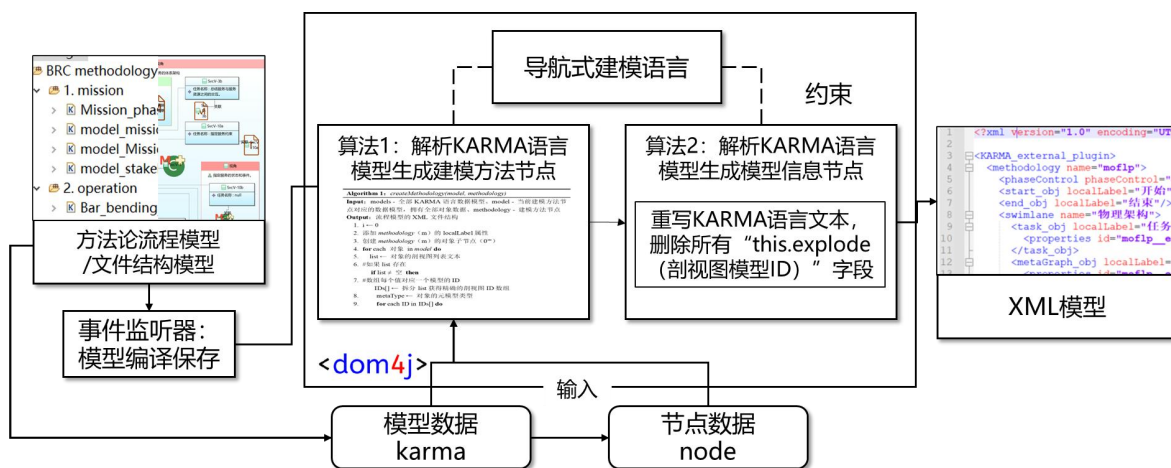


图 4.5 代码生成器 workflow

(1) 定义代码生成器的触发机制

由于方法论流程模型和文件结构模型是供设计人员使用或查看的，在查看的过程中并不会对模型本身产生修改，如果对这些模型进行实时的代码生成，将会占用大量资源，降低导航式建模系统的使用效率。因此，代码生成器的触发定义在设计人员对方方法论流程模型或文件结构模型做修改并保存的时候。

(2) KARMA 语言生成

导航式建模构建的流程模型和文件结构模型都需要首先转换为统一的 KARMA 语言格式，然后才能被本节设计的代码生成器所读取并解析。对于导航式建模构建的流程模型，其本身基于 KARMA 语言建模平台，因此其模型数据可以自动转换为 KARMA 语言描述。而文件结构模型属于文件节点类型的数据，不能自动转换，因此本节主要研究文件结构模型到 KARMA 语言的映射。

从模型设计目的的角度，文件结构模型主要用于表达模型的层级结构，这与 XML 模型的描述一致。由于 KARMA 语言同样是建立在 GOPPRR-E 元元模型之上开发的建模语言，因此本研究通过构建元模型与文件结构模型的映射，实现其与 KARMA 语言的映射。如图 4.6 所示，图映射为文件夹，与文件相关的对象映射为文件，也就是将方法论流程图映射为文件夹，视角、视图和活动作为方法论流程图的泛化同样映射为文件夹，其余关联文件的对象映射为文件。

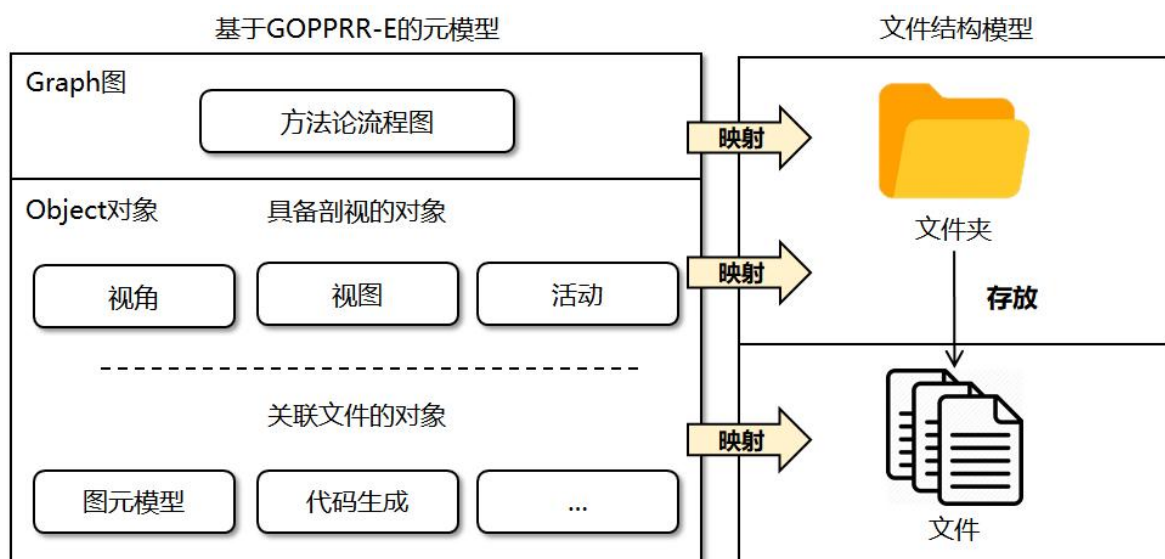


图 4.6 元模型与文件结构模型的映射规则

通过定义代码模板实现文件结构模型到具体 KARMA 语言的生成。实际上在代码生成的过程中，有大量用于描述元元模型引用信息、外观的代码，而与文件结构模型相关的只有该文件的文件名称、本地名称、文件类型和文件层级关系。因此设计代码模板可以有效对映射目标模型进行封装，减少重复工具提升代码生成效率。

本研究设计的文件结构模型到 KARMA 语言的代码模板主要分为文件夹类和文件类。如下所示，文件夹的代码生成模板以泳道类对象——视图为例，文件的代码生成模板以文件类对象为例，其中<>表示需要填充到代码模板里的内容。此外，由于

通过文件结构创建的模型对象，不具备充分的布局参数（映射至方法论流程模型中的具体位置、大小），因此有关位置参数均采用默认。

泳道类是具有剖视的特殊对象，因此其代码生成模板包含视图的对象和剖视图两部分。对象作为图的一部分，该部分代码需要通过文件夹的层次结构查找对应的上级图模型，并添加至其内部。<对象 ID>在设计人员创建文件夹后按一定规律随机生成。因为泳道类的对象用于盛放剖视的图模型，所以在第 14 行需要生成其剖视图的文件名称及其相对位置。

```

1.      Object <对象 ID> refer lane_obj //视图对象元模型
2.
3.      annotation(
4.          initialLocation=(0,0,0,0), //对象元模型于画布的坐标，默认值
5.          localLabel="视图",
6.          shape="rectangle",
7.          color="0,0,0;0,0,0;0,0,0;194,239,255;255,255,255",
8.          screenMode=decompositionMode,
9.          imageAddress="",
10.         SDIdentification="",
11.         text=("Microsoft YaHei UI",8,Normal,Normal),
12.         iconDisplay=true,
13.         frame=(1,"solid"),
14.         visualizationList="<剖视图 ID>:<剖视图坐标>",
15.     ); //剖视关系
16.     end <对象 ID>;

```

泳道类对象对应的剖视图模型单独作为一段空白的 KARMA 语言模型存在，其模型 ID 随机生成，本地名称表示为设计人员创建的文件夹名称。当设计人员在这个文件夹下添加文件时，相应地就会在这个空白 KARMA 语言模型中添加特定对象，这与前文在图中添加泳道类对象的模式一致。

```

1.      partial Model <modelID> instanceof methodologyProcess_graph //方法论流程图
2.
3.      annotation(
4.          localLabel="<文件夹名称>",
5.      );
6.     end <modelID>;

```

文件类对象和泳道类对象的代码生成模板几乎一致，相当于简单的泳道类对象，因为其不具备剖视规则，所以不涉及第 14 行剖视图列表和图元模型的创建。与泳道类对象不同的是，文件类对象代码生成模板额外具有对象属性，设计人员创建的文件

名对应<value>值。

```
1. Property pro_id = "<value>" refer metaObjPro_id annotation(localLabel = "文件名称",color="0,0,0",text=("Microsoft YaHei UI",7,Normal,Normal),underline=false,strikeThrough=false);
```

(3) XML 生成

XML 模型的代码生成分为 KARMA 语言处理和 XML 构建两部分,其中 KARMA 语言处理用于提供 XML 构建需要的全部信息。采用的 XML 解析生成工具为 DOM4J,相对于其他 XML 解析工具,选择 DOM4J 的原因在于其性能高。DOM4J 采用了一种基于缓存和懒加载的内存管理机制,可以大幅减少内存占用和提高性能。同时,DOM4J 使用 SAXReader 来解析 XML 文档,相比其他 DOM 解析器更快速并易于使用,而且具有高度可读性和高度可扩展性。

根据定义的 XML 语法规则,具体的 XML 文件涉及复杂的嵌套,具体构建流程遵循以下步骤:

1) 构建根节点(一级)。根节点具有属性 name,值来自导航模型文件名。

2) 构建阶段控制节点(二级),初始值为“dev”。方法论流程模型的阶段控制由 XML 文件自身决定,因此阶段控制节点的属性不会因方法论流程模型的改变而改变,因此除非手动编辑此处,该值不会被修改。

3) 构建建模方法节点(二级),定义该节点为标记符号 m。建模方法节点的属性值来自对主模型的 KARMA 语言的 localLabel 的解析。

4) 构建当前层级节点的全部对象子节点(三级及以上)。这些子节点通过解析当前节点对应的图模型的对象部分得到,建模方法对应主图,泳道类节点对应各剖视图。第 4 步的执行永远在建模方法节点或泳道类节点之后执行,用于构建这些节点的下一层级的文件类对象节点,如图元模型子节点。

5) 构建当前层级节点的全部泳道类子节点(三级及以上),这些子节点通过解析当前节点对应的图模型的剖视部分得到。如图 4.7 所示,定义当前层级节点的第 i 个泳道类子节点为 i^M , M 为该子节点的父节点标记符号,例如第三级第一个泳道类子节点为 1^m ,其所生成的第四级子节点为 i^{1^m} ; I 为 M 节点所拥有的泳道类子节点数量, $0 \leq i \leq I$;特殊情况,当 $i = 0$ 时, 0^M 表示父节点 M 的全部对象子节点。根据建模语言语法的定义,泳道类子节点均继承自建模方法节点,因此每构建一个泳道类子节点相当于重新从第 3 步出发,并执行第 4 步和第 5 步,在这个过程中不断深入模型的层级实现节点嵌套,从而结构化地描述方法论的流程模型信息。当第三层的 I^m 节点完

成其最后一个泳道子节点构建，递归结束。

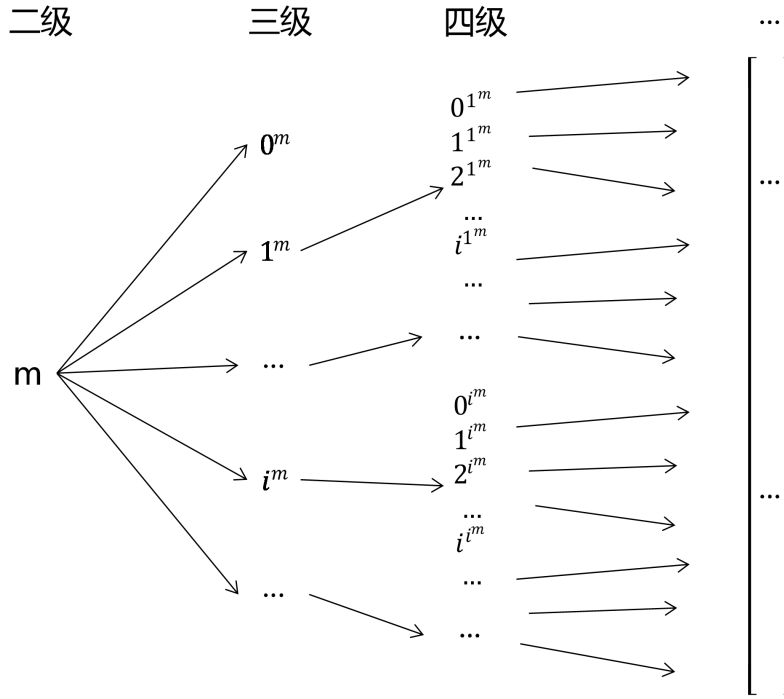


图 4.7 遍历生成泳道类子节点的概念图

第 3-5 步骤中建模方法和泳道类节点代码生成的伪代码如算法 1 所示：

Algorithm 1: *createMethodology(model, methodology)*

Input: models - 全部 KARMA 语言数据模型、model - 当前建模方法节点对应的数据模型，拥有全部对象数据、methodology - 建模方法节点

Output: 流程模型的 XML 文件结构

1. $i \leftarrow 0$
2. 添加 *methodology* (m) 的 localLabel 属性
3. 创建 *methodology* (m) 的对象子节点 (0^m)
4. **for each** 对象 **in** *model* **do**
5. $list \leftarrow$ 对象的剖视图列表文本
6. #如果 list 存在
6. **if** list $\neq$ 空 **then**
7. #数组每个值对应一个模型的 ID
7. $IDs[] \leftarrow$ 拆分 list 获得精确的剖视图 ID 数组
8. $metaType \leftarrow$ 对象的元模型类型
9. **for each** ID **in** $IDs[]$ **do**
10. **if** $metaType =$ 活动 **then**
11. $i^m \leftarrow$ 创建当前节点下的活动子节点
12. **else if** $metaType =$ 视角 **then**
13. $i^m \leftarrow$ 创建当前节点下的视角子节点
14. **else if** $metaType =$ 视图 **then**

15. $i^m \leftarrow$ 创建当前节点下的视图子节点
16. #获得当前 ID 对应的数据模型
 $nextModel \leftarrow \text{get_model_from_models}(ID)$
17. #递归, $nextModel$ 相当于新的 $model$, i^m 相当于新的 $methodology(m)$
18. $createMethodology(nextModel, i^m)$
19. 添加 i^m 的 $name$ 属性
20. $i \leftarrow i + 1$
21. **Return methodology**

6) 构建模型信息节点(二级)。流程模型中每存在一个主模型或剖视图模型都将生成一个模型信息节点, 该节点具有一个属性和一段文本, 对象名称属性来自对应图模型的解析。文本段落用于存放主模型或剖视图模型对应的 KARMA 语言文本内容, 但是首先需要对其进行文本处理。具体为, 重写 KARMA 语言文本, 删除所有“this.explode(剖视图模型 ID)”字段, 该字段定义了该泳道类对象对应的剖视图模型 ID。原因在于 KARMA 语言对于剖视的定义是多文件的, 即每有一个剖视图模型都将生成一个 KARMA 语言文件, 然而实际上导航式建模的体系下只存在一个导航式建模文件, 而不存在上述多个语言文件, 若保留这部分字段将影响语法解释器部分调用 KARMA 语言特定方法对这些模型信息的处理。

4.4.3 语言解释器

如图 4.8 所示, 语言解释器用于将 XML 模型解释为方法论流程模型和文件树状结构模型。

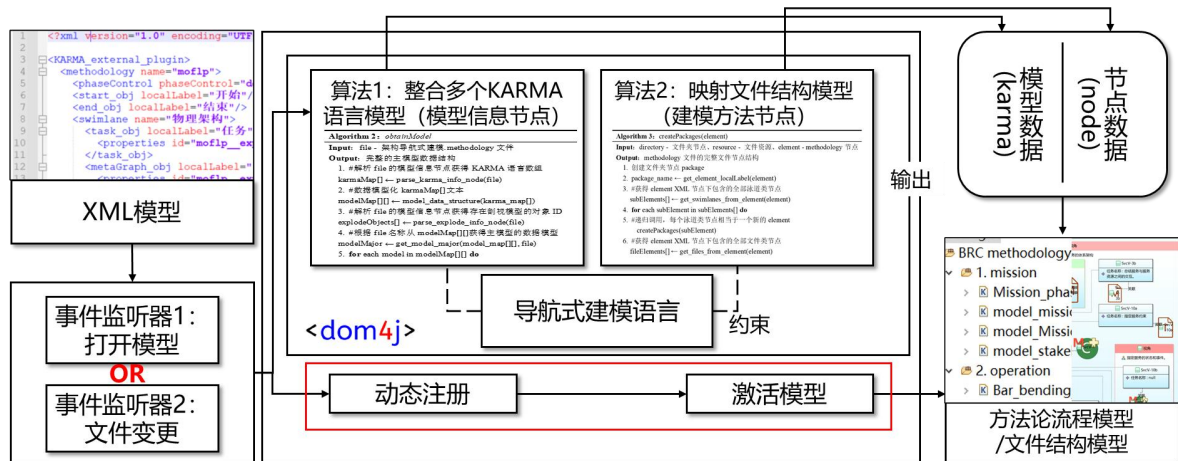


图 4.8 语言解释器 workflow

(1) 定义语言解释器的触发机制

由于语言解释器用于将 XML 模型文件解释为流程模型或文件结构模型, 因此定

义解释器的触发机制为首次打开模型时以及 XML 模型文件内容变更时。

（2）外部库动态注册

这一步是 XML 模型文件向流程模型解释的基础。在 KARMA 语言建模工具中，设计人员所构建的模型必须基于工具项目下已存在并完成注册的特定元模型，包括元模型文件、ecore 和 odesign 数据，元模型文件直接暴露在工具中供设计人员自主开发，后两者根据元模型文件自动生成。第 4 章所开发的元模型即是利用 KARMA 语言建模工具的元模型开发技术开发，作为一个 KARMA 语言项目存在，如图 4.9 所示。

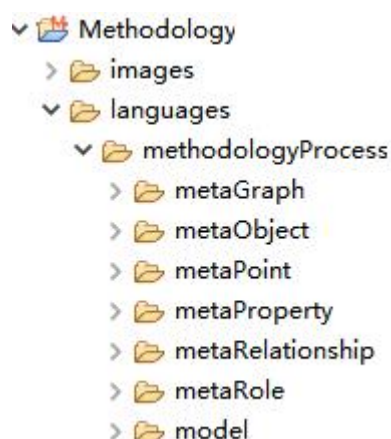


图 4.9 KARMA 语言项目的元元模型结构

从实际业务角度出发，如果用户想要使用导航式建模，则必须先在自己工作的项目中创建导航式建模元模型或由工具将这些元模型导入到项目中，这将导致工作项目中产生多余的元模型文件，影响正常的设计和项目的规划。因此，本研究将导航式建模元模型库作为 KARMA 语言建模工具的外部库，研究外部库动态注册的模式， workflow 如图 4.10 所示。

首先，导航式建模元模型作为文件库必须存在于设计人员的计算机中。本研究对建模工具的安装程序进行了开发，使该项目文件在工具安装过程中自动安装至系统的 AppData（应用数据，计算机软件常用的存放个人数据的目录）文件夹中。

随后，在实际使用过程中一旦触发编译导航模型的操作，系统将添加 ecore 和 odesign 至项目空间中，添加后这二者隐式存在，用户不可见，且保存项目时会被自动清理，这两类文件用于建模系统中导航式建模元模型的注册，注册后不需要特定元模型项目存在，即可在导航式建模系统中建模。除上述两个文件外，元模型文件用于在模型转换过程中为 KARMA 语言文件的解析提供支持。

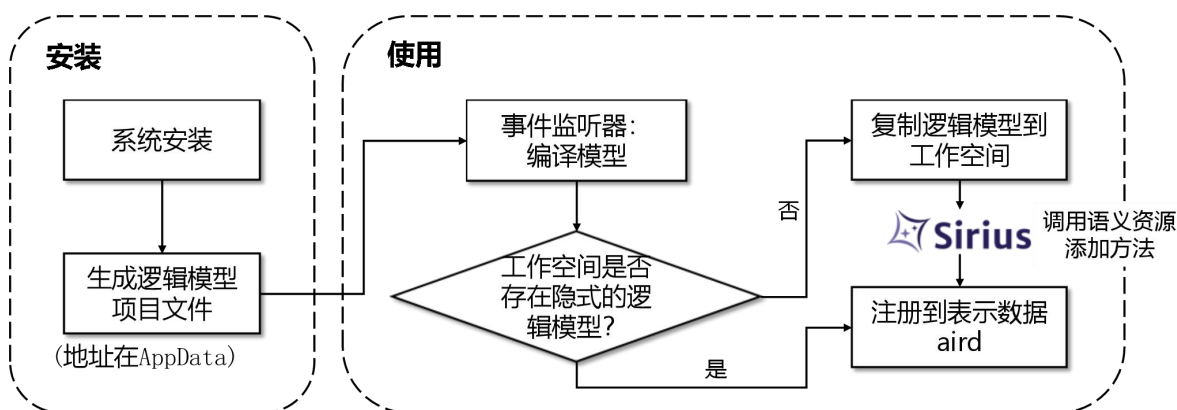


图 4.10 动态注册工作流程

（3）XML 模型解释为流程模型

XML 向流程模型解释的过程所使用的工具同为 DOM4J。这一步的工作就是将代码生成过程中拆分的 KARMA 语言模型组装为一个完整的数据模型，该数据模型由导航式建模系统后台进一步解析并生成流程模型。

1) 解析 XML 模型的模型信息节点

包括以下内容：通过对所有模型信息节点的解析和处理，获得主模型和各剖视图模型与其 KAMRA 语言映射的矩阵；将 KARMA 语言解析为数据模型，获得主模型和各剖视图模型与数据模型映射的矩阵（由于在代码生成中删除了剖视字段，所以数据模型之间并未产生连接关系，也正因为此才能成功解析每个数据结构，避免其解析过程中查找不存在的文件）；从模型信息节点中提取各剖视图对应的对象名称属性；提取主模型的数据模型。上述解析的数据作为后续语言解释的输入。

2) 遍历所有数据模型

每一个数据模型都是一个 KARMA 语言模型的计算机描述。在遍历过程中查找数据模型中是否具有泳道类对象，若具有，则查找该泳道类的剖视图列表（在代码生成中只删除了剖视字段，但仍保留了剖视图列表，二者是两个字段），并将剖视图列表对应的数据模型以剖视的形式添加至该泳道类的数据模型中；若不具有，则继续向后遍历，重复上述流程直至遍历完所有数据模型。这一步的本质是通过读取 XML 的属性信息，将主图和剖视图、剖视图之间相互连接起来，构成完整的模型网络。

3) 返回拼接完成的主模型

遍历所有数据模型将重新建立起各数据模型之间的关联关系，通过该连接关系能从主模型访问到任意模型的数据。最后返回主模型即可驱动导航式建模系统从 XML 生成方法论的流程模型。

第 1-3 步骤的代码生成伪代码如算法 2 所示：

Algorithm 2: *obtainModel*

Input: file - 导航式建模.methodology 文件

Output: 完整的主模型数据结构

```

1. #解析 file 的模型信息节点获得 KARMA 语言数组
   karmaMap[]  $\leftarrow$  parse_karma_info_node(file)
2. #数据模型化 karmaMap[] 文本
   modelMap[][]  $\leftarrow$  model_data_structure(karma_map[])
3. #解析 file 的模型信息节点获得存在剖视模型的对象 ID
   explodeObjects[]  $\leftarrow$  parse_explode_info_node(file)
4. #根据 file 名称从 modelMap[][] 获得主模型的数据模型
   modelMajor  $\leftarrow$  get_model_major(model_map[], file)
5. for each model in modelMap[][] do
6.   for each object_id in explodeObjects[] do
7.   #从 model 中获取对象
       object  $\leftarrow$  get_object_from_model(model, object_id)
8.   # 如果对象存在，则继续
       if object  $\neq$  null then
9.     list  $\leftarrow$  对象的剖视图列表文本
10.    ids[]  $\leftarrow$  拆分 list 获得精确的剖视图数组
11.    for each id in ids[] do
12.    #将 modelMap[id] 对应的剖视模型添加到 object
        object  $\leftarrow$  add_explode_model_to_object(modelMap[id], id)
13. Return modelMajor

```

(4) XML 模型解释为文件结构模型

如图 4.6 所示，XML 模型与文件结构模型有直接的映射关系，建模方法、视图、视角和活动节点映射为文件夹，文件类节点映射为文件，具体解释流程遵循以下步骤：

1) 解析建模方法节点，根据该节点的本地名称，生成文件结构的一级项目文件夹；

2) 解析当前节点的全部文件类对象子节点，定义 $i = 0$ ，根据文件类子节点的文件名称属性，得到各文件子节点的文件名称，例如“代码生成”节点属性为 codeGen，映射在文件结构的文件即 codeGen.karma。同时，按照文件类子节点的类型不同，其文件后缀名和实际访问路径也不同。这一步用于创建文件夹下的文件，设计人员可以从文件结构中直接打开文件；

3) 解析当前节点的第 $i+1$ 个泳道类子节点，泳道类子节点与建模方法节点一致，同样映射为文件夹。定义当前层级的第 i 个泳道类子节点为 i^M ， M 为当前节点的标记符号，例如一级项目文件夹的第一个泳道类子节点（二级文件夹）为 1^m ，其所生成的

第三级第 i 个泳道类子节点为 i^{1^m} ; I 为 M 节点所拥有的泳道类子节点数量, $0 \leq i \leq I$; 特殊情况, 当 $i = 0$ 时, 0^M 表示 M 节点的全部对象子节点。

4) 判断若存在 i^M 子节点, 即根据该子节点的 `localLabel` 生成文件夹, 并且 i^M 作为下一层级的父节点, 并执行第二步, 进入循环; 若不存在 i^M 子节点, 则返回该子节点的父节点 M , 若其为建模方法节点 m , 则循环结束, 否则返回第三步。实际上, 这一步与代码生成中由流程模型生成 XML 的过程类似, 是相反的过程, 即通过高度结构化的 XML 反向映射生成所有文件夹、文件模型。

第 1-4 步骤的代码生成伪代码如算法 3 所示:

Algorithm 3: createPackages(element)

Input: directory - 文件夹节点、resource - 文件资源、element - methodology 节点

Output: methodology 文件的完整文件节点结构

1. 创建文件夹节点 package
 2. package_name $\leftarrow$ get_element_localLabel(element)
 3. #获得 element XML 节点下包含的全部泳道类节点
subElements[] $\leftarrow$ get_swimlanes_from_element(element)
 4. **for each** subElement in subElements[] **do**
 5. #递归调用, 每个泳道类节点相当于一个新的 element
createPackages(subElement)
 6. #获得 element XML 节点下包含的全部文件类节点
fileElements[] $\leftarrow$ get_files_from_element(element)
 7. **for each** fileElement in fileElements[] **do**
 8. fileName $\leftarrow$ fileElement XML 节点的 fileName 属性
 9. metaType $\leftarrow$ fileElement XML 节点的 metaObject 属性
 10. **switch the value of metaType do**
 11. #此处是一个省略, 包含全部文件类型的情况
 case 不同的文件类型
 12. **do** 创建文件节点
 13. #根据不同文件类型, 连接到不同的文件资源
 file $\leftarrow$ link_to_file_location(resource, metaType, fileName)
 14. **break**
 15. **Return** package
-

(5) 阶段控制节点解析

阶段控制节点的解析在生成方法论流程模型之后, 但在该模型打开之前。这是因为受编译机制影响, 一旦触发模型的只读, 系统将不再执行编译活动, 因此若在未编译出方法论流程模型的情况下先执行该节点的只读控制, 方法论流程模型将编译为空, 画布无法打开。

具体的只读控制基于图形化建模框架 Sirius 二次开发, 过程如图 4.11 所示。首先

解析得到阶段控制节点的属性值。然后，在设置应用阶段前，需要先执行该模型是否来自方法论流程模型的判断。这是因为 Sirius 框架下只读和非只读用于控制全局模型，而本研究只需控制当前方法论流程模型只读和非只读。最后，根据阶段控制节点的属性值改写模型的只读状态。

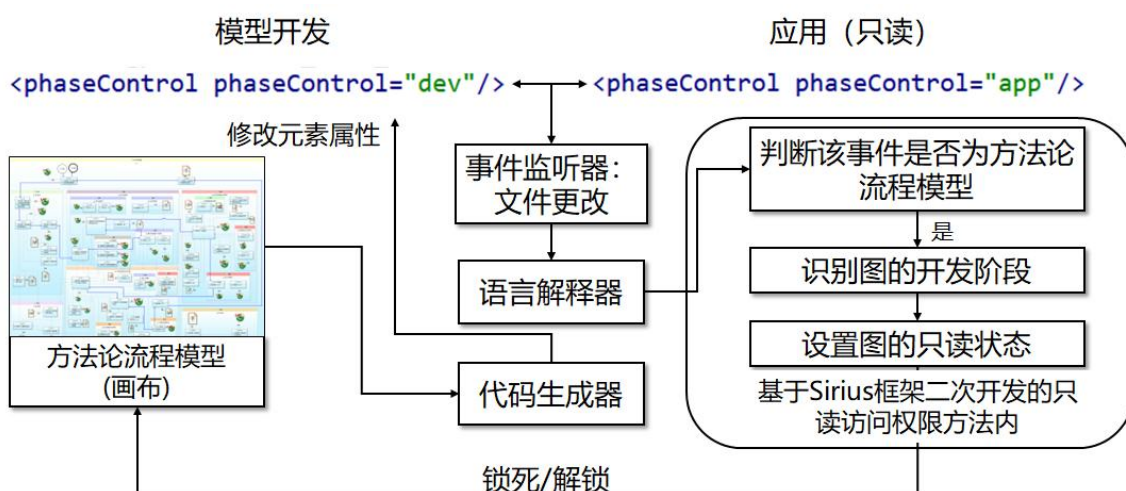


图 4.11 阶段控制工作流程

4.5 案例验证

基于本章所设计导航式建模语言和模型转换技术可以实现对第 3 章构建的方法论流程模型的存储，以及流程模型、XML 模型和文件结构的模型转换。本章以第 3 章的 DoDAF 流程模型为基础，验证本章所提技术的有效性。

1) DoDAF 流程模型在保存后相关信息将存储至 DoDAF.methodology 文件。如图 4.12 所示，第 12 行展示了能力视角，其子节点包括能力视角执行的六个任务和相关视图（图元模型、甘特图、表、关系映射表单）。

```

1 <?xml version="1.0" encoding="UTF-8"?>
2
3 <KARMA_external_plugin>
4   <physical name="DoDAF">
5     <phaseControl phaseControl="dev"/>
6     <methodology localLabel="DoDAF" name="DoDAF">
7       <viewpoint localLabel="定义所有视图" name="DoDAF_expgraph_d747">
8         <task localLabel="AV-1" metaObject="task_obj" taskName="关于体系结构的顶层模型信息。"/>
9         <task localLabel="AV-2" metaObject="task_obj" taskName="描述结构中使用的所有元素。"/>
10        <metaGraph localLabel="AV-1" metaObject="metaGraph_obj" metaGraph="Model_Diagram" fileName="Model_AV1"/>
11        <table localLabel="AV-2" metaObject="table_obj" fileName="AV2"/>
12        <viewpoint localLabel="定义能力" name="DoDAF_expgraph_d747_expgraph_522f">
13          <task localLabel="CV-1" metaObject="task_obj" taskName="提供与能力互操作性的战略要素，并确定能力所需的时间尺度。"/>
14          <task localLabel="CV-2" metaObject="task_obj" taskName="识别现有的和所需的能力。"/>
15          <task localLabel="CV-4" metaObject="task_obj" taskName="识别能力依赖关系。"/>
16          <task localLabel="CV-3" metaObject="task_obj" taskName="分析能力差距"/>
17          <task localLabel="CV-5" metaObject="task_obj" taskName="计划能力交互"/>
18          <task localLabel="CV-6" metaObject="task_obj" taskName="跟踪任务链的能力"/>
19          <metaGraph localLabel="CV-1" metaObject="metaGraph_obj" metaGraph="Model_Diagram" fileName="CV1"/>
20          <metaGraph localLabel="CV-2" metaObject="metaGraph_obj" metaGraph="Model_Diagram" fileName="Model_CV2"/>
21          <gantt localLabel="CV-3" metaObject="gantt_obj" fileName="CV3"/>
22          <metaGraph localLabel="CV-4" metaObject="metaGraph_obj" metaGraph="Model_Diagram" fileName="Model_CV4"/>
23          <table localLabel="CV-5" metaObject="table_obj" fileName="CV5"/>
24          <mTable localLabel="CV-6" metaObject="mTable_obj" fileName="CV6"/>
25          <task localLabel="CV-7" metaObject="task_obj" taskName="跟踪任务的能力需求"/>
26          <table localLabel="CV-7" metaObject="mTable_obj" fileName="CV7" filterType="null" filterMetaType="null" filter="null"/>
27        </viewpoint>
28        <viewpoint localLabel="定义作战要素" name="DoDAF_expgraph_d747_expgraph_6bc0">
29          <task localLabel="OV-1" metaObject="task_obj" taskName="将作战要素或要素置于上下文中。"/>
30          <task localLabel="OV-2" metaObject="task_obj" taskName="定义作战要素并详细说明能力需求。"/>
31          <task localLabel="OV-3" metaObject="task_obj" taskName="null"/>
32          <metaGraph localLabel="OV-1" metaObject="metaGraph_obj" metaGraph="Model_Diagram" fileName="Model_OV1"/>
33          <metaGraph localLabel="OV-2" metaObject="metaGraph_obj" metaGraph="Model_Diagram" fileName="Model_OV2"/>
34          <table localLabel="OV-3" metaObject="table_obj" fileName="OV3"/>
35          <task localLabel="OV-4" metaObject="task_obj" taskName="定义人员角色"/>

```

图 4.12 DoDAF 流程模型的 XML 文件

利用本章所设计的 XSD 文件验证模型转换结果，在生成的 XML 文件中引用 XSD 文件的地址，Eclipse 工具自动执行验证。如图 4.13 所示，验证结果没有出现错误，验证了方法论流程模型到 XML 模型转换的有效性和完整性。

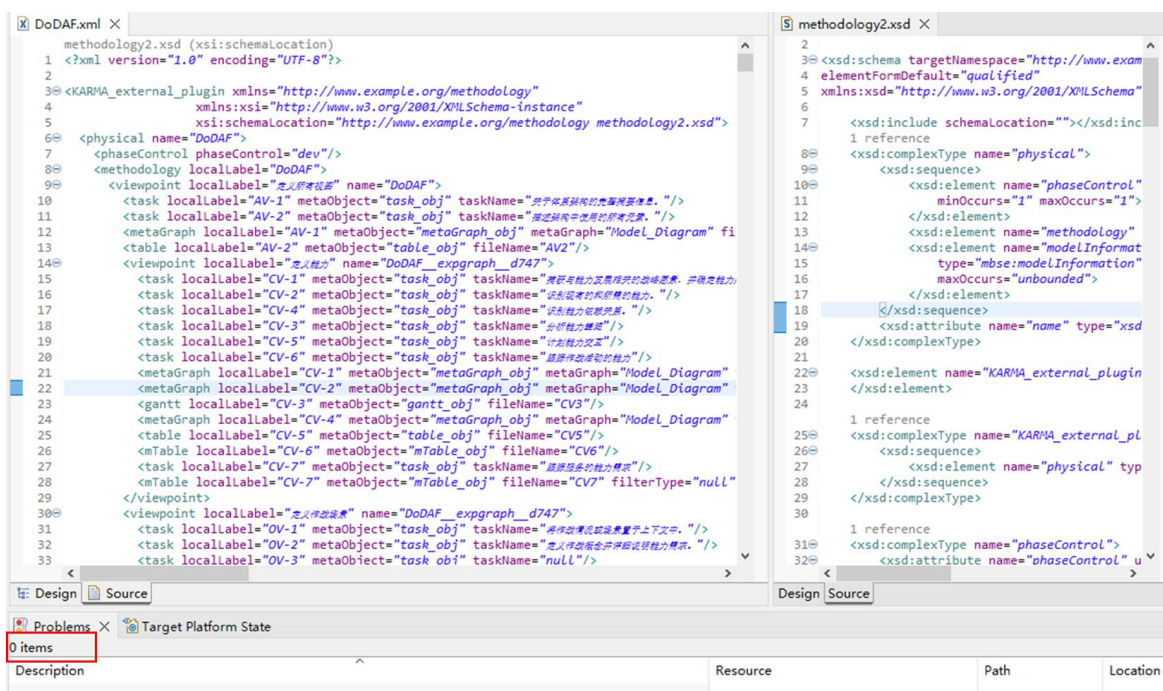


图 4.13 XML 文件的语法验证

2) 修改.methodology 文件的 XML，通过模型转换流程模型也将会对应修改。以

图 4.14 为例，将阶段控制改为“app”，DoDAF 模型被锁定，流程模型的最上侧增加了一个“锁”，进入只读状态，拖动图元模型无法拖动，但是对象的属性值等信息仍可以被修改。

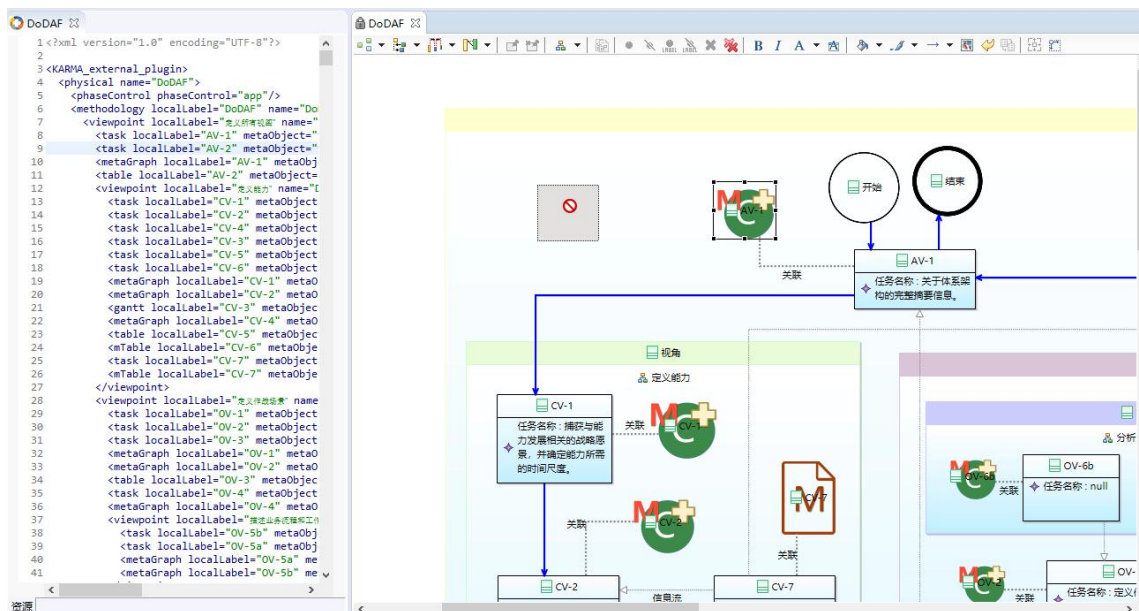


图 4.14 流程模型在应用阶段锁定

3) 通过 XML 模型文件, DoDAF 的文件结构模型可以自动生成。图 4.15 为 DoDAF 的文件结构模型，模型中包括文件夹及具体的文件，所有文件均可直接打开使用。仍以能力视角为例，其六个视图所对应的文件均可直观地展示，并且快速访问。

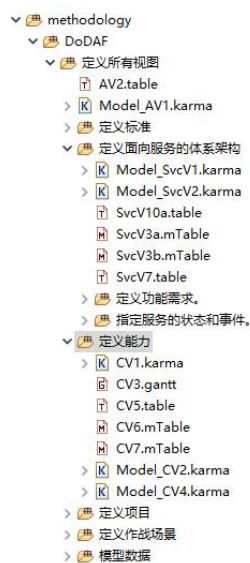


图 4.15 流程模型自动转换生成文件结构

案例分析结果表明,本章所设计的基于 XML 的导航式建模语言能够有效实现对方方法论流程模型的存储与简要表达,并以该 XML 模型为介质实现了图模型与文件结构模型的转换。

4.6 本章小结

为了保证构建的方法论流程模型能够为计算机所识别和存储,本章提出基于 XML 设计导航式建模语言。在构建建模语言的同时,对其与流程模型、文件结构之间的模型转换手段进行了分析,设计了模型转换算法,提升了系统设计效率,自动转换的同时也保障了模型的一致性。最后,在第 3 章案例的基础上,实践了流程模型、文件结构模型到 XML 模型的生成和 XML 模型向流程模型、文件结构模型的解释。结果表明,本章提出的导航式建模语言能够有效地将图形数据转换为规范定义的存储格式,并实现描述方法论的不同视角模型之间的转换。

第 5 章 复杂装备系统导航式建模系统设计及验证

5.1 引言

在 MBSE 的设计活动中，所有模型都依靠建模工具构建。因此，第 3 和 4 章所提出的元模型及建模语言处于概念级，并不能直接创建。本章旨在解决上述问题，在 KARMA 语言建模工具的基础上研发复杂装备系统导航式建模系统，为设计人员创造交互良好的可视化设计环境，除了实现方法论三种视角模型的创建和模型转换外，还将流程模型与系统模型、仿真文件等模型关联，从而打通流程模型与系统全生命周期模型之间的通道。

基于该平台，通过分析该系统对当前主流建模工具中方法论表达形式的支持情况，考察其能够表达的方法论数量，验证本研究在多场景下的支持能力；通过与成熟建模工具横向对比，验证所提系统性能。最后开展以“毅力号”火星车为对象的案例验证，通过在系统架构设计的全生命周期中应用该系统功能，包括方法论定制化开发、驱动系统模型开发和仿真验证等功能，验证所提方法和系统的可行性和有效性。

5.2 导航式建模系统设计

5.2.1 系统功能概览

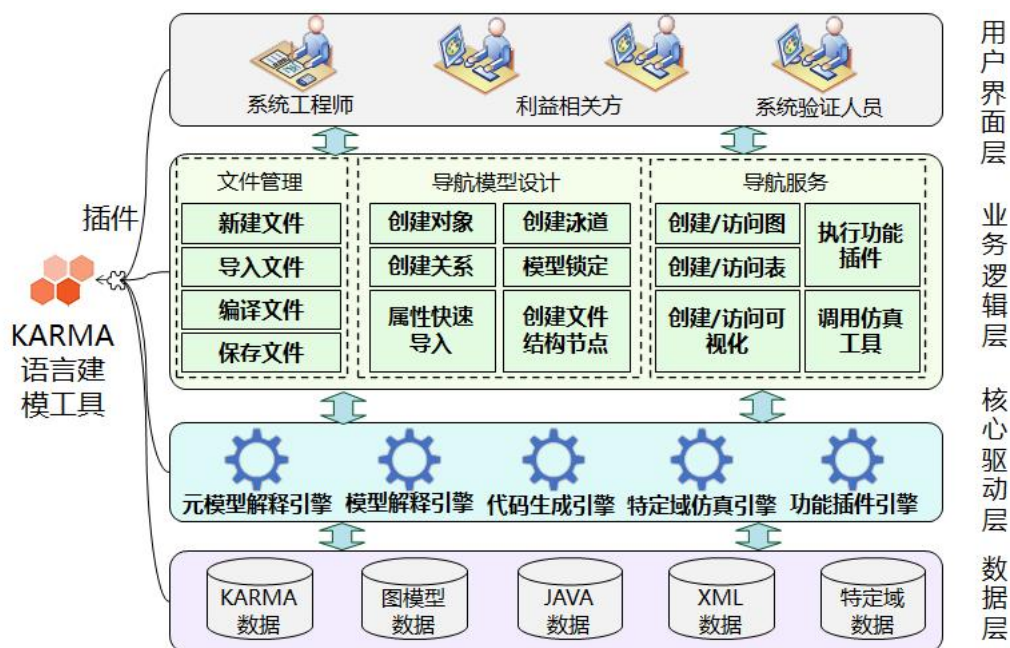


图 5.1 导航式建模系统功能架构

针对导航式建模元模型和基于 XML 的建模语言，本文设计开发了支持 MBSE 方法论表达的复杂装备系统导航式建模系统。该系统实现了从流程模型创建，到驱动架构模型设计，到各类仿真验证执行的一体化过程，辅助设计人员依照定制化的方法论开发系统架构模型。本系统包括数据层、核心驱动层、业务逻辑层和用户界面层，具体架构如图 5.1 所示。

数据层表示复杂装备系统导航式建模系统中涉及的建模、模型转换和仿真的具体数据形式，由核心驱动层驱动为业务逻辑的实现提供了底层数据支持。包括以下五种：

- **KARMA 数据：**分为前端和后端的数据两部分。前端数据用于记录和保存导航式建模系统元模型及其连接规则数据，包括图元模型、对象元模型、属性元模型、点元模型、关系元模型和角色元模型，这部分数据是构建导航式建模图模型的基础；后端数据为 KARMA 语言转换计算机语言过程中产生的数据。
- **图模型数据：**用于记录并存储流程模型的数据，包括布局、对象、关系等，支持导航式建模，支持与 KARMA 数据的相互转换。
- **JAVA 数据：**工具底层的代码数据类型，其中后端 KARMA 数据和文件结构模型数据均以 JAVA 形式存在。JAVA 底层数据类型的存在不仅支持了三种导航模型的相互转换，还支持了导航式建模工具的导航、执行和仿真功能实现。
- **XML 数据：**用于保存流程模型，并具有简要表达方法论结构的能力，支持与 KARMA 数据、JAVA 数据的相互转换。
- **特定域数据：**包含本文所设计的导航式建模系统所支持导航、执行的全部数据类型，包括 KAMRA 语言（架构模型、架构驱动、代码生成、指标验证均为这一类型）、桑基图、关系图、树状映射图、设计结构矩阵、需求管理、表、关系映射表单、甘特图、AFsim、Simulink、Modelica 数据类型。

核心驱动层表示复杂装备系统导航式建模系统获得的底层技术支持，用于驱动数据层的数据，包括以下核心驱动：

- **元模型解释引擎：**支持解析基于 GOPRR-E 开发的复杂装备系统导航式建模元模型，通过该引擎实现对导航式建模元模型的读取，并将其解释为 KARMA 后端数据结构，实现元模型库的生成。
- **模型解释引擎：**支持解析导航模型的 XML 模型文件，通过该引擎实现对 XML 数据的读取，一方面将其解释为 KARMA 后端数据结构后，解释为图模型数据，实现流程模型生成；另一方面直接将 XML 数据解释为文件节点的 JAVA 数据，实现

文件结构模型生成。

- 代码生成引擎：支持解析流程模型或文件结构模型，通过该引擎实现对流程模型和文件结构数据的读取，最终生成对应的 XML 模型文件。
- 特定域仿真引擎：用于驱动体系仿真、多物理场仿真、Simulink 仿真等外部（相对于复杂装备系统导航式建模系统）独立工具，为系统模型执行领域仿真提供交互接口，驱动数据层的特定数据。
- 功能插件引擎：用于驱动指标验证、代码生成、架构驱动、表单、可视化等功能插件的构建和执行，为流程模型和工具中特定域数据提供交互接口，驱动数据层的特定域数据。

业务逻辑层表示复杂装备系统导航式建模系统允许用户完成的具体操作，业务逻辑层由核心驱动层驱动，参与业务级功能实现。包括以下七种功能：

- 导航模型文件操作：支持用户在导航式建模系统中创建导航式建模文件、编辑文件、保存文件和复制文件等基本操作。
- 导航模型文件导入：支持用户从导航式建模系统外部导入新的导航模型文件，支持模型协作和模板使用。
- 导航模型文件编译：支持用户打开导航模型文件时，文件自动编译，根据 XML 生成方法论流程图，展示画布模型。
- 导航模型设计：支持用户在模型画布中通过拖拽或配置的方式设计，具体支持增加、删除、修改、复制的操作，从而实现方法论流程模型开发；支持用户在配置模型属性时通过快速筛选填入属性值，提升设计效率；支持方法论流程模型的只读锁定控制，管理设计阶段；支持用户通过右键创建文件夹或文件的方式实现文件结构模型的设计。
- 不同模型的创建与访问：支持用户通过构建元模型中的特定对象并使用该项功能，实现对系统架构设计过程中的不同模型的创建与访问。
- 特定功能执行：支持用户通过构建元模型中的特定对象并使用该项功能，实现对系统架构设计过程中具有执行操作的模型的功能执行，例如可以通过指标验证对象执行指标验证、通过代码生成模型执行代码生成。
- 调用特定域仿真工具：支持用户通过构建元模型中的特定对象并使用该项功能，直接从导航式建模系统实现快速调用系统架构设计过程中需要采用的外部仿真工具执行仿真，例如通过体系仿真对象调用 AFsim 工具进行体系仿真。

用户界面层通过对业务逻辑层的具体功能组合表示复杂装备系统导航式建模系统为用户提供的具体应用。包括以下三种应用：

- 导航模型文件管理：包括对导航模型文件操作、导入和编译。复杂装备系统导航式建模系统支持对导航模型文件的管理。
- 导航模型设计：包括流程模型和文件结构模型的设计及其他设计相关的辅助功能。
- 导航服务：包括创建、访问、执行内部功能、调用外部仿真的操作。通过提供导航服务，能够充分结合模型的特点，发挥模型驱动设计的能力。

5.2.2 技术框架

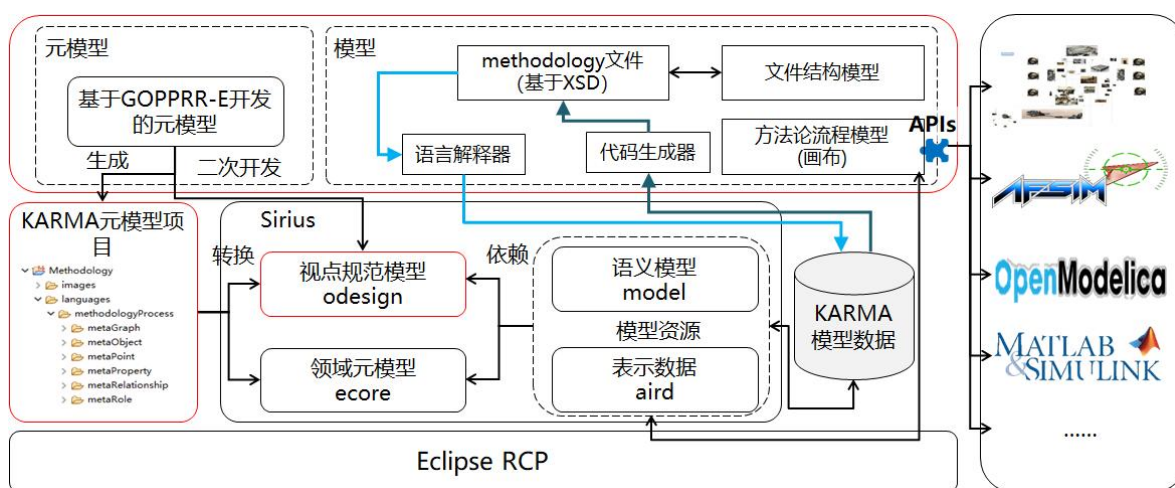


图 5.2 导航式建模系统技术框架

导航式建模系统的具体技术框架如图 5.2 所示，技术框架分为三层，最底层为开发基础，中间层为 KARMA 语言建模工具，最顶层为导航式建模系统，红色椭圆框内的部分为导航式建模系统的开发设计部分。

（1）开发基础

导航式建模系统的基础架构为 Eclipse RCP（Rich Client Platform），它提供了丰富的组件和框架，使开发人员能够快速构建基于插件的应用程序。由于本研究所开发的系统建立于 KARMA 语言建模工具的基础之上，采用 Eclipse RCP 框架可以将导航式建模系统模块化，以插件的形式将其集成至 KARMA 语言建模平台，插件的开发形式也有利于两部分独立的开发、测试、调试和维护。此外，Eclipse RCP 提供许多内置的工具和组件，包括图形和数据可视化工具、SWT、JFace 和 WindowBuilder 等。这些工具和框架有助于更轻松集成第三方工具和技术，并创建出具有高度可定制化和专业外观的应用程序。

(2) KARMA 语言建模工具

KARMA 语言建模工具的建模活动基于 Sirius (一款开源的 Eclipse 插件, 用于快速创建和定制专用的图形化建模工具)。在元模型构建结束后, KARMA 语言通过模型转换自动生成相应 `ecore` 和 `odesign` 文件, `ecore` 负责定义元模型数据, `odesign` 负责控制元模型的外观、界面、二次开发设计等。这两个文件相当于 Sirius 概念下的元模型。KARMA 语言建模工具构建画布模型的过程就是利用 Sirius 构建画布模型的过程, 建模过程中将生成 Sirius 的图模型数据文件 `model` 和 `aird`, 用于存储画布模型的全部信息(例如存在什么对象、存在数量、布局信息等), 二者的生成依赖于 `ecore` 和 `odesign` 文件。`model` 和 `aird` 文件也可以在 KARMA 语言建模工具中自动转换生成为 KARMA 语言模型数据。

(3) 导航式建模系统

导航式建模系统基于 KARMA 语言建模工具开发, 其元模型构建的方法和 KARMA 语言建模工具一致。这个过程除了获得 KARMA 元模型项目外, 还得到一组 `ecore` 和 `odesign` 文件。由于 `odesign` 文件用于管理导航式建模画布页面的前端展示, 所以对生成的 `odesign` 文件进行二次开发, 开发不同对象元模型的属性配置页面, 并提供属性的查找与快速填写功能, 创建不同工具类实体创建/访问/执行等功能实现的接口和界面, 将特定工具类与模型、表、建模工具内部功能插件和外部仿真工具进行关联, 实现导航和操作执行。

在实例层开发 XML 编辑器, 用于高亮关键词展示和编辑第 4 章设计的导航式建模语言的 XML 模型; 开发文件结构浏览器, 用于展示由 XML 模型映射的方法论文件结构, 并支持文件的创建与访问; 导航式建模画布的开发由上文的 `odesign` 二次开发完成; 语言解释器和代码生成器为后端, 同第 4 章所述用于实现 XML 文件和 KARMA 模型数据(KARMA 模型数据进一步实现与流程模型的相互转换)以及 XML 文件和文件结构模型的相互转换。

5.3 能力验证

5.3.1 定量分析

定量分析是一种研究方法, 通过对设计指标量化来分析评估方法的有效性。针对前文分析得到的当前 MBSE 领域设计过程中的问题——支持方法论流程表达的模型无法适应多架构建模以及其与方法论自身未形成规范的映射, 分析导航式建模及语言

的通用表达能力。通过定量分析探究导航式建模能支持表达方法论流程的数量，论证其对多架构建模的适应能力。

具体过程为，构建方法论与导航式建模元模型的映射关系，当方法论的所有要素都可以被导航式建模描述后，则证明导航式建模适应了该方法论所描述的架构设计。DoDAF、MagicGrid、OOSEM、Arcadia、Harmony-SE 是五种成熟的 MBSE 建模方法论，得到了广泛的应用实践。通过抽取这五种建模方法论的关键要素并将其映射至导航式建模的元模型，实现对这五种建模方法论的构建，具体的过程示例见 3.5。

基于以上的方法，设计并构建建模方法论的流程模型，如表 5.1。其中，提供模型支持指建模方法论所使用的建模语言；视角数和任务数是 MBSE 建模方法论表达的重要流程对象；管理模型数为方法论中明确定义的在特定任务中使用的模型，未统计未在建模方法论定义的模型，而且实际过程中设计人员也可能重复使用模型，所以是最小值。表中结果表明，DoDAF、Arcadia、MagicGrid、OOSEM、Harmony-SE 多种建模方法论均可由导航式建模描述，所提方法实现了对方法论流程的兼容性表达。因此，本研究所设计的导航式建模系统及语言支持通过统一的规范表达体系、系统、分系统级，和软件系统等特定领域的方法论。同时，基于 KARMA 语言建模平台提供的多架构建模的能力，支持通过构建导航式建模系统管理复杂装备系统设计过程中涉及的多架构模型，验证了通用表达能力指标。

表 5.1 导航式建模支持表达的方法论概览

方法论/框架	提供模型支持	视角数	任务数	管理模型数 (≥)	层级/领域
DoDAF (图 3.6)	UPDM	8	52	52	体系
Arcadia (图 5.3)	Capella	5	22	61	系统、分系统
MagicGrid (图 5.4)	SysML	3	11	13	系统、分系统
OOSEM (图 5.5)	SysML	3	9	未指定	软件系统
Harmony-SE (图 5.6)	SysML	3	44	22	系统、分系统

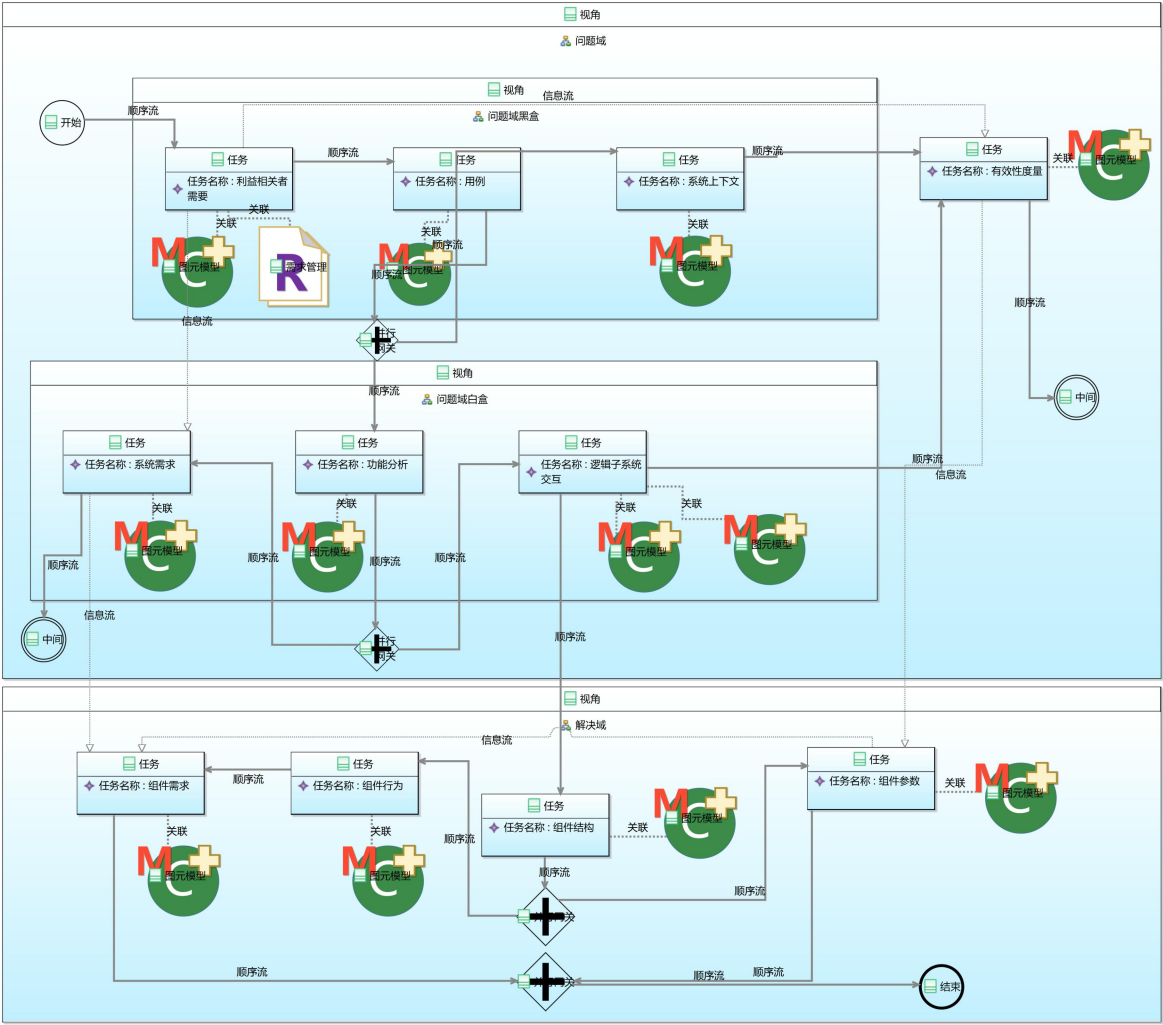


图 5.4 MagicGrid

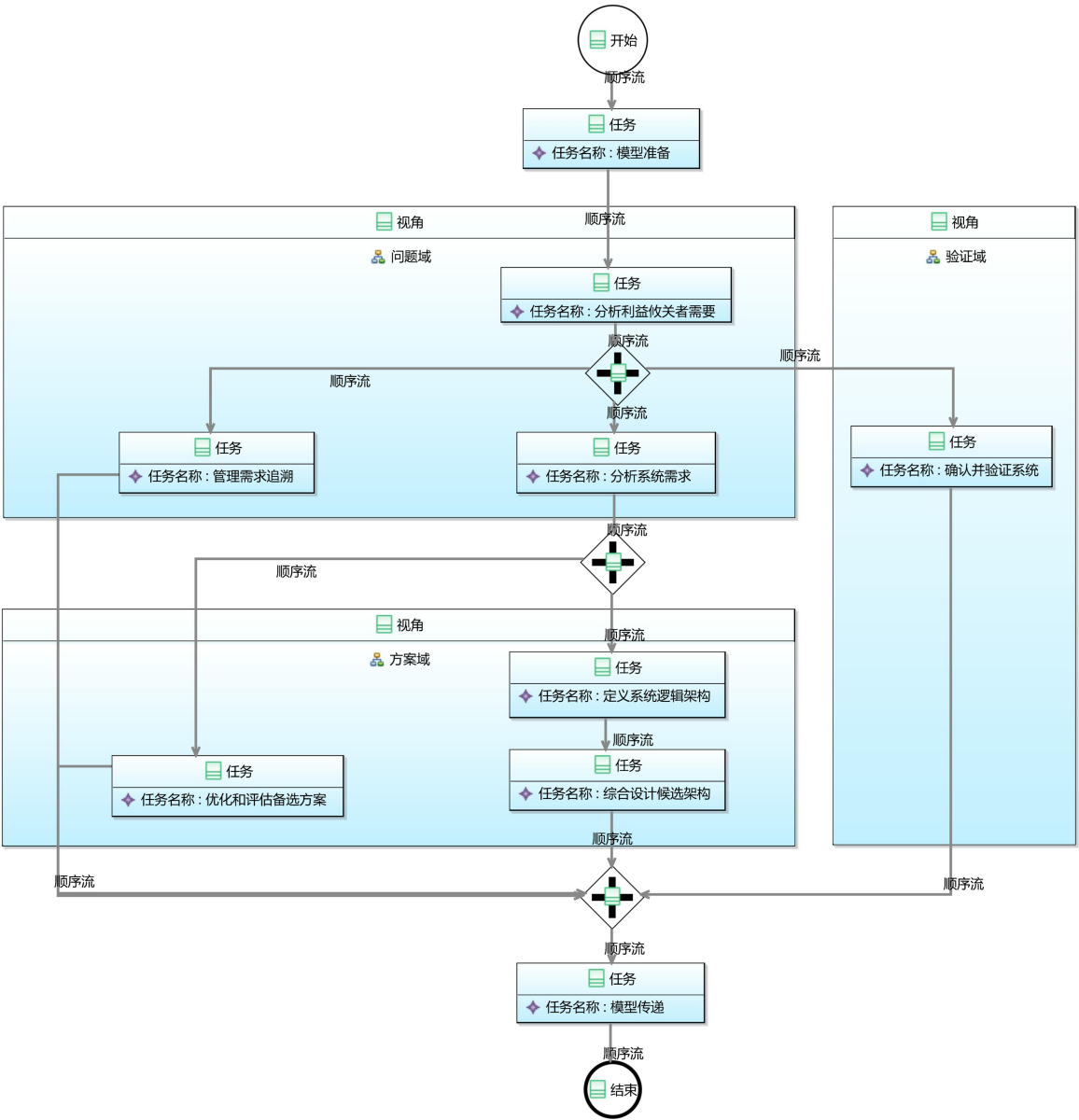


图 5.5 OOSEM

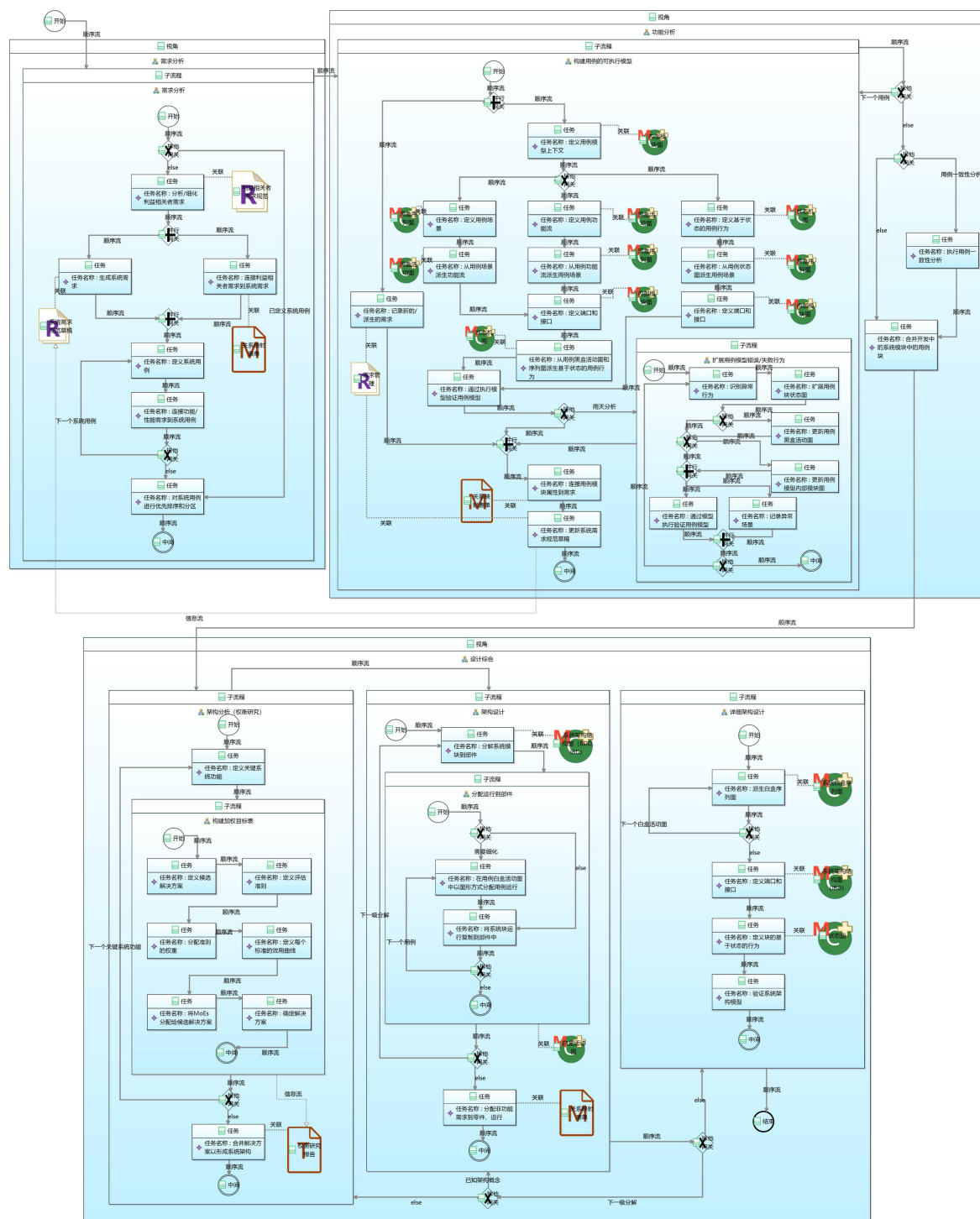


图 5.6 Harmony-SE

5.3.2 定性分析

定性分析是对事物的质量进行分析和描述的方法，与依赖于数量化数据的定量分析不同，定性分析通常是基于主观性的观察和分析。通过对比进行定性分析是一种常

用的研究方法，通常用于比较两个或多个对象、事件或概念之间的差异和相似之处，判断依据是对比对象的特征、属性、关系和环境等方面的信息。本小节从应用实现的角度，展开本研究所设计复杂装备系统导航式建模系统的定性分析，对比本研究所设计的建模系统与当前流行 MBSE 建模软件——MagicDraw、Capella 和 Rhapsody 在支持方法论模型要素表达及模型驱动设计方面的性能差异。

如表 5.2 所示，模型要素表达关注方法论在建模工具中不同表现形式和转换，模型驱动设计关注方法论流程模型在指导设计人员开展复杂装备系统架构设计提供的功能，重点关注流程模型驱动设计是因为文件结构和文本模型本身用于描述结构，不具有明确的顺序过程，重在服务于管理而不是指导设计。

表 5.2 与同类系统的对比

与方法论相关功能	系统	导航式建模系统	MagicDraw	Rhapsody	Capella
模型要素表达	流程模型	支持	支持	支持	支持
	文件结构	支持	支持	支持	支持
	文本模型	支持	不支持	不支持	不支持
	模型转换	支持	支持包图到文件结构的转换	不支持	不支持
流程模型驱动设计	定制化	支持	支持，但不提供专用工具	支持，但不提供专用工具	半定制化
	导航	支持	支持	支持通过文件结构导航	支持通过文件结构导航
	创建架构模型	支持	不支持	不支持	支持
	辅助功能	支持	不支持	不支持	支持执行架构驱动
	外部仿真	支持	不支持	不支持	不支持
	多架构设计	支持	不支持	不支持	不支持

（1）模型要素表达

1) 流程模型：导航式建模系统支持利用导航式建模元模型设计流程模型；MagicDraw 和 Rhapsody 支持使用表达流程的建模语言（如 SysML 活动图）设计流程模型。Capella 也可以通过这种方式支持流程模型，但是其自身已经提供表单式的 Arcadia 方法论流程模型，因此一般没有设计人员会利用 Capella 单独构建方法论的流程模型。

2) 文件结构：导航式建模系统、MagicDraw、Rhapsody 和 Capella 都提供模型浏览器用于展示和管理方法论的文件包结构。

3) 文本模型：导航式建模系统根据其设计原理存在能够描述方法论结构的 XML 模型，同时起到模型存储功能；MagicDraw、Rhapsody 和 Capella 不提供文本模型，无论是文件结构还是流程模型都依托于建模项目存在，不具备轻量化存储、自我描述的特点。

4) 模型转换：导航式建模系统支持方法论的流程模型、文件结构和文本模型自动转换，提升设计效率；MagicDraw 支持设计人员在使用包图时，包图模型向文件结构的单向转换；Rhapsody 和 Capella 不支持这三类模型的转换，每类模型均需手动构建。

（2）流程模型驱动设计

1) 定制化：导航式建模系统提供专用的建模手段支持方法论的定制化设计；MagicDraw 和 Rhapsody 不提供专门的设计工具，但是用户可以利用工具中的其他功能进行定制化设计；Capella 工具内嵌了 Arcadia 方法论，设计人员在该工具中只能利用该方法论进行系统性地设计，支持根据情况增减 Arcadia 方法论的设计阶段，不是完全的定制化。

2) 导航：导航式建模系统和 MagicDraw 支持通过流程模型和文件结构模型导航至相关复杂装备系统架构模型视图；Rhapsody 和 Capella 只支持从文件结构的导航。

3) 创建架构模型：导航式建模系统和 Capella 支持通过流程模型中的特定元模型创建特定架构模型，包括图模型、表模型等；MagicDraw 和 Rhapsody 不支持这项功能。

4) 辅助功能：导航式建模系统支持通过流程模型中的特定元模型执行指标验证、架构驱动、仿真等操作，Capella 流程模型支持架构驱动功能；MagicDraw 和 Rhapsody 尽管提供诸多辅助功能，但是这些功能一方面脱离于流程模型存在，另一方面在国内禁用。

5) 外部仿真：同辅助功能的描述。

6) 多架构设计：导航式建模系统支持体系、系统、分系统和各领域的多架构一体化设计；其余三款软件不支持，方法论流程提供的设计支持不会超出当前项目的建模语言所表达的范围，适用范围较单一。

综上所述，针对本研究所总结的研究问题，对比其余流行的建模工具，导航式建模系统在方法论的模型要素表达和模型驱动设计方面均具有优势。导航式建模系统有

效地将架构设计过程中所需要的功能元素整合至流程模型中，通过方法论流程模型所描述的工作流和产物，直接驱动架构设计。而不是在其他建模工具中类似于两层皮的表达，流程只用于示意，最终的架构设计过程仍需要单独设计，这样丧失了模型化的意义。此外，导航式建模系统实现了流程模型、文件结构和文本模型三类模型的转换，有效节省了设计人员的开发时间，提升了复杂装备系统研发效率，模型的自动转换也保障了模型的一致性，维护系统模型的统一真相源。

5.4 应用验证

5.4.1 案例背景

近一个世纪以来，人类一直在不断地探索火星，随着科学技术的不断发展，火星车成为主要的探索工具。和火箭等复杂工程产品类似，火星车涉及传动、姿控等多学科耦合和机械、控制、电气等多领域交叉特性，具有系统集成度高、系统间交互频繁等特点，因此，火星车属于典型的复杂产品，其产品研制过程也适用于复杂产品的系统工程研制方法。在“好奇号”火星探测器探测任务的成功执行后，作为“好奇号”火星车的后续延伸计划，美国国家航天局（NASA）公布的新一代火星车“毅力号”，其在整体飞行系统设计上很大程度上复用了“好奇号”火星车的设计，主要用于搜寻火星上过去生命存在的证据，并进行一系列的科学探索研究。

然而，“好奇号”火星车的研制过程主要采用的是基于文档的设计方法，设计数据全部以文档形式存储，包含 Word、Excel 或 PPT 等格式，且在表达方面缺乏统一标准，同一个概念可能会使用不同的概念形状进行表达，存在设计信息歧义和一致性差等问题。根据公开资料显示，在“毅力号”火星车（图 5.7）的研制过程中，NASA 设计团队采用了 MBSE 的方法进行设计，“毅力号”的设计以“好奇号”火星车的文档形式的数据为基础，进行了模型化的翻译，形成了基于 SysML 的系统架构模型，并进一步开展了复杂行为分析、设计数据的正确性验证、自动生成项目节点交付物、模型的布局分析和可视化等研制活动。

然而，在上述火星车概念建模过程中，需要针对不同系统层级、领域构建不同的模型，例如架构建模需要采用 SysML，特定域建模需要采用 Modelica，现有的建模方法论往往基于单一规范，难以有效支持建模过程中对方法论定制化的需求。此外，目前多种建模语言之间存在建模语言的语义语法异构问题，而现有建模工具中方法论应用服务不具备同时驱动这些异构模型的能力。

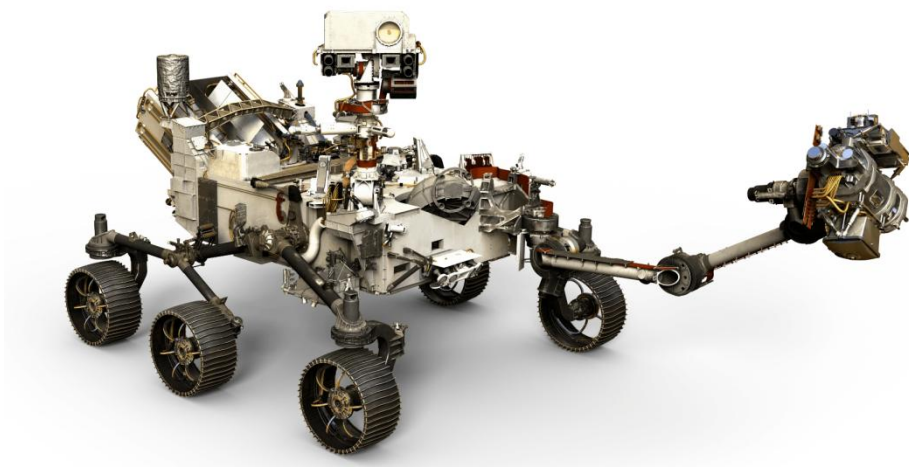


图 5.7 “毅力号”火星车

针对上述问题，基于“毅力号”火星车研发文件^[75]，本文采用复杂装备系统导航式建模系统及语言和 KAMRA 语言建模工具，面向“毅力号”火星车开展基于使命、运行、功能、逻辑和物理（MOFLP）建模方法论的概念建模，解决现有方法论在多架构建模及模型驱动方面的问题，验证本文所提系统在实际应用中的可行性和有效性。

5.4.2 MOFLP 建模方法论

结合“毅力号”火星车的模型特点，本研究提出了一种 MBSE 建模方法论——MOFLP 方法论，将需求分析融入到使命、运行、功能、逻辑和物理分析中，通过使命和运行分析充分分析问题空间和场景，在实际的系统分析过程中定义各类需求模型，指导系统工程师从事建模活动。所提方法论如图 5.8 所示，分为使命、运行、功能、逻辑和物理五个视角。为了更适当地表达 MOFLP 方法论，本研究基于 KARMA 语言构建了适应于“毅力号”火星车设计的特定域模型库，扩展定义的部分术语解释见表 5.3。

表 5.3 MOFLP 方法论术语解释

术语	含义
使命	需要实现的一系列目标的集合
目标	实现使命所期望达到的效果
效果	预期结果的描述
实体	实体可以参与目标的实现，或者它受到了目标的影响，涵盖利益相关者和实体
利益相关者	与使命有利害关系，但不参与使命实现的实体
能力	运行场景下可以实现的效果

任务	实体执行的活动
运行系统	运行场景下的系统
兴趣系统	运行系统的泛化
使命阶段	使命的实现场景的一部分
运行阶段	从兴趣系统角度描述的使命阶段

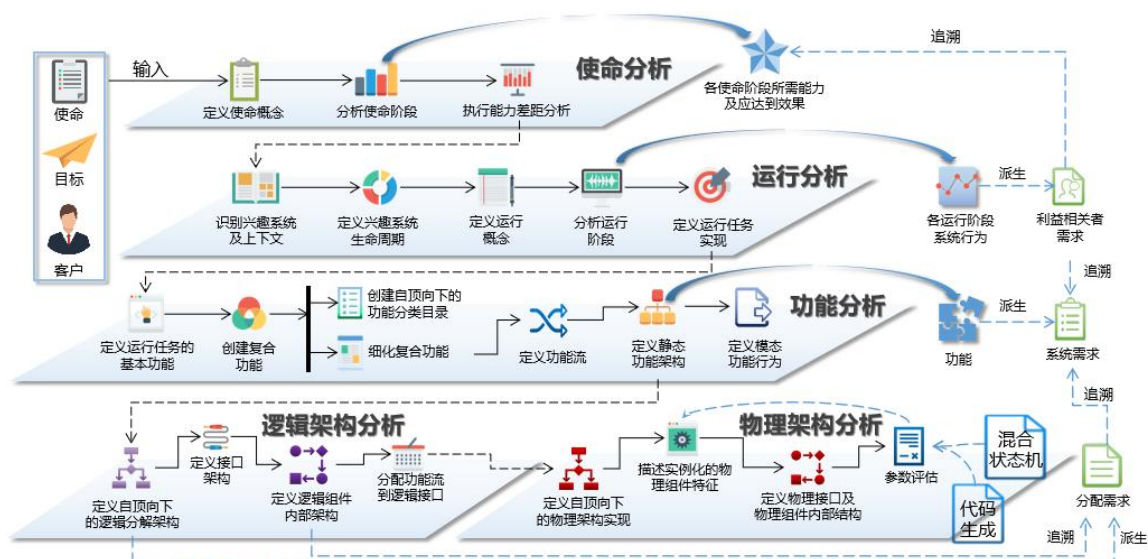


图 5.8 MOFLP 建模方法论

（1）使命分析

根据系统输入的使命，使命分析将定义系统的业务问题、表征问题空间、确定解决问题的潜在方案。

主要包含以下活动：1) 规范化客户、组织的对使命和相关目标的描述。使命是从客户的角度定义的，应包括运行、生产和支持；2) 识别参与使命的实体和利益相关者，并将其与目标关联，目的是识别使命中非客户实体及其目标；3) 定义使命解决方案的概念（称为使命概念）及其相关的使命阶段；4) 分析使命阶段，分析参与使命阶段实现的实体所执行的任务，并确定相关能力；5) 能力差距分析：分析实体和运行系统如何执行功能。

（2）运行分析

运行分析目的是进一步分析外部实体（用户、外部系统等）将如何与兴趣系统（SOI, System of Interest）交互，在早期阶段对利益相关者进行沟通 and 验证，确认 SOI 将要做什么。

主要包含以下活动：1) 指定兴趣系统和上下文：识别使命概念运行系统中的兴

趣系统，并对其进行运行分析；2）确定兴趣系统生命周期，这是系统分析的出发点；3）定义运行概念，从运行的角度解释使命概念解决方案是如何运行的；4）分析运行阶段，与用户和操作人员精确定义各个运行阶段系统的预期行为；5）从运行行为中派生利益相关者需求。同时，定义运行任务实现，即实现运行任务的复合行为。

（3）功能分析

功能分析主要关注功能定义，这基于运行分析所得到的参与者用例（即运行任务实现）。该阶段系统需求与功能架构处于同一层级，重点是提供利益相关者所期望的系统特性的技术描述。

共包含三条主线，第一条主线为：1.1）定义参与运行任务实现的基本功能，识别运行任务实现中由运行信号触发的基本功能；1.2）从 L_{n-1} 功能建立 L_n 功能，在系统的更高抽象层级范围内创建所有功能；2）细化复合功能：预计逻辑组件（在逻辑架构分析期间）的功能分配，并向下到下一个级别（ L_n to L_{n+1} ）；3）定义每个复合功能的连接和流，识别由连接和流突出的基本功能的交互；4）建立静态功能架构，获得基于所有功能行为的整体功能架构；5）建立功能集目录，将功能（复合和基本功能）分类，以便储存和提高可重用性。第二条主线，定义功能模态行为，用于验证系统需求和分析系统行为优化功能架构。第三条主线：1）定义从功能架构派生的系统需求，目的是将利益相关者、需求驱动的预期服务视图转换为能够交付这些服务的所需产品的技术视图；2）追溯利益相关者需求到系统需求，用于验证功能层与运行层是否兼容。

（4）逻辑架构分析

逻辑架构定义了兴趣系统的组成。在架构层面，逻辑架构分析回答了兴趣系统是如何分解的，该过程实现了所有分配的功能，考虑了相关的功能流和交互。逻辑架构相对于物理架构仍保持了一定的抽象层级，定义了所需的子系统类型和逻辑项目，但仍然为具体的技术实现（例如软件/硬件）保留了一定的余地。与逻辑架构层级一致的是分配需求，并承接了系统需求。分配需求是专门为其所分配的一个或几个模块而产生的需求，以便形式化地解释相应模块考虑到的需求。

分为两条并行的主线，第一条主线内容包括：1.1）构建逻辑分解结构，确定当前抽象层级所需的逻辑组件；1.2）定义接口架构，指定当前抽象层级的各个接口类型；2）分配功能到逻辑组件，确保系统逻辑组件与所分配功能的一致性；3）通过相关接口，定义逻辑组件的内部架构；4）分配功能流到逻辑接口，确保功能流和系统逻辑

接口的一致性。第二条主线内容包括：1) 定义从逻辑架构派生的分配需求：细化系统需求和逻辑架构到分配需求，确定一个能够实现预期功能前物理方案；2) 追溯分配需求到系统需求，验证分配需求是否涵盖系统需求。

(5) 物理架构分析

物理架构模型是对物理元素的管理，是兴趣系统的解决方案，是架构和技术以及设计准则权衡的结果，旨在满足逻辑架构元素和系统需求。通过对物理架构模型的参数研究，实现物理架构的权衡和整体优化，经过验证后，成为详细设计的基础。

主要活动为：1) 定义总装分解结构，将第 L 级的物理组件分解为第 $L+1$ 级的子组件；2) 描述物理组件特征，描述每个物理组件具有的物理特性和特征类型，源于架构、技术的选择以及设计准则应用。物理特性和特征的设置是参数分析的基础。第 N 级的物理组件可以分解为第 $N+1$ 级的物理组件，物理组件将通过详细的设计活动加以改进；3) 定义特定规程分解结构，根据一个或几个特定于规程的分组标准对物理组件进行分组；4) 定义物理接口，定义物理组件如何使用其物理特性和特征相互交互，最后物理接口将由详细设计活动细化；5) 评估参数定义，确保物理架构满足设计需求，用于分析和权衡；6) 将逻辑组件映射到物理组件，确保每个具有物理实现的逻辑组件映射到一个或多个物理组件。确保每个物理组件对于一个或几个逻辑组件是可跟踪的，或者追溯到一个或几个派生的需求。

尽管上述文字对 MOFLP 建模方法论展开了描述，但部分建模流程涉及并行或选择，文字描述仍不清晰，下节利用复杂装备系统导航式建模系统构建 MOFLP 建模方法论流程模型，用模型驱动“毅力号”火星车设计。

5.4.3 导航式建模驱动火星车设计

(1) 定制化

结合毅力号火星车设计特性和 MOFLP 建模方法论，利用复杂装备系统导航式建模系统对“毅力号”火星车的建模方法论进行定制化开发，包括使命分析、运行分析、功能分析、逻辑架构分析和物理架构分析五个阶段。

首先，如图 5.9 所示，在 KARMA 语言建模项目的 model 文件夹下，新建导航式建模文件，文件后缀为 .methodology。

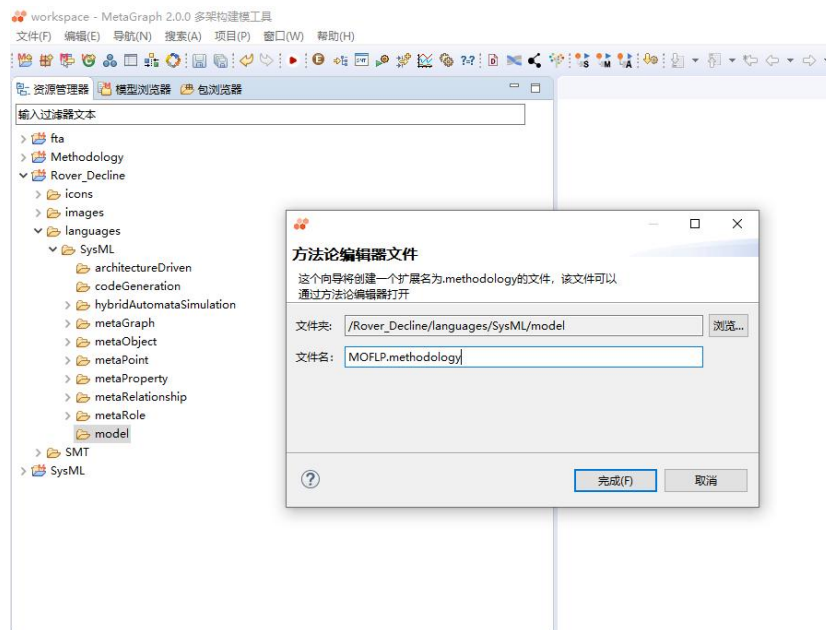


图 5.9 新建导航式建模文件

其次，打开创建的文件画布，在画布中构建 MOFLP 方法论流程模型，如图 5.10。

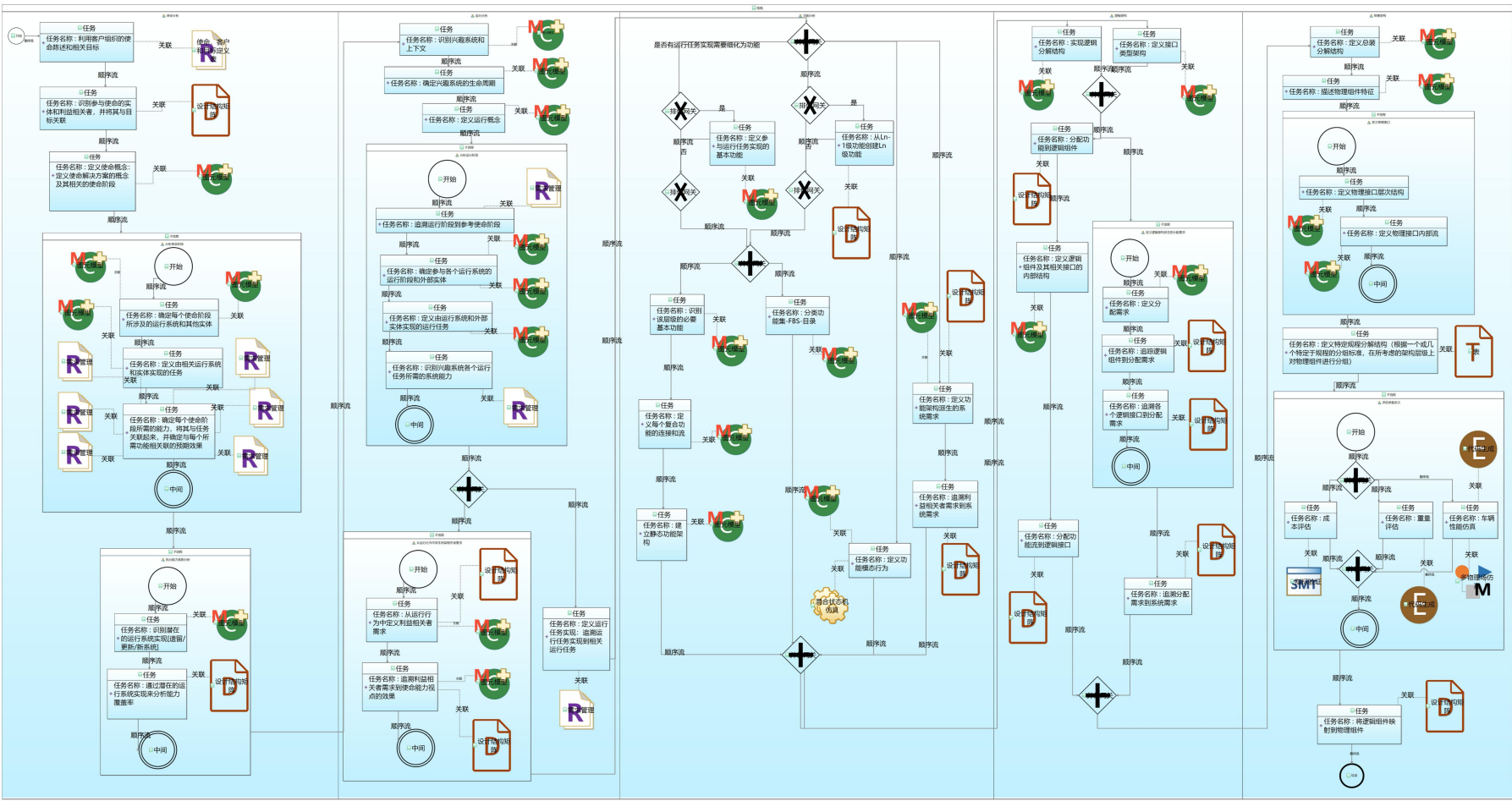


图 5.10 MOFLP 方法论流程模型

(2) 创建架构模型并支持多架构

如表 5.4 所示, 方法论流程模型除了描述“毅力号”火星车设计流程外, 还利用特定的工具类对象元模型描述了各流程节点关联的模型或特定工具操作, 有 57 个工具类的对象与 81 个 KARMA 语言模型、13 个设计结构矩阵、1 个表、10 个 ReqIF 需求管理模型、2 个代码生成模型、1 个指标验证模型建立了关联。57 小于实际架构模型型数的原因在于, 存在特定任务需要构建多个类似模型的情况, 例如分析过程中, 需要对前期准备、火箭发射等多个并列的使命阶段构建活动图, 导航式建模系统支持通过以“,”分割这些模型名称, 支持在一个工具对象中同时建立这些相似模型, 减少反复建模操作。

如图 5.11 所示, 当建模流程进入到确定每个使命阶段所涉及的运行系统和其他实体任务时, 在该任务关联的图元模型对象的属性设置中填入要创建或者访问的具体文件名称(也可以通过按钮快速选择现有模型填入属性值), 例如 Mission_Phase_BDD, 这一过程支持填入多个文件名称。填好属性值后, 点击新建/访问文件, 导航式建模系统将直接在建模项目系统中创建所有关联的空白模型, 如图中最左侧红框中的 karma 文件。此外, 如果该模型已存在, 则变为访问操作。通过这种方式能够提升“毅力号”火星车架构设计过程和方法论要求的一致性, 在零散的 KARMA 或其他格式模型文件之间建立依赖关系, 加强模型文件管理能力, 提升效率。

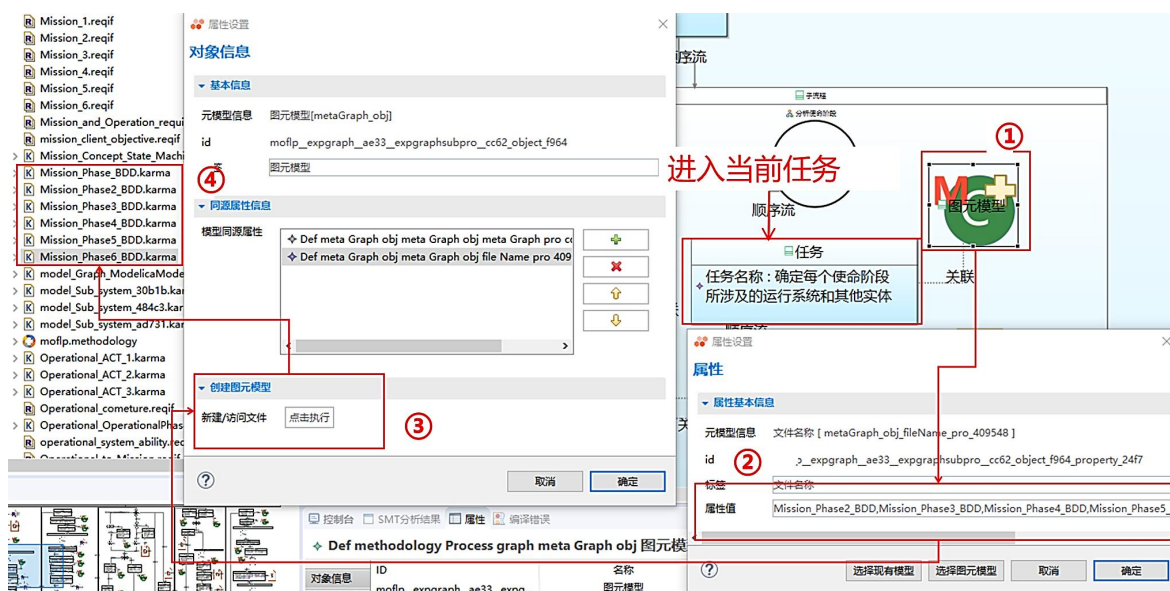


图 5.11 通过方法论流程模型创建/访问架构模型的工作流

此外, 在“毅力号”火星车物理架构设计过程中需要对各物理组件的子组件进行

详细设计，自上而下确立各系统级、子系统级的详细架构以及单机、零部件的初始选型及参数，便于后续进行仿真验证。由于火星车是涉及机械、电气、控制等相关领域子系统的多领域耦合系统，相比较于使用内部模块图或者参数图描述物理系统的架构，多领域建模语言 Modelica 更适合在早期对组件级设计展开仿真验证。因此，本文所构建的 MOFLP 方法论流程模型通过访问 KARMA 语言建模工具中构建的与 Modelica 模型具有相同语义的特定域模型库，支持关联到 Modelica 的特定域模型，从而在“毅力号”设计中涵盖 SysML 和 Modelica 的多架构模型。

表 5.4 “毅力号”火星车架构模型汇总

视角	模型名称	模型类型	文件类型
使命分析	使命、客户和目标定义表	需求管理	.reqif
	目标到相关实体/利益相关者追溯矩阵	设计结构矩阵	.dsm
	定义使命概念	图元模型（状态机图）	.karma
	前期准备	图元模型（模块定义图）	.karma
	发射火箭	图元模型（模块定义图）	.karma
	JPL 团队巡航	图元模型（模块定义图）	.karma
	火星 2020 航天器和毅力号进入、下降和着陆	图元模型（模块定义图）	.karma
	JPL 团队进行准备工作	图元模型（模块定义图）	.karma
	毅力号进行火星探索	图元模型（模块定义图）	.karma
	使命任务阶段	图元模型（包图）	.karma
	前期准备	图元模型（活动图）	.karma
	发射火箭	图元模型（活动图）	.karma
	JPL 团队巡航	图元模型（活动图）	.karma
	火星 2020 航天器和毅力号进入、下降和着陆	图元模型（活动图）	.karma
	JPL 团队进行准备工作	图元模型（活动图）	.karma
	毅力号进行火星探索	图元模型（活动图）	.karma
	前期准备	需求管理	.reqif
	发射火箭	需求管理	.reqif
	JPL 团队巡航	需求管理	.reqif
	火星 2020 航天器和毅力号进入、下降和着陆	需求管理	.reqif
	JPL 团队进行准备工作	需求管理	.reqif
	毅力号进行火星探索	需求管理	.reqif
	识别潜在的火星车系统	图元模型（模块定义图）	.karma
	分析能力覆盖率	设计结构矩阵	.dsm
	指定兴趣系统和上下文	图元模型（用例图）	.karma
	毅力号生命周期	图元模型（状态机图）	.karma
运行分析	运行阶段图	图元模型（状态机图）	.karma
	追溯运行阶段到参考使命阶段	需求管理	.reqif

功能分析	到达发射场	图元模型（模块定义图）	.karma
	安全到达火星表面	图元模型（模块定义图）	.karma
	进行科学探索研究	图元模型（模块定义图）	.karma
	到达发射场	图元模型（内部模块图）	.karma
	安全到达火星表面	图元模型（内部模块图）	.karma
	进行科学探索研究	图元模型（内部模块图）	.karma
	到达发射场	图元模型（活动图）	.karma
	安全到达火星表面	图元模型（活动图）	.karma
	进行科学探索研究	图元模型（活动图）	.karma
	确定运行任务所需的系统能力	需求管理	.reqif
	从运行行为中派生利益相关者需求	设计结构矩阵	.dsm
	利益相关者需求图	图元模型（需求图）	.karma
	实体到利益相关者需求图	图元模型（用例图）	.karma
	利益相关者需求到使命能力视点的效果追溯矩阵	设计结构矩阵	.dsm
	运行任务实现到相关运行任务追溯矩阵	需求管理	.reqif
	毅力号发射功能复合行为图	图元模型（活动图）	.karma
	协助毅力号安全着陆复合行为图	图元模型（活动图）	.karma
	毅力号安全着陆复合行为图	图元模型（活动图）	.karma
	控制毅力号和直升机进行火星探索复合行为图	图元模型（活动图）	.karma
	基本功能到复合功能追溯矩阵	设计结构矩阵	.dsm
	功能结构图	图元模型（模块定义图）	.karma
	细化复合功能图	图元模型（模块定义图）	.karma
	修正火星 2020 轨迹复合功能图	图元模型（活动图）	.karma
	提供所需动力复合功能图	图元模型（活动图）	.karma
	测试科学试验复合功能图	图元模型（活动图）	.karma
	测试飞行复合功能图	图元模型（活动图）	.karma
	控制毅力号复合功能图	图元模型（活动图）	.karma
	毅力号进行火星探索复合功能图	图元模型（活动图）	.karma
	静态功能架构图	图元模型（内部模块图）	.karma
	加速模态行为	图元模型（状态机图）	.karma
	测试地面行驶功能模态行为	图元模型（状态机图）	.karma
	测试岩石收集功能模态行为	图元模型（状态机图）	.karma
	减速模态行为	图元模型（状态机图）	.karma
	测试摄像功能模态行为	图元模型（状态机图）	.karma
	提供牵引力模态行为	图元模型（状态机图）	.karma
	提供向下反推力模态行为	图元模型（状态机图）	.karma
	登陆确认模态行为	图元模型（状态机图）	.karma
	提供驱动力模态行为	图元模型（状态机图）	.karma
	接收并传递数据模态行为	图元模型（状态机图）	.karma
	监测航天器模态行为	图元模型（状态机图）	.karma

逻辑架构分析	提供正确着陆方案模态行为	图元模型（状态机图）	.karma
	毅力号火星车下降过程模态行为	图元模型（状态机图）	.karma
	系统需求	图元模型（需求图）	.karma
	系统需求到相关功能和流追溯矩阵	设计结构矩阵	.dsm
	利益相关者需求到系统需求追溯矩阵	设计结构矩阵	.dsm
	逻辑分解结构图	图元模型（模块定义图）	.karma
	定义接口类型图	图元模型（模块定义图）	.karma
	分配功能到逻辑组件表	设计结构矩阵	.dsm
	逻辑组件内部结构图	图元模型（内部模块图）	.karma
	分配功能流到逻辑接口表	设计结构矩阵	.dsm
	分配需求	图元模型（需求图）	.karma
	追踪逻辑组件到分配需求表	设计结构矩阵	.dsm
	逻辑接口到分配需求追溯矩阵	设计结构矩阵	.dsm
	分配需求到系统需求追溯矩阵	设计结构矩阵	.dsm
	降落组件	图元模型（模块定义图）	.karma
	着陆组件	图元模型（模块定义图）	.karma
	行驶组件	图元模型（模块定义图）	.karma
	能源供给组件	图元模型（模块定义图）	.karma
	交流组件	图元模型（模块定义图）	.karma
	管理与运行组件	图元模型（模块定义图）	.karma
物理架构分析	避险组件	图元模型（模块定义图）	.karma
	导航组件	图元模型（模块定义图）	.karma
	声音记录组件	图元模型（模块定义图）	.karma
	样本收集组件	图元模型（模块定义图）	.karma
	样品处理组件	图元模型（模块定义图）	.karma
	科学试验组件	图元模型（模块定义图）	.karma
	视觉组件	图元模型（模块定义图）	.karma
	温度控制组件	图元模型（模块定义图）	.karma
	机械臂保护组件	图元模型（模块定义图）	.karma
	样本研究分析和寻找组件	图元模型（模块定义图）	.karma
	躯体组件	图元模型（模块定义图）	.karma
	描述物理组件特征参数	图元模型（参数图）	.karma
	定义物理接口	图元模型（模块定义图）	.karma
	毅力号内部结构	图元模型（内部模块图）	.karma
	物理组件分组	表	.table
	毅力号重量指标验证	代码生成	.karma
	毅力号总价格指标验证	指标验证	.karma
	逻辑组件映射到物理组件表	设计结构矩阵	.dsm
	火星车行驶组件特定域模型	图元模型（Modelica）	.karma
	火星车行驶组件代码生成	代码生成	.karma

（3）辅助功能

实际应用中，导航式建模系统的辅助功能是为了协助设计人员在方法论流程模型

中按流程使用操作，相对于单独通过建模工具自身执行操作，这样的好处在于：1) 模型中绘制的流程元素清晰地指明了不同任务要执行的工作，设计活动有据可依；2) 设计人员可以完全停留在方法论流程模型中，避免在执行不同功能时，反复切换模型的问题，提高架构设计自动化程度。以上二者，都能够避免设计活动中出现漏操作或操作顺序出错的问题。

以“毅力号”火星车的架构设计过程中下降过程模态行为的仿真过程为例，验证导航式建模技术在支持辅助功能上的可行性和性能。

毅力号下降火星的过程极为困难，被称为“恐怖七分钟”，这是因为火星车自身与地球的通信过程远超过这七分钟，因此整个下落过程要依靠火星车自身处理计算，而且目标着陆点是一个极为复杂的撞击盆地（Jezero Crater）。为了保障火星车下降过程的可靠性和安全性，通过流程模型在早期设计阶段对下降过程的功能模态行为进行仿真，验证方案的可行性。

火星车下降过程是一个复杂的曲线运动，在概念设计阶段难以通过功能模态行为模拟整个曲线流程，本研究取下降过程最后接近于直线运动的部分进行仿真，包括后壳分离、下降级发动机关闭、火星车分离和着陆，不考虑从火星接入到调姿到求解着陆点的过程。表 5.5 参考 NASA 毅力号任务的模拟数据 (<https://eyes.nasa.gov/apps/mars2020/>)，将最后的下降过程视为匀加速/减速运动，此外后壳分离状态根据实际情况细分为短暂加速、自由飞行、垂直下落和快速下落四个状态，并以飞入火星进入点为时间 0 点。

表 5.5 NASA 公布的“毅力号”下降过程数据

功能模态行为	距离 (km)	高度 (km)	速度 (km/h)	时间 (s)	描述
	2.23	2.18	293.51	+344	下降级从降落伞上分离
后壳分离	1.99	1.94	301.88	+3	下降级开始在火星上空自由飞行，以降低垂直速度并消除水平速度
	0.243	0.24	115.31	+29	下降级基本到目标上空
	0.120	0.118	115.31	+4	下降级快速下降
发动机关闭	0.0248	0.02201	6	+5	当火星车距离地面 21 米时，一半的发动机关闭，下降阶段趋于平稳，最终速度达到 2.7 km/h
	5				火星车从下降级被释放，并在一组延伸至 20 多英尺长的电缆上下降。在此过程中，火星车解锁并展开其车轮。
毅力号分离	0.02297	0.01998	2.74	+2	

着陆	0.0080 2	0	0	+16	火星车降落在火星上。
----	-------------	---	---	-----	------------

如图 5.12 所示，基于设计的下降过程，建立状态机图进行行为描述。状态机图从初始状态出发，依次转换为后壳分离短暂加速状态、后壳分离自由飞行状态、后壳分离垂直下落状态、后壳分离快速下落状态、平稳下降状态、毅力号分离状态，最后转换为最终状态，即毅力号着陆。

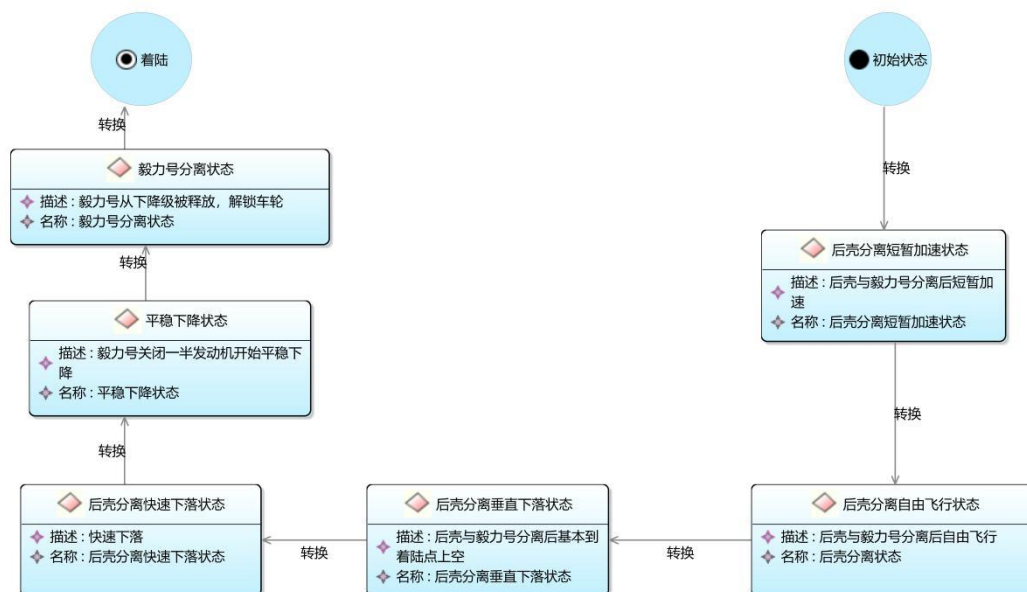


图 5.12 毅力号火星车下降过程状态机图

如图 5.13 所示，当方法论流程进入到当前任务时，表示进入模态行为仿真验证节点。点击当前任务关联的混合状态机仿真对象以打开其属性栏，点击编译文件实现对毅力号火星车下降过程模态行为状态机图的编译，最后仿真该编译后的文件获得该行为的仿真结果；传统过程中，设计人员需要先查找当前流程下应执行什么任务，然后根据该仿真任务找到对应的状态机图，最后对其进行编译和仿真，涉及到模型的反复切换和查找，这对于大型项目来说是极其耗费时间的。根据图 5.14 仿真结果，“毅力号”着陆时，即第 407 s，毅力号火星车高度为 14.88 m（应为 0，但对于概念阶段的简化模型来说，误差在可接受范围内），验证了下降过程设计基本合理。同时结果表明，利用导航式建模工具在流程模型中一站式地执行 KARMA 语言建模工具提供的辅助功能是可行且有效的。



图 5.13 通过方法论流程模型执行行为仿真的 workflow

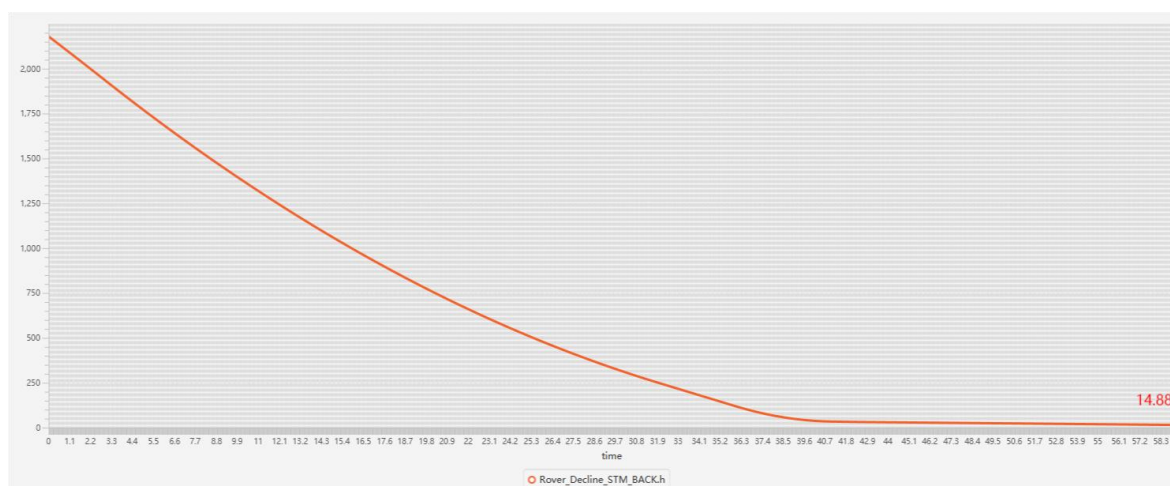


图 5.14 毅力号下降过程高度仿真结果

(4) 外部仿真

导航式建模系统外部仿真能力用于关联建模工具之外的特定域仿真模型，并执行仿真。例如，流程模型通过多物理场仿真对象关联“毅力号”火星车的 Modelica 仿真，实现一体化设计。本研究以此为例，验证导航式建模技术外部仿真的可行性和性能。

首先在流程模型自动创建的火星车行驶组件特定域模型的基础上建模，该模型用于模拟火星车在火星表面进行科学探索任务时，不同路面情况下，维持正常行驶状态所需要的驱动扭矩。根据 NASA 发布的“毅力号”相关的资料^[75]，“毅力号”质量约为 1025 kg，车轮直径 0.2625 m（共六个），长×宽×高：3 m×2.7 m×2.2 m，重力加速度取 3.27 m/s^2 （约地球重力加速度的 1/3）。滚动摩擦系数取 0.3，地表平均风速为 5 m/s。为了简化模型，本研究将“毅力号”假定为质量均匀的刚体，此外“毅力号”悬架系统始终能够保持车轮上压力均匀，因此仿真模型只考虑单个车轮，假定

车轮不打滑，动力学模型需要考虑三种阻力。

1) 空气阻力 f_{Drag} :

$$f_{\text{Drag}} = C_d * \rho * A * (v - v_{\text{Wind}})^2 / 2 \quad (5.1)$$

其中 C_d 为空气阻力系数，取 0.5； ρ 为火星表面空气密度，取 0.02 kg/cm^3 ； A 为行驶方向的横截面积取 $2.7 * 2.2 = 5.94 \text{ m}^2$ ； v (m/s) 为火星车的行驶速度； v_{Wind} 为火星表面的风速 5 m/s 。

2) 滚动摩擦力 f_{Roll} :

$$f_{\text{Roll}} = C_r * m * g * \cos(\text{atan}(\text{inclination})) \quad (5.2)$$

其中 C_r 为滚动摩擦系数，取 0.5； m 为质量，由于问题简化为一个轮子的仿真，取 $1025/6 = 171 \text{ kg}$ ； g 为火星表面重力加速度， 3.27 m/s^2 ； inclination 为坡度值（倾角的正切值）。

3) 坡度阻力 f_{Grav} :

$$f_{\text{Grav}} = m * g * \sin(\text{atan}(\text{inclination})) \quad (5.3)$$

其中参数同上。

结合上述参数和数学模型，构建的该系统的详细仿真模型如图 5.15 所示，系统的输入是一个三维数组，第一列为时间；第二列为按时间分布的坡度值（倾角的正切值），用于模拟火星车行驶的路面，作为滚动摩擦力和坡度阻力计算的输入参数；第三列为按时间分布的标称扭矩（定义为没有坡度和风速情况下阻力之和乘半径的值）的倍数，用于驱动理想情况下“毅力号”的质量块。质量块连接一个理想状态下的轮子，并与形成空气阻力、滚动摩擦力和坡度阻力的三组组件连接。仿真从 0 时刻 0 m/s 的速度开始，为了保持需求规定的 0.042 m/s 的行驶速度，火星车车轮不同时刻马达输出的扭矩不同。

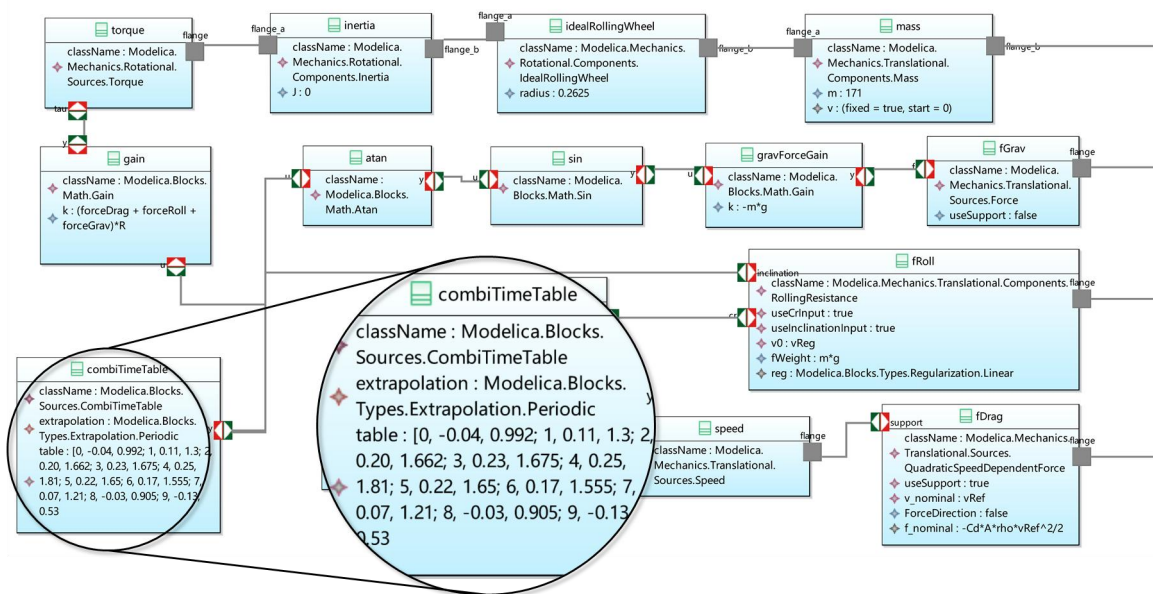


图 5.15 动力系统仿真模型

因为在 KARMA 语言建模工具中, 构建的架构模型尽管具备足够的语义表达能力, 但是其不能被特定域仿真工具识别, 如 OpenModelica 工具。为了保证数据的一致性, 本研究使用导航式建模系统的代码生成功能, 如图 5.16 所示, 当流程进入到该任务时, 点击相应的代码生成对象, 执行该对象所对应模型的代码生成, 随后系统将自动在计算机的指定位置生成火星车行驶性能仿真的特定域架构模型对应的 Modelica 模型, 此模型转换的详细技术路线参考 2.4.2。

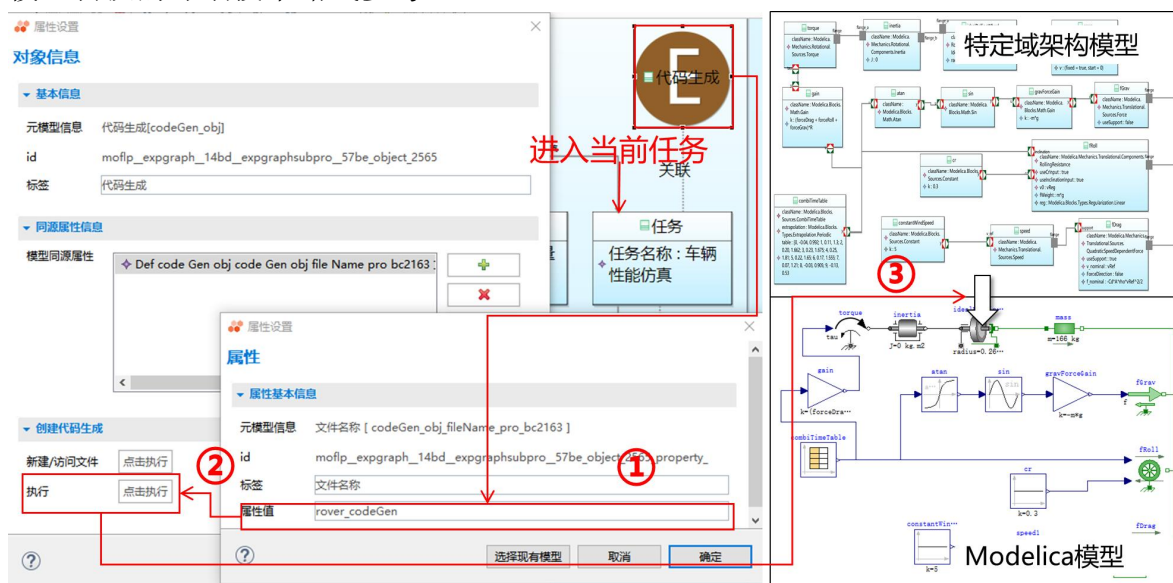


图 5.16 通过方法论流程模型生成 Modelica 模型的工作流

执行完上述代码生成操作后将获得可执行的火星车行驶组件仿真模型, 继续在方

方法论流程模型中点击多物理仿真对象，并执行所获得的模型文件，系统将调用 OpenModelica 工具执行火星车行驶性能的仿真（图 5.17）。

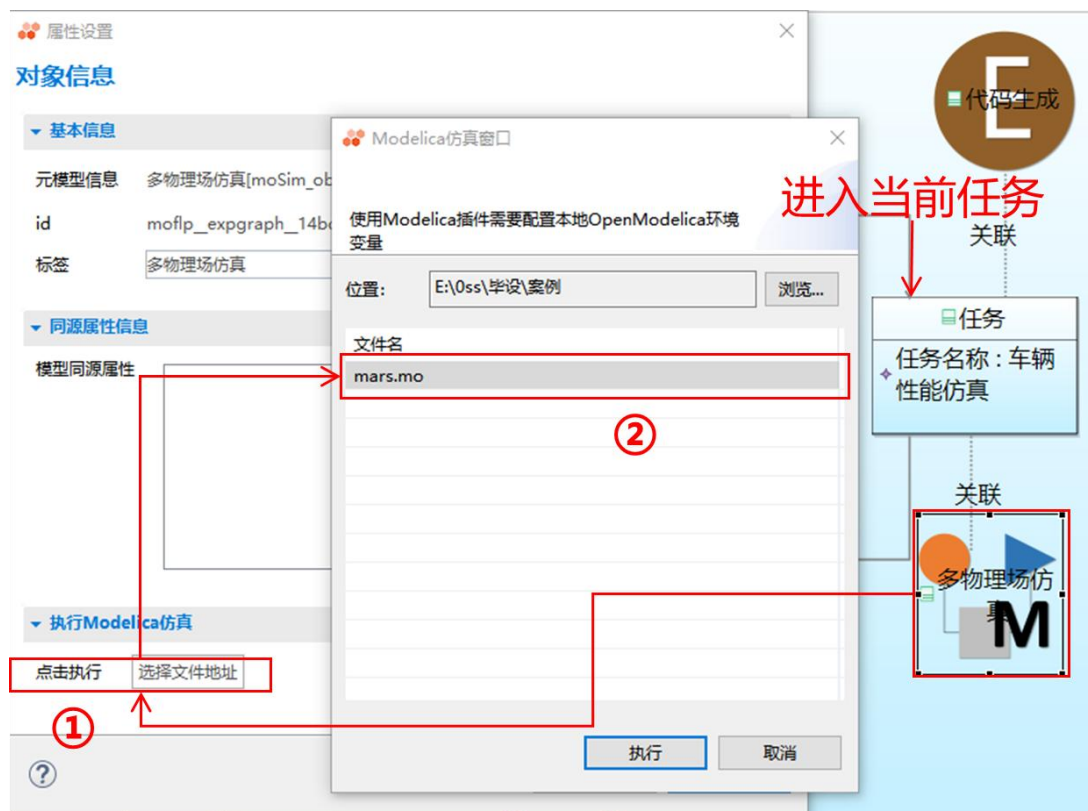


图 5.17 通过方法论流程模型执行 Modelica 仿真的 workflow

最终结果如图 5.18，红色实线为速度曲线。根据输入的扭矩，火星车在该地形下基本保持了 0.042 m/s 的速度，输出的扭矩满足需要。

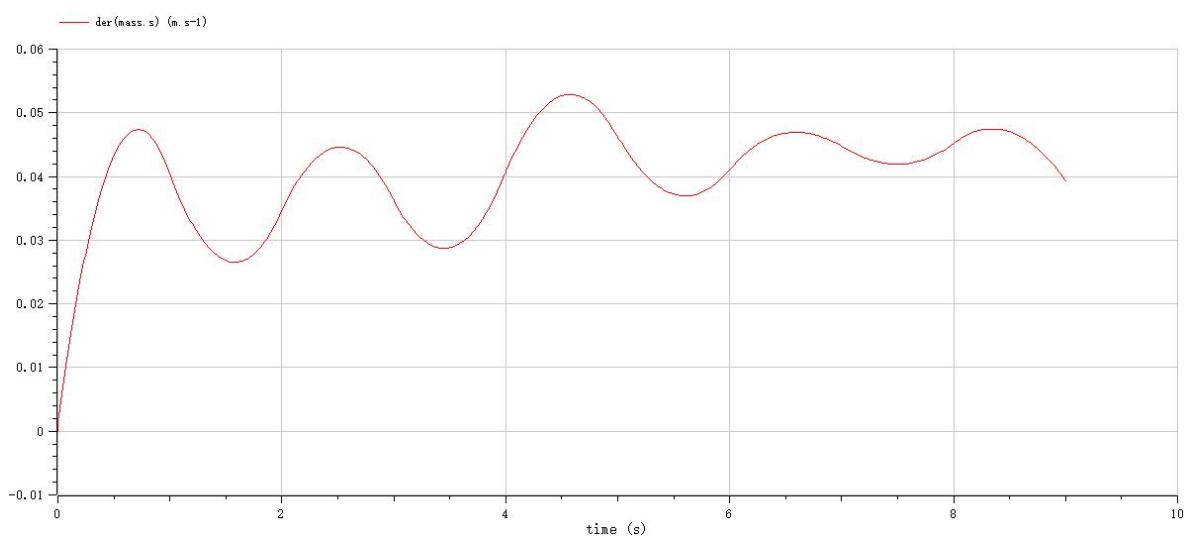


图 5.18 行驶速度的仿真结果

5.4.4 应用总结

本案例以“毅力号”火星车为对象，在概念设计过程中应用了本研究所设计的导航式建模系统。如表 5.6 所示，相对于直接基于 KARMA 的设计流程，本文提供了定制化开发建模方法论的专用建模语言，保障了方法论流程的规范性，通过系统模型导览、辅助架构模型快速生成、驱动 KARMA 语言建模工具功能执行和外部特定域工具仿真等功能，解决了方法论在实际模型设计中驱动能力弱的问题，减少了设计人员的操作量，通过严谨的流程编排降低了设计难度，这在面对大数量级的模型时优势更加明显。

表 5.6 与基于 KARMA 的设计流程对比

设计流程 操作	基于 KARMA 的设计流程	基于导航式建模系统的设计流程
方法论定制化	利用非标准语言设计方法论	提供专用方法论建模语言
模型转换	单独编写脚本生成文件结构	支持流程模型、文件结构模型和 XML 模型的相互转换
架构模型生成	离开方法论模型，进入项目文件界面，在该界面逐个创建模型	在导航模型中直接创建，支持同时生成多个模型
各类功能执行	离开方法论模型，在项目文件中找到对应文件后执行功能	根据流程指引，在导航模型中直接操作。
总结	没有标准的流程指引，设计过程涉及模型、工具的连续切换，操作繁琐	导航模型提供标准的流程指引，提供集多种系统设计操作于一体的服务

5.5 本章小结

本章从工程应用角度在 KARMA 语言建模平台的基础上，以插件形式设计并开发了导航式建模系统，提供良好的设计交互界面用于支持模型文件管理、导航式建模和驱动架构设计的服务。然后基于该系统开展了定量和定性分析验证，结果表明该系统支持 DoDAF、MagicGrid、OOSEM、Harmony-SE、Arcadia 五种不同形式的方法论的表达，具备较强的定制化设计能力。相对于其他建模工具的方法论功能模块，具有更强的驱动架构设计能力。通过在“毅力号”火星车架构设计的应用，验证了系统在实际架构设计过程中能够显著减少设计人员操作，提升设计效率。

结论

MBSE 是智能制造发展实施的重要一环，是数字化转型的核心要素，而方法论是 MBSE 的支柱之一。考虑到 MBSE 应用从实验室级向航空航天等重大装备领域转变，设计人员更加关注“如何设计正确的系统”。因此，本文对方法论支持复杂装备系统设计的问题进行重点研究。在综合分析国内外关于 MBSE 和方法论的相关研究的基础上，提出了复杂装备系统导航式建模技术，对与方法论相关的建模和模型转换问题进行深入研究。导航式建模是 MBSE 应用深入推进的结果，对导航式建模的深入研究有助于型号设计人员更好地应对日渐复杂的产品，提升设计效率，确保设计“对”的产品，为 MBSE 应用推进创造更大的价值。论文的主要内容总结如下：

（1）研究了复杂装备系统导航式建模技术。针对当前研究缺乏从模型角度有效使用方法论的问题，提出了一种复杂装备系统导航式建模技术。首先，根据 KARMA 语言建模工具和 MBSE 设计流程，总结了导航式建模技术的设计需求。然后，分析了导航式建模的技术实现，提出了结合数据建模和元对象机制的技术框架，保障了导航式建模语言术语、元模型定义的全面性和一致性。最后，探讨了所提技术在整个系统设计周期中的工作流。

（2）研究了面向导航式建模的元模型。针对当前方法论在模型元素表达过程中缺乏一种专用于建模的元模型的问题，提出基于元建模构建导航式建模元模型，包括概念数据模型和逻辑数据模型设计两个阶段。概念数据模型基于实体-关系图，旨在保障元模型定义的全面性。在概念数据模型的基础上，逻辑数据模型基于 GOPRR-E 元模型构建，以采用丰富的图形元素充分描述方法论的设计流程。

（3）研究了导航式建模语言和模型转换。为了保证构建的方法论流程模型能够为计算机所识别、存储和高效显示，提出基于 XML 设计导航式建模语言。在定义 XML 模型的同时，分析了其与流程模型、文件结构之间的模型转换方法，设计了模型转换算法，支持模型自动转换的同时也保障了模型的一致性。

（4）设计开发了复杂装备系统导航式建模系统并验证。从工程应用角度以插件形式在 KARMA 语言建模平台的基础上，设计并开发了导航式建模系统。然后基于该系统开展了定量和定性分析验证，结果表明该系统具备较强的定制化设计能力。相对于其他建模工具的方法论功能模块，具有更强的驱动架构设计能力。通过工程实例应

用，验证了系统在实际架构设计过程中能够显著减少设计人员操作，提升设计效率。

本文的创新点如下：

(1) 复杂装备系统导航式建模语言：针对 MBSE 方法论应用中缺乏专门手段对方法论进行建模的问题，提出了复杂装备系统导航式建模语言，建模语言围绕方法论、国际标准和 KARMA 语言功能体系设计。相对于传统建模语言，导航式建模语言兼容图形和文本两种表达形式，通过利用 GOPRR-E 元元模型构建的元模型建模，支持方法论的流程模型表达，提升了数据管理和维护的效率；利用 XML 自描述性的优势，支持对上述流程模型的数据存储，并实现对方法论流程模型的文本描述。可读的文本内容提升了方法论文件的使用效率，直接通过 XML 文本式的数据交流省去了借助特定建模工具或使用整个项目模型展示的过程，推进了设计文件的轻量化，提升了协作能力。

(2) 方法论和建模工具的深度融合方法：针对现有建模工具中侧重于对方法论的展示而实际应用的功能性较弱的问题，提出了方法论和建模工具的深度融合方法。设计了方法论表达常用的三种模式，包括流程模型、文件结构和 XML 模型，支持三种模式的模型自动转换，减少了设计人员构建不同形式的方法论模型的时间。功能上导航模型支持与系统全生命周期模型的集成，包括需求、功能、逻辑、物理等系统模型，表单、甘特图等插件，代码生成、架构驱动、SMT 等功能以及 Modelica 等仿真工具，实现了系统设计全生命周期模型快速创建、导航与执行，提升了系统开发过程中设计、分析和验证的效率。

导航式建模是 MBSE 应用深入推进的结果，具有辅助设计人员开展快速且有效的架构设计的功能。虽然本文对复杂装备系统导航式建模技术进行了深入的研究，但是这方面研究工作还处于起步阶段。在导航式建模付诸实践之前，还有很多理论和技术工作需要做，未来的研究工作可以从以下几个方面展开：

(1) 尽管复杂装备系统导航式建模提升了建模方法论在架构设计过程中的应用价值，但是对于如何将传统复杂装备系统实践经验固化，形成一套符合 MBSE 设计要求的方法论，本文只做了简单探讨，后续值得深入挖掘。

(2) 本研究在方法论驱动系统架构设计的过程中，只关注了导航模型到系统架构设计的过程，并没有反过来对系统架构模型是否满足了方法论的要求进行考虑，未来的研究工作可以从这方面深入，加强导航模型检验系统架构设计的能力，进一步确保设计的正确性。

(3) 本文提出的复杂装备系统导航式建模技术理论上能够支持集成系统全生命周期过程中产生的所有模型,然而在本文中只重点关注与 KARMA 语言建模工具内的模型和部分特定域工具 (Modelica 等) 的模型集成。在后续研究中,需要通过案例实现多种工具和多种模型的集成,例如 CAD、CAE 模型等。

参考文献

- [1] Harvey D, Liddy T. Architecting the Problem Space - Model-Based User Needs Analysis: Proceedings of the 4th Asia Pacific Conference on Research in Industrial and Systems Engineering (APCORISE'14)[C], 2014.
- [2] Heihoff-Schwede J, Kaiser L, Dumitrescu R. An MBSE-based approach for the analysis of requirements towards engineering IT architectures: 2019 International Symposium on Systems Engineering (ISSE)[C], 2019. IEEE.
- [3] Lu J, Wen Y, Liu Q, et al. MBSE Applicability Analysis in Chinese Industry[J]. INCOSE International Symposium, 2018,28(1):1037-1051.
- [4] Lu J, Chen D J, Gürdür D, et al. An Investigation of Functionalities of Future Tool - chain for Aerospace Industry[J]. INCOSE International Symposium, 2017,27(1):1408-1422.
- [5] INCOSE. Vision 2020 (INCOSE-TP-2004-004-02)[Z]. 2007.
- [6] 李晓红, 常家辉, 徐可. MBSE 深化应用加速武器系统研制数字化转型[J]. 国防科技, 2022,43(4):59-64.
- [7] 马启铭. 基于模型的无人机控制站系统架构的研究与实现[D]. 吉林大学, 2020.
- [8] Haskins C. 4.6. 1 A historical perspective of MBSE with a view to the future: IncoSE international symposium[C], 2011. Wiley Online Library.
- [9] Linden A, Fenn J. Understanding Gartner's hype cycles[J]. Strategic Analysis Report N° R-20-1971. Gartner, Inc, 2003,88:1423.
- [10] Chami M, Bruel J. A survey on MBSE adoption challenges[J]. 2018.
- [11] Oliver D W. Descriptions of systems engineering methodologies and comparison of information representations: INCOSE International Symposium[C], 1993. Wiley Online Library.
- [12] Wayne W A. Model-Based systems engineering[Z]. CRC Press, Ann Arbor MI, 1993.
- [13] Dori D. Analysis and Representation of Computer Vision Systems by the Object-Process Methodology.: MVA[C], 1994.
- [14] Fowler M. UML distilled: a brief guide to the standard object modeling language[M]. Addison-Wesley Professional, 2004.
- [15] Friedenthal S, Moore A, Steiner R. A practical guide to SysML: the systems modeling language[M]. Morgan Kaufmann, 2014.
- [16] Douglas B P. Harmony MBSE Modeling Standards for use with UML, SysML, and Rhapsody[J]. IBM Rational, San Jose, CA, USA, 2015.
- [17] Kaslow D, Soremekun G, Kim H, et al. Integrated model-based systems engineering (MBSE) applied to the Simulation of a CubeSat mission: 2014 IEEE Aerospace Conference[C], 2014. IEEE.
- [18] Claver C F, Dubois-Felsmann G, Delgado F, et al. Using SysML for MBSE analysis of the LSST system: Modeling, Systems Engineering, and Project Management for Astronomy IV[C], 2010. SPIE.
- [19] Bonnet S, Voirin J L, Normand V, et al. Implementing the mbse cultural change: Organization, coaching and lessons learned: INCOSE International Symposium[C], 2015. Wiley Online Library.
- [20] Kelly S, Lyytinen K, Rossi M. MetaEdit+ a fully configurable multi-user and multi-tool CASE and CAME environment[M]//Seminal contributions to information systems engineering. Springer,

- 2013:109-129.
- [21] Collaboration O S F L. Oslc primer[J]. OSLC Primer-Learning the Concepts of OSLC, 2008:30.
 - [22] Zimmerman P, Gilbert T, Salvatore F. Digital engineering transformation across the Department of Defense[J]. The Journal of Defense Modeling and Simulation, 2019,16(4):325-338.
 - [23] Negri E, Fumagalli L, Macchi M. A review of the roles of digital twin in CPS-based production systems[J]. Procedia manufacturing, 2017,11:939-948.
 - [24] Spangelo S C, Kaslow D, Delp C, et al. Applying Model Based Systems Engineering (MBSE) to a standard CubeSat[C], 2012. IEEE, 2012.
 - [25] Spangelo S C, Cutler J, Anderson L, et al. Model based systems engineering (MBSE) applied to Radio Aurora Explorer (RAX) CubeSat mission operational scenarios[C], 2013. IEEE, 2013.
 - [26] Anderson L, Cole B, Yntema R, et al. Enterprise modeling for CubeSats[C], 2014. IEEE, 2014.
 - [27] Kaslow D, Ayres B, Cahill P T, et al. Developing a CubeSat Model-Based System Engineering (MBSE) reference model - Interim status #3[C], 2017. IEEE, 2017.
 - [28] Kaslow D, Ayres B, Cahill P T, et al. A model-based systems engineering approach for technical measurement with application to a CubeSat[C], 2018. IEEE, 2018.
 - [29] 王崑声, 袁建华, 陈红涛, 等. 国外基于模型的系统工程方法研究与实践[J]. 中国航天; Aerospace China, 2012(11%V):52-57.
 - [30] Santosa K, Maerani R. Decomposed Functional Behavior of Helium Purification Support System for Experimental Power Reactor (RDE) Type Using Sequence Diagram: Journal of Physics: Conference Series[C], 2019. IOP Publishing.
 - [31] Wolny S, Mazak A, Carpella C, et al. Thirteen years of SysML: a systematic mapping study[J]. Software and Systems Modeling, 2020,19(1):111-169.
 - [32] Dori D, Kohen H, Jbara A, et al. OPCloud: An OPM Integrated Conceptual - Executable Modeling Environment for Industry 4.0[J]. Systems Engineering in the Fourth Industrial Revolution, 2019:243-271.
 - [33] Seal D. The System Engineering 'V' - Is it Still Relevant in the Digital Age?[J]. Boeing Company, Global Product Data Interoperability Summit, Presentation, 2018.
 - [34] Margaria T, Schieweck A. The digital thread in industry 4.0: International Conference on Integrated Formal Methods[C], 2019. Springer.
 - [35] McDermott T, DeLaurentis D, Beling P, et al. AI4SE and SE4AI: A research roadmap[J]. Insight, 2020,23(1):8-14.
 - [36] Lu J, Wang G, Ma J, et al. General Modeling Language to Support Model - based Systems Engineering Formalisms (Part 1): INCOSE International Symposium[C], 2020. Wiley Online Library.
 - [37] Estefan J A. Survey of model-based systems engineering (MBSE) methodologies[J]. Incose MBSE Focus Group, 2007,25(8):1-12.
 - [38] 胡峻豪, 冯雷, 朱睿, 等. 支持 MBSE 的系统全过程设计应用框架研究与实现[J]. 航空制造技术, 2016,59(13):105-109.
 - [39] Hoffmann H. Deploying model-based systems engineering with IBM® rational® solutions for systems and software engineering: 2012 IEEE/AIAA 31st Digital Avionics Systems Conference (DASC)[C], 2012. IEEE.
 - [40] Yang Z, Du H, Liu Y, et al. Use the Harmony-SE Approach to Extend the Advantages of MBSE: 2021 IEEE 16th Conference on Industrial Electronics and Applications (ICIEA)[C], 2021. IEEE.

- [41] Lykins H, Friedenthal S, Meilich A. 4.4. 4 Adapting UML for an Object Oriented Systems Engineering Method (OOSEM): INCOSE International Symposium[C], 2000. Wiley Online Library.
- [42] Waseem M, Sadiq M U. Application of model-based systems engineering in small satellite conceptual design-A SysML approach[J]. IEEE Aerospace and Electronic Systems Magazine, 2018,33(4):24-34.
- [43] Roques P. MBSE with the ARCADIA Method and the Capella Tool: 8th European Congress on Embedded Real Time Software and Systems (ERTS 2016)[C], 2016.
- [44] Bonnet S, Voirin J, Exertier D, et al. Not (strictly) relying on SysML for MBSE: Language, tooling and development perspectives: The Arcadia/Capella rationale: 2016 Annual IEEE Systems Conference (SysCon)[C], 2016. IEEE.
- [45] Dori D. Object-process methodology[M]//Encyclopedia of Knowledge Management, Second Edition. IGI Global, 2011:1208-1220.
- [46] Morkevicius A, Aleksandraviciene A, Armonas A, et al. Towards a common systems engineering methodology to cover a complete system development process: INCOSE International Symposium[C], 2020. Wiley Online Library.
- [47] 张贺, 魏强, 陈余军, 等. 基于 SysML 的通信卫星转发器系统设计[J]. 环境技术; Environmental Technology, 2019,37 (2):106-112.
- [48] Long J E. Systems Engineering (SE) 101: CORE: Product & Process Engineering Solutions[J]. Vitech training materials, Vitech Corporation, Vienna, VA, 2000.
- [49] Long J E. MBSE in Practice: Developing Systems with CORE[J]. Vitech briefing slides, Vitech Corporation, Vienna, VA, 2007.
- [50] Mittal S. Extending DoDAF to allow integrated DEVS-based modeling and simulation[J]. The Journal of Defense Modeling and Simulation, 2006,3(2):95-123.
- [51] Lin S, Wu X. Mission Reliability Modeling for Equipment System Based on the Operational Viewpoint of DoDAF 2.0: International Conference on Logistics Engineering, Management and Computer Science (LEMCS 2014)[C], 2014. Atlantis Press.
- [52] Sango M, Vallée F, Vié A, et al. MBSE and MBSA with Capella and safety architect tools: International Conference on Complex Systems Design & Management[C], 2016. Springer.
- [53] Mazeika D, Morkevicius A, Aleksandraviciene A. MBSE driven approach for defining problem domain: 2016 11th System of Systems Engineering Conference (SoSE)[C], 2016. IEEE.
- [54] Biggs G, Sakamoto T, Fujiwara K, et al. Experiences with model-centred design methods and tools in safe robotics: 2013 IEEE/RSJ International Conference on Intelligent Robots and Systems[C], 2013. IEEE.
- [55] 刘秀, 孔祥银, 张海军. 国外基于模型的系统工程实践应用[J]. 中国核工业, 2022(6):43-44.
- [56] 姬晓慧, 陈国定. MBSE 方法论实施方法研究[J]. 中国新技术新产品, 2022(4):33-38.
- [57] Z. L, G. W, J. L, et al. Bibliometric Analysis of Model-Based Systems Engineering: Past, Current, and Future[J]. IEEE Transactions on Engineering Management, 2022:1-18.
- [58] Ma J, Wang G, Lu J, et al. Semantic Modeling Approach Supporting Process Modeling and Analysis in Aircraft Development[J]. Applied Sciences, 2022,12(6):3067.
- [59] Guo J, Wang G, Lu J, et al. General modeling language supporting model transformations of mbse (part 2)[C], 2020. Wiley Online Library, 2020.
- [60] Hall, ArthurDavid. A methodology for systems engineering[M]. A methodology for systems

- engineering, 1962.
- [61] A. D H. Three-Dimensional Morphology of Systems Engineering[J]. IEEE Transactions on Systems Science and Cybernetics, 1969,5(2):156-160.
- [62] 段海波. 从霍尔模型技术系统的发展进化看传统系统工程到现代系统工程的演变[J]. 智能制造, 2016(Z1):88-95.
- [63] Estefan J. Survey of Candidate Model-Based Systems Engineering (MBSE) Methodologies: INCOSE-TD-2007-003-02[Z]. Seattle, WA, USA: International Council on Systems Engineering (INCOSE), 2008.
- [64] 王西超, 曹云峰, 丁萌, 等. 元对象机制驱动的复杂系统开放式顶层建模[J]. 电子科技大学学报, 2012,41(04):482-490.
- [65] Simsion G, Witt G. Data modeling essentials[M]. Elsevier, 2004.
- [66] 张津, 闫龙, 宫磊, 等. 数据建模方法标准及其在装备管理领域中的应用建议[J]. 航空标准化与质量, 2021(06):31-37.
- [67] Franke H J, Löffler S, Deimel M. The database "Methodos" assists an effective application of design methods[C], 2003.2003.
- [68] 陆福德. 基于 MBSE 的动车组车载 ATP 系统建模与验证[D]. 兰州交通大学, 2022.
- [69] Forsberg K, Mooz H. The relationship of systems engineering to the project cycle[J]. Engineering Management Journal, 1992,4(3):36-43.
- [70] Chen P P. The entity-relationship model—toward a unified view of data[J]. ACM transactions on database systems (TODS), 1976,1(1):9-36.
- [71] Kern H, Hummel A, Kühne S. Towards a comparative analysis of meta-metamodels[C], 2011.2011.
- [72] Martin J N. Systems engineering guidebook: A process for developing systems and products[M]. CRC press, 2020.
- [73] Flowers R, Edeki C. Business process modeling notation[J]. International Journal of Computer Science and Mobile Computing, 2013,2(3):35-40.
- [74] 南松辉, 田佳, 张海波, 等. 基于 UML 和 XML Schema 的航天飞行数据建模[J]. 兵工自动化, 2008,27(1):46-48.
- [75] von Ehrenfried M D. Perseverance and the Mars 2020 mission: Follow the science to Jezero Crater[M]. Springer Cham, 2022.

攻读学位期间发表论文与研究成果清单

发表论文：

- [1] Li Z, Wang G, Lu J, Broo DG, Kiritsis D, Yan Y. Bibliometric Analysis of Model-Based Systems Engineering: Past, Current, and Future[J]. IEEE Transactions on Engineering Management, 2022. (SCI, 论文第 1 章)
- [2] 李子航,王国新,马君达,等.复杂系统的 MBSE 建模及可靠性评估方法[J].国防科技大学学报, 2023. (已录用, EI, 论文第 2、5 章)
- [3] Li Z, Lu J, Wang G, et al. A bibliometric analysis on model-based systems engineering[C]. In 2021 IEEE International Symposium on Systems Engineering (ISSE), 2021, pp. 1-8. (EI)
- [4] 李子航,王国新,马君达,等.基于贝叶斯网络的复杂系统可靠性设计方法[C]. 第四届体系工程学术会议, 2022. (已录用, 获学术会议优秀论文一等奖, 推荐至国防科技大学学报发表)

参与科研项目：

- [1] 2020-2023 年. 科技部国家重点研发计划项目“复杂产品设计制造服务一体化系统建模理论”(参与)
- [2] 2021-2024 年. 装备预先研究共用技术项目“多系统建模与设计空间智能决策技术”(参与)

相关竞赛：

- [1] 2021 年 9 月, 北京理工大学第十七届“世纪杯”学生创业竞赛银奖
- [2] 2021 年 9 月, 第六届中国国际“互联网+”大学生创新创业大赛国赛入围
- [3] 2021 年 6 月, 北京市优秀创业团队一等奖
- [4] 2022 年 6 月, 第十三届“挑战杯”中国大学生创业计划竞赛北京理工大学校级金奖(团队负责人)

致谢

硕士三年经历了许多，也成长了许多。其间在上海出差半年，后又因疫情在家耽搁半年，一路上困难重重，痛并快乐着。借毕业论文的最后，我由衷地向各位老师、同学、朋友和家人表达感谢。

感谢我的导师王国新老师。在对待学术上，王老师治学严谨、一丝不苟，PPT、文章的每一个字句都会仔细过目，总是能一阵见血地指出我的问题。在生活中，王老师十分平易近人，经常关心同学们的生活，与王老师沟通从未感觉有任何障碍。作为课题组的负责人，王老师永远能在百忙之中为我答疑解惑。王老师对待科研和生活的态度始终是我学习的榜样。

感谢鲁金直博士。尽管鲁博士并非北理工的老师，但是他仍然认真负责地帮助我修改文章，制定研究方向，攻破技术难题。我各个学术成果的产出都离不开鲁博士的指导，再次感谢。

感谢遇到的各位同学。感谢马君达师兄，经常在学习工作中罩着我们。感谢师兄师姐，郭江敏、余士严、靳益利、栗寒、陈婧琦，为我提供了学习生活中的各种资料，让我少走了许多弯路，带我快速融入课题组的生活。感谢同级生丁洁、吴绶玄，和我在科研的路上相互勉励、相互提醒、相互分享。感谢师弟师妹，董梦如、陈锐、刘智清、李嘉伟，为我分担了许多科研压力，也为课题组的团建活动出谋划策。感谢支委孙延劭、张馥琳与我一同为支部建设出力。感谢我的室友张非凡、王新宇、赖丰羽、张仕科三年来的照顾，和谐的宿舍氛围总是能让我得到放松。感谢其余课题组的同级生们，有缘再见！

感谢我的父母对我各方面的支持，尊重我的选择。过去疫情那段时间里，没法返校一直在家，因此在家休息了大半年，也是自上大学以来在家最久的一段时间，也感谢能在那段时间陪在他们身边，家始终是最温馨的地方。

感谢我的女朋友阎思艺始终陪伴着我，陪我一起成长，让我在任何时候、任何事情都有能倾诉的对象，让我永远不会感到孤独。尽管她并非相关的研究者，但是依然能在写作方面为我出谋划策。

感谢各位评审专家在百忙之中抽出时间评阅本篇论文，您的宝贵意见和建议对我十分重要，是我论文质量升华的关键。

最后，再次感谢所有提到的老师、同学、朋友和家人，祝您们未来事业顺利！